Fundamentals of Information Systems
for Information Sciences

Second edition

Cinzia Cappiello, Mariagrazia Fugini, Paul Grefen,
Barbara Pernici, Pierluigi Plebani, Monica Vitali

October 13, 2025

DISCLAIMER This book is an English translation of the original and official Italian version. The translation has been carried out using an automatic tool. While every effort has been made to ensure accuracy, there may be errors, inaccuracies, or misinterpretations in the translated text. For authoritative reference, please consult the original Italian edition.

This work is released under the terms of the Creative Commons Attribution-ShareAlike 3.0 Italy license, the text of which is available on the Internet page
`http:creativecommons.org/licenses/by-sa/3.0/it/`

Contents

I Definitions and Characteristics of an IS **1**

1 Introduction **3**
1.1 Data and Information 4
1.2 Organizations 6
1.2.1 Resources 6
1.2.2 Processes 8
1.3 Definition and Role of Information System 10
1.4 Classification of Information Systems 11
1.4.1 OLTP and OLAP Systems 14
1.5 Organization, IT, and Components of an IS 15
1.6 Presentation of the Book Structure 18
1.7 Questions 19

2 Enterprise Architecture **21**
2.1 Enterprise Architecture Framework 21

II Enterprise Modelling **27**

3 Modeling with ArchiMate **29**
3.1 Introduction to ArchiMate 29
3.2 Main Elements of the Model 32
3.2.1 Components 32
3.2.2 Relationships 32
3.2.3 Business layer 33
3.2.4 Application layer 34
3.2.5 Technology layer 36
3.3 Examples 39
3.3.1 Example of representing a document composition system . . 39
3.3.2 Example of an insurance service 39

4 Process Modelling **43**
4.1 Process Modelling in BPMN 43
4.1.1 Definitions 43

4.1.2 Business Process Modeling Notation 44
4.2 Process Identification 47
4.2.1 Sectoral Checklists 49
4.2.2 Porter's Value Chain 50
4.2.3 Analytical Approach 51

III Technological Aspects 53

5 Introduction to Technological Elements 55
5.1 Introduction 55
5.1.1 Application-level technology 55
5.1.2 Platform-level technology 57
5.1.3 Physical architecture-level technology 58
5.2 Application and Technological Components in ArchiMate 59

6 Application Layer: ERP and CRM 61
6.1 Evolution of Application Components 61
6.2 Functional Components 62
6.3 ERP 63
6.3.1 History of ERP 64
6.3.2 Properties of ERPs 65
6.3.3 ERP Functionalities 66
6.3.4 Architectural Aspects 67
6.3.5 ERP Implementation 67
6.3.6 ERP Benefits 69
6.4 CRM 69
6.4.1 Operational CRM 72
6.4.2 Analytical CRM 75
6.4.3 Collaborative CRM 75
6.5 Questions 75

7 Application Layer: Data Analysis 77
7.1 Data Warehousing 77
7.1.1 Data Characteristics 78
7.1.2 Data Warehouse 80
7.1.3 Data Warehouse Architecture 81
7.1.4 Conceptual Model of the Data Warehouse 83
7.1.5 Logical Models of the Data Warehouse 84
7.1.6 Data Warehouse Operations 87
7.1.7 Life Cycle of the Data Warehouse 89
7.2 Data Mining 90
7.2.1 Characterization and Discrimination 91
7.2.2 Association Rules 92
7.2.3 Classificazione 93
7.2.4 Clustering 96

7.2.5 Examples on CRM . . . 98
7.3 Process mining . . . 99
7.3.1 Event Logs . . . 100
7.3.2 Process Discovery . . . 101
7.3.3 Conformance Checking . . . 102
7.3.4 Process Improvement . . . 102
7.4 Questions . . . 103

8 Platform-Level Technologies 105
8.1 Integration Architectures . . . 105
8.2 Application levels and integration techniques . . . 107
8.3 Application Integration Models . . . 110
8.3.1 Point-to-Point Integration Architecture . . . 110
8.3.2 Hub-and-Spoke Integration Architecture . . . 111
8.3.3 Integration with Business Process Management System . . . 112
8.3.4 Service-based Models . . . 113
8.3.5 Service Composition and Orchestration . . . 116
8.4 Questions . . . 117

9 Technologies at the Physical Architecture Level 119
9.1 Description . . . 119
9.1.1 Evolution of Physical Architectures . . . 119
9.2 Distributed Applications . . . 121
9.3 Scalability . . . 126
9.3.1 Server Farm . . . 127
9.4 Virtualization . . . 129
9.5 Cloud Computing . . . 132
9.5.1 Characteristics of Cloud Computing . . . 132
9.5.2 Service Models . . . 133
9.5.3 Deployment Models . . . 133
9.6 Questions . . . 134

10 Information Systems Security 135
10.1 Security Properties . . . 135
10.2 Threats, Breaches, Vulnerabilities, Attacks . . . 136
10.3 Cryptography for Data Protection . . . 139
10.3.1 Symmetric Cryptography . . . 140
10.3.2 Asymmetric Cryptography . . . 142
10.3.3 Integrity and hash function . . . 144
10.3.4 Digital signature . . . 145
10.3.5 Aspects of key management and digital certificates . . . 147
10.4 User management and access control . . . 149
10.4.1 Authentication and authorization . . . 149
10.4.2 Two- or multi-factor authentication . . . 150
10.4.3 Data access control systems . . . 151
10.5 Infrastructure Security Mechanisms . . . 160

10.5.1 Firewall 160
10.5.2 Intrusion Detection Systems - IDS 164
10.6 Elements of Cybersecurity 167
10.6.1 Attacks, Victims, and Countermeasures 167
10.6.2 Zero Trust 168
10.7 Evolution 170
10.8 Questions 172

IV Information System Design 173

11 The Design of Information Systems 175
11.1 Information System Management Process 175
11.2 Planning 177
11.2.1 Strategic planning 177
11.2.2 Feasibility study 181
11.2.3 Make-or-Buy Analysis 184
11.2.4 Analysis of sourcing strategies 184
11.3 Information System Development Life Cycle 188
11.3.1 Development cycle with Make strategy 188
11.3.2 Development cycle with Buy strategy 189
11.4 Questions 193

Part I

Definitions and Characteristics of an IS

Chapter 1

Introduction

In today's corporate, organizational, and social realities—characterized by a vast and ever-increasing amount of data and information—there is a growing interdependence between the company's information system (the system that manages information and processes) and the company's ability to operate. Changes in business strategy, rules, and processes (Business Processes) increasingly require adjustments in hardware, software, databases, communication networks, and user interfaces. Often, the way an organization operates and presents itself on the international stage depends on the functionalities provided by its information system. The Internet and the World Wide Web (referred to simply as the Web) have become one of the foundations on which organizations build their information systems to develop an information-driven economy. Among the various methods to address global economic challenges, one approach involves acquiring increasingly relevant information for the organization and leveraging existing data assets. Platforms and applications available on the Web serve these needs well, as they allow for the collection, management, and distribution of large volumes of information in various formats, originating from heterogeneous sources. Often, such sources are conceptually and physically distributed, as in multinational companies, public administrations, and industrial, commercial, financial, and research environments. In these contexts, immediate and effective access to critical data—extracted from large data volumes—becomes both strategic and essential. Mergers, acquisitions, and partnerships are additional strong drivers for integrating both information and the processes that manage it.

In this context, information systems can be defined as tools supported by Information Technology (IT), composed of software applications, data management systems, user interfaces (mostly Web-based today), and communication networks that assist organizations in managing information. Information systems involve various complexities in design, control, and management. These complexities arise from multiple information sources, heterogeneous databases with different models and technologies, communication networks based on diverse protocols, and from the presence of distributed and interconnected business processes and actors across different organizational entities. Furthermore, the information system is closely tied to business rules, organizational structure, and enterprise actors, and as such, must

be designed and managed to align technologies with organizational goals.

To handle these complexities, success today lies in designing and building distributed information systems—systems consisting of applications and data distributed across different computing nodes. These systems enable multiple organizations to work synergistically in managing applications and networked information in an interconnected yet semi-autonomous way. This vision contrasts with centralized information systems, often mainframe-based, which until a few decades ago represented the primary solution and, despite their obsolescence, are still part of many organizations' IT infrastructures—thus making synergy with external entities difficult. Designing and building an information system is therefore a complex endeavor that must both address future challenges—such as the explosion of available data (Big Data) and the emergence of vast, low-cost computational power (Cloud Computing)—and enable integration among heterogeneous systems.

Managing the resulting complexity is no easy task. For this reason, methodologies, techniques, and tools have been proposed to support the design and development of an information system capable of equipping an organization that aims to operate "in the network" with appropriate tools for governing both internal organizational processes and those that require interaction with other organizations.

1.1 Data and Information

Before delving into the definition of an information system and its components, it is useful to clarify the meaning of certain terms that are often considered synonymous but in fact have specific and strong meanings within the context of information systems. The terms "data" and "information" are mistakenly regarded as synonyms. To illustrate the differences, we will refer to the so-called knowledge pyramid, also known as the DIKW pyramid (Data Information Knowledge Wisdom), shown in Figure 1.1.

As you can see, data forms the base of everything. The word *data* should be interpreted as the past participle of the verb "to give." By "data," we mean a fact or a measurement—an element that models or describes a portion of the reality we wish to represent. Every piece of data is associated with a type that specifies the domain of values it may take. For example, if we want to represent the number of students in a class, the data type must necessarily be numeric—in particular, a natural number—since it would not be correct to represent this part of reality using negative numbers or decimal numbers. Besides the type, a data element may also have a unit of measurement. For instance, when representing classroom temperature, in addition to associating a numeric type—a real number in this case—we must also indicate the scale of measurement (e.g., degrees Celsius or Fahrenheit).

Because data can be numerous, the discipline of *databases* defines methods and tools for organizing them within more or less complex and flexible structures that make it easier and more efficient to store and retrieve them. The purpose of a database is to represent a portion of reality through a set of stored data that reflects it. This goal is supported by the conceptual model, the first design model

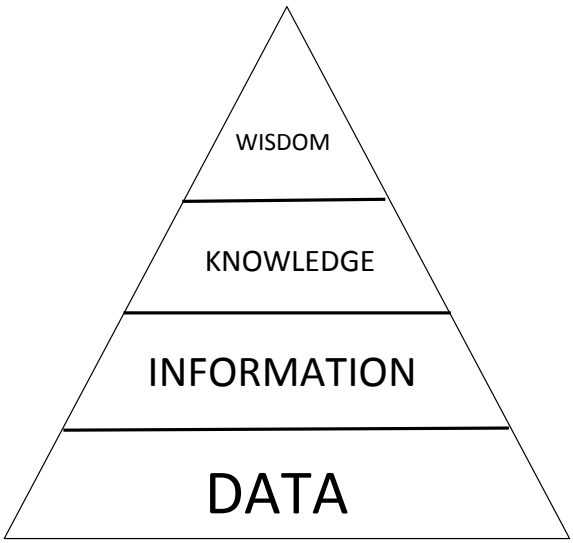


Figure 1.1: Knowledge pyramid (source: Wikipedia)

used in database design, whose purpose is to represent reality at a high level so that it can later be codified into an information system (i.e., the collection of hardware, software, and telecommunications components—also called an IT system—that supports the functioning of the information system; we will explore these definitions further in the book).

Following the structure of the knowledge pyramid, above data is placed *information*. As mentioned earlier, these two terms are often used interchangeably. However, information is built from data and can be defined as the interpretation of a single data item or a set of data. In fact, a piece of data without any reference context is often useless. For example, a temperature reading is meaningless unless we know whether it refers to an indoor setting—such as a classroom—or an outdoor one. Therefore, for data to be useful, it must be combined with other data to define a context that allows for better characterization of the represented reality. For instance, it is useful to associate the number of students with the academic year to which it refers, or to associate the temperature with the date and the classroom where the measurement took place. Information can thus be defined as the output of queries on a dataset (presumably organized in a database), such as "What is the temperature in the main lecture hall today?" or "What is the average number of students enrolled in mechanical engineering over the past five years?"

Climbing further up the pyramid, above information lies *knowledge*, which is obtained by integrating information with experience. Knowing that the main lecture hall is 32 degrees in mid-December suggests something is wrong, since that value deviates from the typical temperatures usually observed during that period. While information allows for a complete and useful representation of reality, knowledge guides decisions that may affect the current situation. It's worth emphasizing that

knowledge is acquired over time through experience and is generally not transferable between individuals. On the other hand, information—and to some extent, experience—can be transferred, though formalizing the latter remains challenging. For example, how should one define the historical range for comparing temperature data? Should it include just the same day? A week? A month?

Lastly, at the top of the pyramid is *wisdom*, which extends the classic knowledge pyramid (whose name suggests that knowledge is at the top) and can be defined as experience applied to knowledge, helping a person take the most appropriate action in a given situation. Returning to our example, realizing—by comparing the current temperature in the lecture hall with the historical series—that an anomaly exists may lead someone to turn off the heating system, as this action affects the temperature.

The choice of a pyramid as a geometric shape for representing these concepts is no coincidence. First, it provides a layered structure, showing how higher levels are built on top of lower ones. Second, the area of each level decreases as you go up. This represents the level of *abstraction*: at the base are data, which are numerous and fine-grained, while at the top is wisdom, consisting of fewer, highly abstracted elements with coarse granularity.

From the perspective of data management in an IT system (i.e., the IT platform), the ability to automatically manage the contents of the knowledge pyramid decreases as one ascends. While data is managed through database systems (with high automation), information can be abstracted and described to support decisions using automated or semi-automated tools, while knowledge can only be partially supported through automation.

1.2 Organizations

The use of information systems requires an understanding of the organization, its management, and the role of IT in the company. In fact, an information system creates value for the company by offering organizational and managerial solutions to the challenges posed by the environment in which it operates. Without delving into topics such as *organizational theory* or *business organization*, for which we refer to specific textbooks, in the context of information systems it is sufficient to focus on the concepts of *resources* and *processes*.

1.2.1 Resources

A **resource** is everything an organization uses—both tangible and intangible—to pursue its objectives. Among the various classifications, one of the most well-known is by Ricciardi, cited in [6], in which resources are categorized as:

- **External Resources**
  - Social and economic environment
  - Market

  - Customers

- **Internal Resources**

  - Exchange resources: products (goods or services)
  - Structural resources: financial tools, people (human resources), infrastructure
  - Management resources: rules, organizational charts, delegations, plans, information

Each resource has its own life cycle, generally including the phases of Planning, Acquisition, Management, and Maintenance. Examples of resources include personnel, money, machinery, and of course data and information, which represent the foundational element of an information system.

Information is used for communication, process support, and decision-making support, which requires information about available strategic or control alternatives. For some companies, information can also be a product. Information has the following characteristics that distinguish it from any other company resource: it is intangible, non-perishable (it is not destroyed through use), and self-regenerating (its use generates new information). For example, in mobile communications, information is used to gain insight about users, propose new pricing plans, collect feedback on marketing initiatives, and update rates. On a production line, it is necessary to store information about completed tasks (e.g., time required, percentage of defective items), identify problems, or improve process execution. Managing information (which, as we will see, is the goal of an information system) therefore translates into numerous activities such as:

- Creating information
- Acquiring information
- Processing information
- Storing information
- Transmitting information
- Presenting information

These activities do not necessarily require IT tools. Information can be managed implicitly—based on the experience and skills of individuals—when dealing with activities that are hard to replicate, or explicitly but not supported by IT (manual management). Alternatively, it can be managed explicitly and supported by IT, in a way that enables efficient and easily replicable organization and retrieval of information.

Figure 1.2: Anthony's Pyramid Model

1.2.2 Processes

A **process** is the set of activities that an organization performs to manage the life cycle of a resource, or a homogeneous group of resources, to achieve a defined and measurable result (product/service). Processes can be classified according to various models:

- Anthony's pyramid
- Porter's value chain

Classification of processes according to Anthony's pyramid. To classify processes and applications, one can refer to the hierarchical organizational model known as *Anthony's Pyramid* [1], shown in Figure 1.2. Anthony's model distinguishes three levels related to the organization's structure: an *operational level*, which includes day-to-day activities; a *programming and control level* (also called middle or tactical management), which includes resource planning and monitoring goal achievement; and a *strategic planning level*, where the organization's goals and policies are defined.

Examples of processes in a public administration, such as a city council, are the following (organized by level):

- *Operational*: recording citizen payments, road maintenance interventions
- *Control*: payment audits, reminders, monthly comparisons between forecasted and actual revenues, pollution monitoring
- *Strategic*: analysis of costs and revenues for social services, definition of new tariffs, urban planning

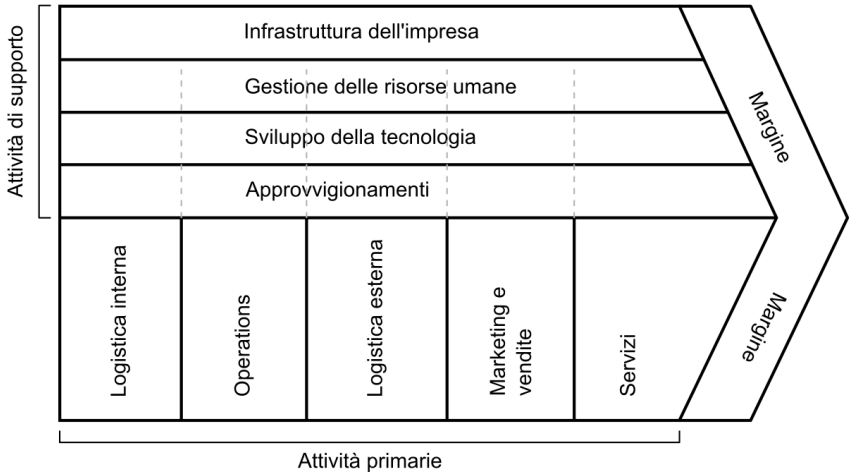


Figure 1.3: Porter's Value Chain Model

In a bank, examples of processes across the three levels include:

- *Operational*: managing current account transactions, ATM withdrawals
- *Control*: reviewing overdrafts, approving mortgage loans
- *Strategic*: evaluating service performance, deciding whether to launch new services

In a company, typical processes include:

- *Operational*: recording job cost data, warehouse management
- *Control*: weekly analysis of budget variances
- *Strategic*: selecting the most profitable market segments

Classification of processes according to Porter's model. A second widely used classification model in literature is based on the *value chain* proposed by Porter [21], shown in Figure 1.3, which considers only operational-level processes. The value chain model views processes and related *functions* as tools that organizations use to deliver value through their products and services. Although originally designed for manufacturing firms, its principles apply to many other types of organizations.

According to this classification, functions are divided into *primary activities*[1], which are directly tied to an organization's core goals and typically define the business sector, and *support activities*, which ensure that the primary activities function

[1]Note that in this case, the terms activity and process are used as synonyms. However, as will be discussed later, these terms have distinct meanings.

correctly. Each activity consumes and produces resources, and effective execution of these activities positively affects profit.

Primary activities are shown as vertical columns at the bottom of Figure 1.3. The *inbound logistics* activity enables the transportation of materials used as production inputs. *Operations* involve the transformation of these inputs into finished products. *Outbound logistics* handles delivery of finished goods to customers. *Marketing and sales* aim to attract customers and manage the sales process, while *services* provide post-sale customer support.

Support activities are shown as horizontal layers at the top of Figure 1.3. The *procurement* function handles purchasing goods and services needed for other functions (especially operations). The *technology development* function focuses on developing new technologies used in production or in the products themselves. *Human Resource Management* (HR) oversees staffing. Finally, the *infrastructure* function provides general organizational support, such as finance and administration.

1.3 Definition and Role of Information System

After clarifying what information is and how it is fundamental to an organization, we can precisely define what an information system is. To do so, we refer to the definition in [20], which starts from the idea that an information system is, first and foremost, a *system* (see Figure 1.4). It is thus a set of procedures, methods, and tools devoted to performing specific functions to achieve a given result. In the case of information systems, this system is fed by events—represented as data—that occur in an organization and transforms them into information. This information must be appropriately modeled and represented according to the needs of its users.

Events may include, in a manufacturing company, the receipt of an order, the delivery of a finished product, or the report of an out-of-stock item in the warehouse. These events can trigger organizational processes (e.g., starting the production of an order, notifying the courier for delivery, or sending a purchase order for restocking), which, as discussed previously, represent the foundation of an organization.

To support this transformation:

"an information system is defined as the set of resources, organizational knowledge, and technical skills used to manage the information resource."

Within an information system, databases allow for storing data that represent events of interest occurring within an organization. Business Process Management (BPM) methods and tools offer a set of techniques for creating synergy between organizational processes and the technology that supports them.

Orthogonally to these aspects, information system design aims to understand and model, starting from organizational needs, how data and processes can be supported by technology. As highlighted in the previous paragraph, an organization is made up of various processes and roles, each of which may require data and information at different levels of detail.

Therefore, data management may involve analytical tools such as data warehousing and data mining, which are more suitable for supporting tactical and strategic levels rather than operational levels.

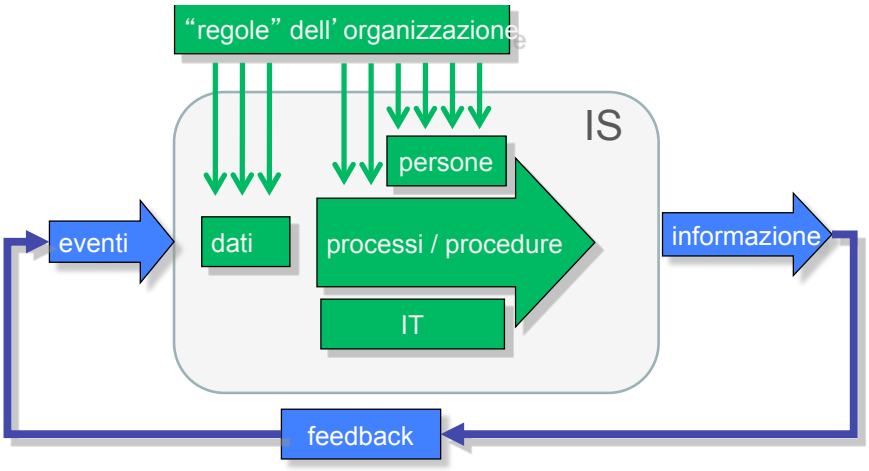


Figure 1.4: Representation of an information system (adapted from [20])

An information system is thus a complex entity that is often mistakenly associated only with its technological component—known as the *information technology system*. In particular, the *information technology (IT) system* is a component of the information system that processes, stores, and manages the exchange of information. The IT system evolves over time along with the company and its technologies.

Thus, the key elements of the information system involve managing the *information resource* according to business rules and goals using technology for one or more organizations. Information plays a fundamental role in any type of company, from manufacturing to service industries. It is not an accessory tool but a strategic asset, just like human, material, or financial resources.

Designing and developing information systems, therefore, includes the design of various aspects: data, processes, and user interaction. A development approach that separates these project areas (data, business logic, and user interface) is one way to manage project complexity effectively.

1.4 Classification of Information Systems

Given the complexity of the application domain of an information system (consider the various business processes, for example, in a manufacturing company), it is unrealistic to expect a single information system to cover all processes and manage all organizational resources. For this reason, there are different types of information systems (which can be integrated with each other) that focus on specific types of processes or are able to manage particular resources (for example, documents or data coming from sensors).

A first classification of these information systems can be made by referring to Anthony’s pyramid, which allows distinguishing between *operational* information systems and *decision support* information systems (also called Management Infor-

mation Systems, or by some authors, informational systems).

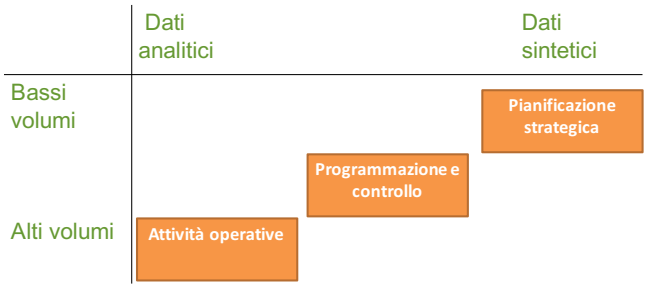
- The purpose of *operational systems* is to perform basic, everyday administrative operations, such as transaction management and office work, accounting, and, in general, the processing of what are called business situations [19]. The fundamental parts of an operational information system are the operational database (which contains the entire operational information in a structured form, i.e., at the operational level in Anthony's pyramid) and the operational functions.

- *Decision support or informational systems* are information systems that support business decision-making and strategic activities, responding to the need to exploit the data assets to identify useful information for the decision-making process. With the increase in available data, the demand for extracting strategic information has also increased. Information systems meet this need with tools that have evolved from original reporting tools or spreadsheet-like tools. Today, there are sophisticated data extraction and analysis functionalities for strategic purposes, so terms such as data warehouse—an informational database that collects all data of interest for the company—data warehousing—the set of activities that define, build, and maintain the structure of the data warehouse—and Decision Support Systems (DSS)—tools that support information extraction for decision-making processes such as OLAP (OnLine Analytical Processing) browsers (see Section 1.4.1)—are widely used. More recently, the term Business Intelligence has been adopted to generally indicate functionalities of this kind.

Anthony's pyramid also helps to understand the nature of the data managed by different information systems. Indeed, data themselves can be classified at operational/control/strategic levels. Examples of operational data are amounts of deposits or employee attendance hours; control-level data examples include monthly balances and the monthly workload of each department; strategic data include macroeconomic data, general indicators, or budget data.

As shown in Figure 1.5, operational information systems handle high volumes of data that detail the events occurring within the organization. Such data are well-structured (usually stored in databases) and come from internal sources. Moving up to managerial information systems, data become increasingly aggregated, since summary information is a necessary requirement for decision-making at these levels. At the same time, data formats may no longer be well-structured (there can also be documents written in natural language), and their sources can be both internal and external.

It is worth noting that the degree of information structuring impacts the level of automation in decision-making. As shown in Figure 1.6, when an information system deals with well-structured data and the decision process is well-defined (e.g., there is an algorithm), then the system itself can autonomously make decisions (for example, automatic reorder of inventory). However, when the decision process cannot be formally represented and the human factor still plays a role, the information system becomes a support tool for the decision-maker. This means that the closure



Profilo informativo	Livello di controllo		
	Controllo operativo	Controllo direzionale	Pianificazione strategica
Formato	Dati elementari	Sintesi	Sintesi
Struttura	Predefinita	Predefinita	Variabile
Frequenza	Continua	Periodica	Non prefissata
Fonte	Interna	Interna ed esterna	Prev. esterna

Figure 1.5: Information profile of information systems [6]

of the loop is represented by the human user. Finally, regardless of the nature of the decision process, having poorly structured data limits the impact of information systems.

Regarding this point, it is worth noting that in recent years there has been a strong push toward adopting Artificial Intelligence (AI) techniques within information systems. This has enabled the development of systems capable of making decisions even when data are unstructured and processes are not well defined, as AI aims to replace humans in decision-making.

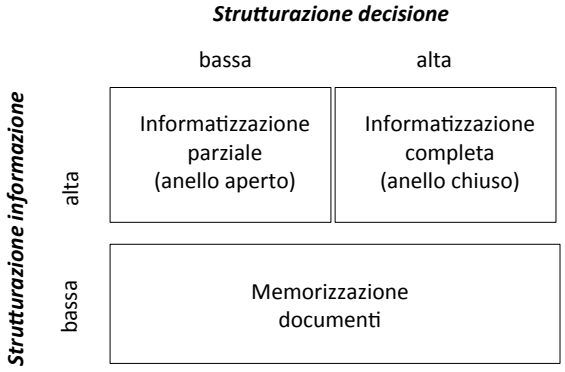


Figure 1.6: Decision profile of information systems [6]

1.4.1 OLTP and OLAP Systems

Information systems can be succinctly defined as a set of applications that interact with databases. The functions either record events and/or query the database that stores the system's permanent information.

The term *transaction* in an information system refers to a broad category of concepts including:

- Exchanges or contracts between the company and the external environment or among units of the same company with a specific objective (for example, receiving customer orders, sending orders to suppliers, etc.).
- Transformations, i.e., operational activities that contribute to the production of goods and/or the delivery of services.
- Movement of physical objects within the company (for example, assembling an electronic board) or between the company and the outside world (for example, shipping a part from the warehouse to the customer).
- Certification of events, such as the introduction of a new product to the catalog or payment of an invoice.
- At the database level, a set of elementary operations that modify the state of the database, leaving the data in a consistent state (for example, executing a bank transfer, which includes several reads and writes on banking databases that must ultimately complete successfully with a status of 'executed' or 'not executed'). In the context of database management systems (DBMS), these types of transactions are defined as ACID transactions (Atomic, Consistent, Isolated, Durable).

Transactions are often certified by the creation of a document, paper or electronic, that testifies to their execution.

OLTP systems (On-line Transaction Processing) are those that handle operations characterized by a large number of short, online transactions[2] (using INSERT, UPDATE, DELETE operations). OLTP systems focus on very fast query handling, maintaining data integrity in multi-access environments, and ensuring system efficiency and effectiveness, for example, by measuring efficiency based on the number of transactions per second (throughput).

In OLTP data management systems, detailed and current data are stored. The schema used to maintain transactional databases is usually a relational model (generally in Third Normal Form—3NF). Queries access individual records (for example, as in update mode access to an enterprise email database). OLTP systems are suitable for managing operational and control-level processes.

OLAP data management systems (On-line Analytical Processing), on the other hand, deal with historical (or archived) data. The OLAP paradigm is characterized

[2]Note that the term "online" here does not indicate a transaction visible on the Web, but a transaction whose effects are immediately visible on the system. This term highlights the evolution compared to previously used batch systems, which made the transaction effects visible only after updates performed at regular intervals (usually overnight).

by few transactions and complex queries that require data aggregation. OLAP systems measure efficiency and effectiveness by response time. OLAP applications are mainly used in Data Warehousing techniques. An OLAP system stores data in aggregated format, maintains historical data archived according to multidimensional schemas (usually star schemas). Queries often access large amounts of data to answer complex questions such as: "What was the company's net profit in a certain geographic area last year?"

OLAP systems are used for data processing oriented toward decision support and are therefore suitable for functions placed at the planning and strategic levels of Anthony's pyramid. They enable complex and ad-hoc operations, where each operation can involve many aggregated, historical, and generally non-current data, and for which ACID properties are not relevant because the operations are read-only.

Table 1.1 shows a comparison between OLTP and OLAP systems based on the criteria described above.

	OLTP	**OLAP**
User	Employee (many)	Manager (few)
Function	Daily operations	Decision support
Design	Application-oriented	Subject-oriented
Data	Current, updated, detailed, relational, homogeneous	Historical, aggregated, multi-dimensional, heterogeneous
Usage	Repetitive	Occasional
Access	Read and write, indexed	Read, sequential
Work Unit	Short transaction	Complex query
Metric	Throughput	Response time

Table 1.1: OLTP and OLAP

1.5 Organization, IT, and Components of an IS

There is a bidirectional relationship between the organization and IT, as shown in Figure 1.7. This complex bidirectional relationship is mediated by various factors, among which are, not least, the decisions made — or not made — by managers. Other mediating factors include organizational culture, social and political structure, business processes, and the environment.

An important aspect of this relationship between technological factors is that decisions made at the strategic or control level can impact technological choices (*requirements pull*), whereas in other cases, changes in business objectives, organizational choices, and activities to be performed may be due to new technologies that offer new opportunities in the company's business sector or for its expansion (*technology push*).

Anthony's pyramid is also a reference for considering alignment aspects between organizational structure and the information system. As shown in Figure 1.8, the

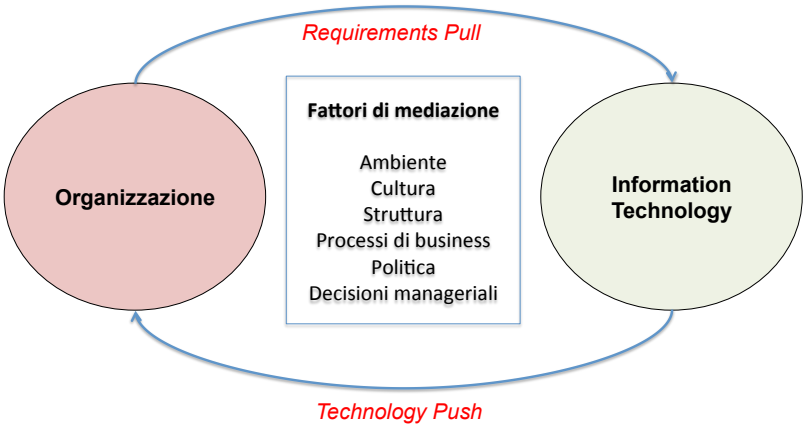


Figure 1.7: Interaction between organization and IT (adapted from [18])

hierarchical structure allows representing control flows between the operational level and the control level. Here we see that the operational level, if a problem arises in an organizational unit (or in a process, if one wants to consider the process carried out by an operational unit and the involved people/users), generates an exception that alters the normal operational flow. To handle the exception, information may be requested from other processes/people (remaining at the operational level) or the intervention of a higher hierarchical level may be requested, moving to Anthony's control level. It is important to reach the point where the correct rights and competencies reside to resolve the exceptions. This generates globally, also through the delegation of control, various information flows related to the vertical organizational structure (exceptions) and horizontal (need for information from other processes/people).

The relationship between the various organizational needs is a complex one. The information system, as illustrated in Figure 1.4, is a set of components and areas that are supported/cooperated by using a computer system: the IT technology used by the company (hardware, software, network infrastructures) to manage information (data), people, procedures, and the organization. The design, development, and management of information systems also include non-technical disciplines that deal with IT acceptance by users (psychology and social sciences), the business value of IT, and business strategy. Therefore, it is an interdisciplinary discipline.

In Part II of the book, the relationships between aspects related to business objectives (business aspects), organizational aspects, and technological aspects are illustrated in more detail. A fundamental aspect for maintaining coherence among the various components of the system is the definition of an overall architecture (*Enterprise Architecture - EA*) at the base of the company's information system(s). The concept of EA is illustrated in more detail in Section 2.1, and the models for its representation will be illustrated in Part II of this text.

Building an information system based on IT can be a difficult, long, complex,

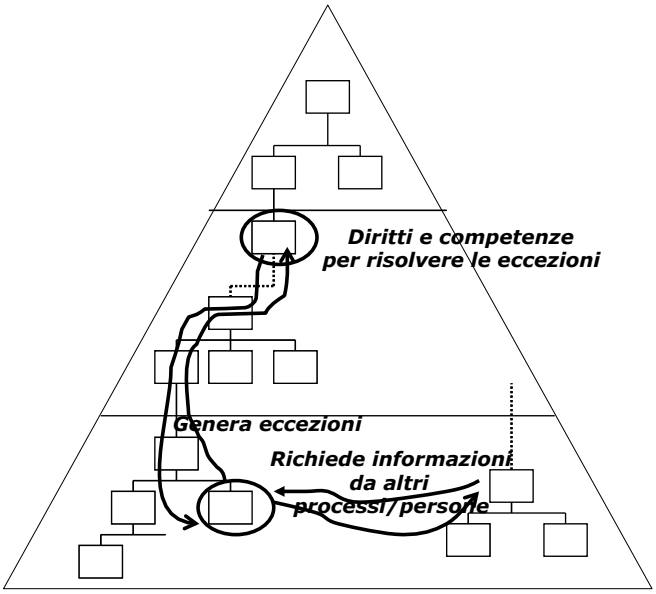


Figure 1.8: Information flows and organizational structure

and costly process. Designing an IT-based information system means thinking about its architecture, the business process model, the user navigation interface, for example based on the Web, its management and control by management, its interaction with tools such as social networks and mobility tools for collecting and disseminating information, and so on.

The essential components of an IT-based information system are on one hand the database (data storage component), and on the other hand the applications that enable business processes and the user interface. The design and development of information systems therefore includes the design of various aspects: data, processes, user interaction. Development according to this separation into areas of project interest (data, business logic, and user interface) is one of the ways to manage the complexity of the project.

Using well-formalized design methodologies that compartmentalize the various aspects can be very useful, separating the various layers (logical levels) for data, business and services (details will be provided later), presentation (user interface), and the technological infrastructure. Interactions with external systems must also be designed (for example, a supplier's information system or the banking system for payments), with various data sources (for example, from a monitoring system on the production chain), with various services (data provision, for example from a weather service). Finally, there are systems common to various levels (called Cross-Cutting components or transversal), such as security, operational management, and communication.

One can also consider the physical components, such as servers, databases, In-

tranet and Extranet areas, protected zones (Firewall) or demilitarized zones (neutral areas where the Web server is generally located), server farm configurations, and use of Cloud services.

As already mentioned, the computer system is a component of the information system, that is what has been identified as *IT*, which processes, stores, and manages the exchange of information. The computer system (IT) changes over time with the evolution of the company and technologies.

Information management activities do not all necessarily require IT tools, because information can also be managed through paper archives or informal documents. Moreover, business processes can have automated parts, fully or partially, and parts performed manually by operators, especially regarding decision-making processes. One of the challenges in designing and managing information systems is to maintain *alignment* between business processes and data and those represented in the information system. As illustrated in Figure 1.7, in the complex relationship between organization and IT supporting business activities, there are several mediating factors, also environmental and cultural, which may be the source of choices that must remain aligned over time, whether the change has organizational or external origins (*requirements pull*) or whether the change is based on the availability of new technologies (*technology push*).

This text will particularly study the techniques, methods, and models that allow designing and maintaining alignment between processes and data management in IT-based IS.

1.6 Presentation of the Book Structure

This book is structured into four parts.

In the first part, which includes this introductory chapter, the necessary concepts for understanding an information system are introduced. In particular, in Chapter 2.1 the centrality of the concept of Enterprise Architecture as the basis for designing and evolving over time the corporate information system is illustrated.

In Part II, Chapter 3 presents the ArchiMate model and notation, especially the elements that allow representing the technological infrastructure and its components through ArchiMate. Chapter 4.1 introduces the BPMN notation (Business Process Modeling Notation) to describe processes in detail.

In Part III of the book, the main technological components of an information system are illustrated. At the application level, ERP and CRM systems are illustrated, mainly used at the operational level, and subsequently the types of components used at the decision-making level, such as data warehouses and data mining. Regarding the platform level, the book focuses on components that enable interaction between information systems and their components, discussing integration architectures and platforms and service models. Regarding the physical architecture, different types of system architectures are introduced, discussing their configuration and virtualization and characteristics of systems based on Cloud computing. The chapter concludes by discussing different options for managing physical architectures. A very important aspect of information systems is security, to which a chapter is

dedicated. Security properties of systems are presented, the use of cryptography to guarantee confidentiality and authenticity of information, and the main mechanisms for data access control. Finally, the main infrastructural security mechanisms are presented.

In Part IV of the book, a chapter illustrates the main phases of designing an information system. The chapter also illustrates the main design phases, focusing on the main choices made in planning information system projects.

1.7 Questions

- Illustrate how Anthony's Pyramid model is used to characterize types of information systems and their relationships.
- Compare the elements represented by Anthony's Pyramid model and Porter's model.
- Illustrate what is meant by Resource in an IS.
- Illustrate the differences between OLTP and OLAP.
- Identify some processes and data at the operational, control, and strategic levels in a supermarket chain.
- Illustrate models for segmenting information systems.

Chapter 2

Enterprise Architecture

2.1 Enterprise Architecture Framework

Initially, information systems were simple and essentially closed systems, meaning they were not connected with systems of other companies or organizations. These systems supported a limited number of enterprise-level functionalities, such as production or accounting, and were organized in silos: consisting of a series of independent applications specialized in supporting and/or executing a specific functionality. Over time, the complexity of information systems has increased. Indeed, these systems are composed of several integrated applications aimed at supporting both internal processes within the company (intra-company) and processes that involve collaboration with other companies (inter-company). Moreover, a company's information system is often the result of incremental and ongoing growth. In fact, the information system must adapt both to business-driven changes, which may involve changing strategies or needs, and to changes related to technological evolution.

Therefore, as introduced in Chapter 1 and illustrated in Figure 1.7, a project related to the evolution of an information system can derive from technological pushes (*technology push*) and business pulls (*requirements pull*) which need to be reconciled and integrated for the organic development of the information system (Figure 2.1).

Business pulls may derive from the need for new functionalities or the necessity to improve the quality of the functionalities offered. Technological pushes, on the other hand, may be caused by necessary technological evolution, which often requires an adjustment of the information system's structure. The two drivers of change (*technology push*) and (*requirements pull*) can influence each other: the request for new functionalities necessarily leads to technological evolution, while technological evolution often enables the possibility of delivering new services and functionalities.

To keep the relationships between the business side and the technological side (IT in Figure 1.7) manageable, it is necessary to design structures of an abstract nature, able to describe correspondences between the two aspects independently of specific

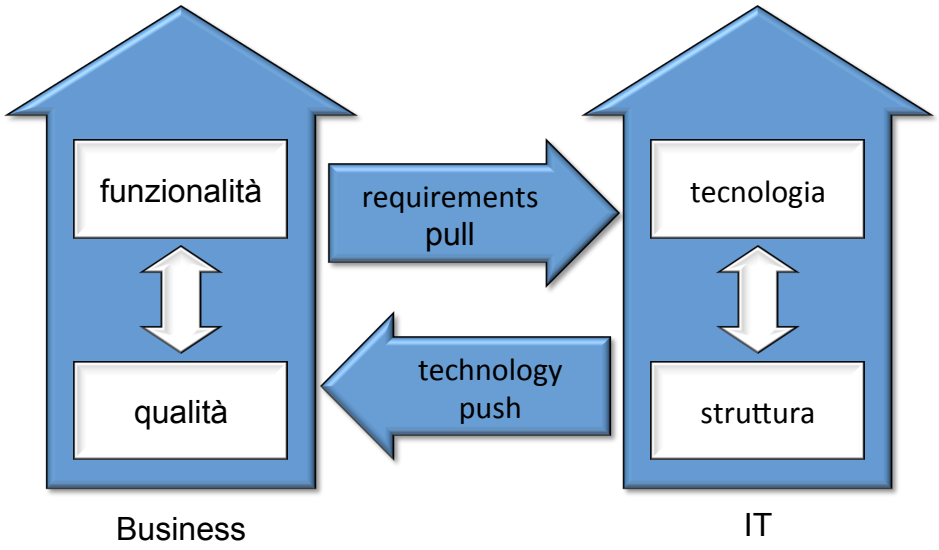


Figure 2.1: Elements motivating an Enterprise Architecture

technological or business choices. This independence guarantees the stability of the structures over time.

To define the structure of the information system, the concept of *Architecture* is used. As defined in the ISO/IEC 42010 standard [16], the architecture of a system consists of the set of components, the relationships among components, and the principles that define their design and evolution over time. The architecture provides three different views: (i) *static view*, related to the description of the components, (ii) *dynamic view*, related to the description of the relationships between components, and (iii) *rules view*, related to the procedures that support the design and evolution of the system over time. The architecture also provides different perspectives to the stakeholders of the system. Stakeholders are defined as those who have an interest in the considered system, for example, throughout the entire lifecycle of the system, because they are owners, users, customers, suppliers, financiers, or end users. Generally, they may need to analyze the architecture from different points of view. Consider, for example, the architecture of a city composed of different components such as buildings, roads, parks, etc. The officials of the city's municipal offices may need an aerial view (i.e., a global perspective) of the city to see the architecture as a whole in order to plan development and growth interventions for the urban area. A local perspective, focused on the surroundings of a single building, might instead be useful to the owner of that building to analyze the possibility of expanding the building and requesting renovation permits. These two stakeholders are interested in the city's architecture for different objectives and thus need to analyze the architecture from two different perspectives. The definition of an architecture must allow and satisfy the different needs of stakeholders by

providing the possibility to analyze the system from different viewpoints and with different levels of granularity. This is why architectural frameworks are used, which establish conventions, principles, and methods to describe architectures in different application domains and for different stakeholders.

If we try to apply this general concept of architecture to an organization and its information system, we arrive at the definition of Enterprise Architecture. In [5] the *Enterprise Architecture* (EA) is defined as the means to analyze and describe an organization in its current and future state starting from an integrated perspective of strategy, business, and technology. EA indeed provides an overview (map) of the processes, systems, technology, and capabilities of an organization. This map provides an operational and strategic context capable of supporting the continuous evolution of the information system driven by business or technological pushes. EA can be seen as a high-level blueprint of the system, useful for understanding its internal structure, as support for its design, redesign, configuration, and maintenance.

This is not very different from the (technical) architecture of a complex building, a bridge, or a subway. The fact that an architecture is a high-level blueprint means the emphasis is on an abstract, conceptual representation, not on individual technical details. For this reason, EA can be compared to other types of architectures: the architecture of a complex building or a bridge, which is not specified in terms of individual columns, structures, or walls, but with conceptual drawings of the construction to be realized.

At the end of the 1990s, John Zachman considered the use of an architecture to describe the enterprise information system fundamental as technology began to enable the distribution of applications and data over different machines. In his opinion, decentralization without a well-defined structure would lead to chaos. Thus, he proposed the framework to define Enterprise Architecture [33, 25]: a taxonomy to organize documents, specifications, and existing models into a general schema and to show the relationships that exist between them.

The *Zachman framework* is organized in rows and columns (see Figure 2.2). The columns define the *Aspects* to analyze; specifically, the following six columns are defined:

- Data (what): considers the data the organization needs to operate.
- Functions (how): analyzes the functionalities performed by the company to conduct its core business.
- Network (where): relates to the geographic distribution of the organization.
- People (who): the people involved in performing the various business functions.
- Time (when): relates to significant business events.
- Motivation (why): considers the goals of the company.

Note that no ordering is defined between the columns in the Zachman framework.

	What Cosa? DATI	How Come? FUNZIONI	Where Dove? RETE	Who Chi? PERSONE	When Quando? TEMPO	Why Perché? MOTIVAZIONE
Contestuale AMBITO *Pianificatore*	Liste elementi importanti per l'impresa	Lista processi eseguiti dall'impresa	Lista località in cui opera l'azienda	Lista unità organizzative importanti per l'azienda	Lista eventi	Lista obiettivi, strategie
Concettuale Modello Impresa *Owner*	Modello Entità-Relazioe	Modello dei processi	Rete logistica	Modello della struttura organizzativa	Modello eventi	Relazioni tra obiettivi
Logico Modello sistema *Progettista*	Modello dei dati	Diagramma dei processi	Architettura dei sistema distribuito	Diagramma ruoli e relazioni	Diagramma eventi	Diagramma regole
Fisico Modello tecnologico *Costruttore*	Progettazione dei dati	Specifica delle funzioni	Architettura del sistema	Interfaccia uomo-macchina	Specifica eventi	Specifica regole
Dettaglio Componenti *Sottocontraente*	Es. Definizione dei dati	Es. Applicazione (codice)	Es. Architettura di rete	Es. Architettura sicurezza	Dettagli eventi	Dettagli regole

Figure 2.2: The Zachman framework for Enterprise Architecture (adapted from [33])

The rows, instead, highlight the different *Points of View* that may interest different stakeholders. Specifically, Zachman identifies the following five points of view:

- Scope (contextual): concerns those who must do planning and provides a high-level representation that describes the system in terms of dimensions, shape, relationships, and basic objectives.
- Business model (conceptual model): concerns the system owner and is a business design-oriented perspective.
- System model (logical model): intended for the designer, providing a detailed specification through the definition of the system model.
- Technology model (physical model): intended for those who must realize the system and translate the logical design into a physical design.
- Detailed representation: gathers the specifications to be given to programmers for the actual system implementation.

The last row in Figure 2.2 then represents the actual implementation of the system, which is part of the organization.

For each row, constraints on the system at the considered level are defined. These constraints are additive, meaning that constraints at a lower level are added to those at higher levels.

The cells of the Zachman framework created by the intersection of rows and columns define the *Views* (called *perspectives* by Zachman) that identify the models and diagrams able to provide information about a particular aspect from a certain point of view. For example, by following the data column, one can see how the scope is described through the list of important things to model in a particular context. The conceptual model of the database to be realized is provided by diagrams known in the literature (e.g., Entity-Relationship (ER) model). The conceptual design is then followed by logical design, which translates the diagram defined at the conceptual level into a data representation appropriate to the chosen technology. The actual realization of the database is preceded by a physical model and by the detailed definition of the data to be entered into the system. The implementation, finally, generates the organizational functionalities constituted by the specific database (for example, the Orders database). Each column, read from top to bottom, specifies how different diagrams and models are refined to move from business indications to the actual system implementation.

The Zachman framework is an effective tool to describe the organization's information system as a whole. Beyond the descriptive purpose, the description of EA can also be used prescriptively to guide the system's evolution.

In this text, we use the ArchiMate approach proposed in [13] for designing the system architecture as EA, in which the aspects to specify are the following:

- Business aspects, related to business goals.
- Application aspects, identifying the system's functionalities.

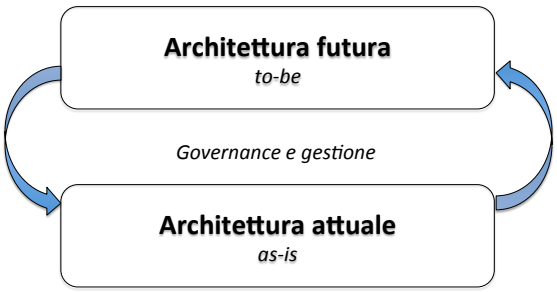


Figure 2.3: Enterprise Architecture management process

- Infrastructure aspects, outlining general technological choices (e.g., platforms for data exchange between modules or functionalities necessary to manage security aspects).

One aspect to underline at this point is that EA itself must be managed, particularly with reference to the continuous system evolution.

First of all, during a project, it is necessary to develop the future EA (*'to-be'*) relative to the current architecture (present architecture, or *'as-is'*). Furthermore, during the development of individual project phases, it will be necessary to ensure that the planned EA and the EA actually implemented remain aligned over time (Figure 2.3).

To achieve these two objectives, both governance activities (strategic management) of the system and management activities of the phases within individual projects are necessary.

The phases of designing the reference architecture and its central role in the system development phases are illustrated in the following.

Part II

Enterprise Modelling

Chapter 3

Modeling an Enterprise Architecture with ArchiMate

3.1 Introduction to ArchiMate

This chapter presents ArchiMate®, a graphical modeling language for Enterprise Architecture proposed by The Open Group[1] [13, 27, 17]. ArchiMate has a *layered* and *service-oriented* approach for architectural models.

As already discussed previously in the Zachman framework, in ArchiMate as well the description of the architecture is presented according to different viewpoints (layers), which allow representing a system with different diagrams connected to each other. The layers examined in this course correspond in part to the concepts of viewpoint and aspect in the Zachman framework.

The three layers considered are called in ArchiMate the Business Layer, Application Layer, and Technology Layer.

Since ArchiMate is a model for describing Enterprise Architectures, its notation and the way diagrams should be created are inspired by the principles of Enterprise Architecture. In particular, through the richness of symbols and the syntactic rules that characterize ArchiMate, the description of a given system can be offered through a series of consistent diagrams, each focused on describing the system according to the interest of one or more stakeholders. Regarding the scope of this course, the focus will be on three main stakeholders covered by the part of ArchiMate called ArchiMate Core. A business stakeholder who needs to know the nature of the service to be offered, to whom it must be offered, how, and which business functions are involved for that purpose. An application stakeholder, often a developer or system integrator, who needs to understand, among other things, which applications are involved, which must be developed, and what data must be stored. Finally, an infrastructure stakeholder who needs an overview of the platforms intended to install and deploy applications to provide business services.

[1] `http://www.opengroup.org/`

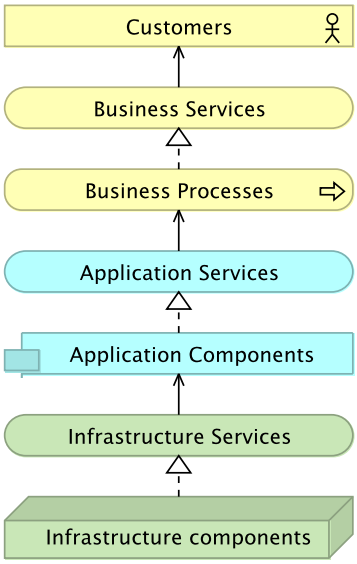


Figure 3.1: Services and ArchiMate (adapted from [17])

In this way, it is possible to design an information system in a comprehensive way according to the logic of the Zachman framework, focusing on different aspects of design while maintaining an overall consistency of the designed information system.

As mentioned, ArchiMate Core groups its graphical constructs into three macro categories: business, application, and technology elements (or components)[2]. These constructs are organized in layers. This is because ArchiMate[3] follows a *service-oriented* approach and, for this reason, places particular emphasis on the distinction between interface and implementation and on the visibility a service consumer has of the service as a whole. In a service-oriented logic, it is useful to distinguish what is visible to the service user (interface) from what is necessary to make the service usable (implementation). For example, consider a catering service: the interface consists of the dining room and waiters who allow ordering and receiving food; the implementation corresponds to the kitchen and its staff.

The adoption of the service paradigm in ArchiMate is illustrated in Figure 3.1. The figure shows how the layers alternate between the service itself (visible to its users) and the components that realize it. Therefore, looking at the colors, the business layer defines the service as seen by the end user. The end user interacts with the *business services* (which represent the interface), which are realized by *business processes*. Business processes in turn consume application services, which

[2]Although the ArchiMate notation is colorless (therefore colors have no formal meaning), it has become customary to associate yellow graphical elements to the business layer, blue to the application layer, and green to the technology layer.

[3]From now on, for simplicity, the generic term ArchiMate will mean ArchiMate Core: the part of ArchiMate covering the Business, Application, and Technology layers.

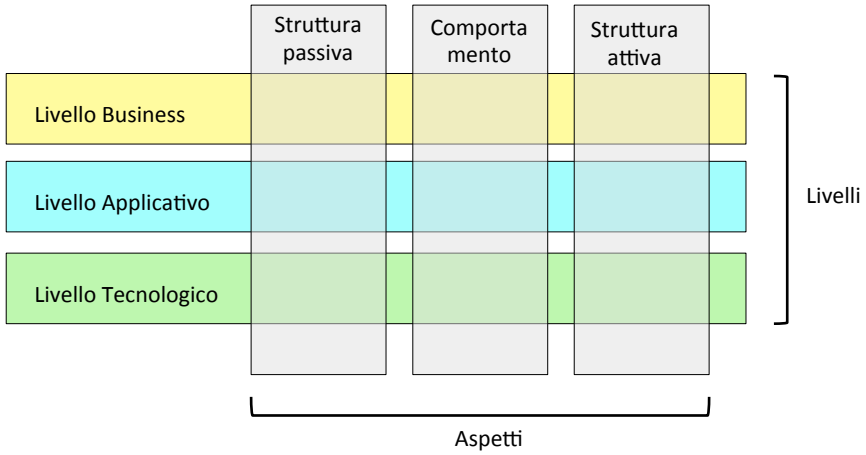


Figure 3.2: Aspects and layers of ArchiMate: core elements (source: The Open Group, 2016)

are realized by application components. Finally, application components — to be realized — rely on infrastructure services. The relationship between services and the components that realize them is called *realisation relation*, while the relationship between components and the services needed for their functioning is defined as *serving relation*. It should be emphasized that the layered architecture thus created assumes that the final consumer sees only the business service and has no awareness of what happens in the underlying layers.

Going more specifically into the notation, Figure 3.2 shows the classification method of ArchiMate Core constructs that will be presented in this text. This classification is structured along two main dimensions:

- *Layers*: a construct can belong to one of the three previously presented layers: Business Layer, Application Layer, Technology Layer.
- *Aspects*: a construct can represent a passive, behavioral, or active element. These three aspects are inspired by natural language, where a sentence has a subject (active structure), a verb (behavior), and an object (passive structure). The *behavior* represents the actions performed by the system. The *passive (or static) structure* defines the objects of the behavior (e.g., an order at the business level, or an executable file at the technology level). The *active structure* defines the elements that provide the behavior and are therefore the subject of the action (e.g., actors at the business level, devices at the infrastructure level).

This chapter illustrates the general ArchiMate approach. In the following sections, first the main ArchiMate elements (ver. 3.0) are presented in Section 3.2, followed by some illustrative examples in Section 3.3. The open source tool Archi[4]

[4] `https://www.archimatetool.com/`

was used for creating the diagrams. For further details, please refer to the specifications [27] and the texts [17, 13].

3.2 Main Elements of the Model

The elements of the model are the components and the relationships to connect the components.

3.2.1 Components

The components in the model are the elements used to describe the structure of the system. For each layer, a set of components that characterize it is defined, as illustrated below.

Graphically, a component is represented by a rectangle. Some variants allow understanding which aspect the component refers to: whether the component is active, behavioral, or passive. In particular, simple rectangles correspond to active elements, rectangles with rounded corners correspond to behaviors, while rectangles with an additional upper band are passive elements.

Regardless of the aspect they refer to, each component also has an associated graphical symbol — also called decoration — that allows understanding the specific type of the component. Usually, the graphical symbol is represented in the upper right corner of the rectangle, but they can also have an alternative graphical representation.

3.2.2 Relationships

The relationships define the possible connections between elements and are illustrated in Figure 3.3. In general, the types of relationships are the same for all layers and are distinguished as "structural", "dependency", "dynamic" and "other" relationships. *Structural relationships* are used to represent the structure of an element, while *dependency relationships* indicate when one element depends on another. *Dynamic relationships* are used to represent behavioral dependencies between elements. *Relationships in the "other" category* are those that do not fall into the previous categories.

The main relationships defined in ArchiMate that are used throughout this chapter are the following:

- *Access*: links a behavioral component to a passive component or object, for example, an application function that accesses data. The arrow, when present, indicates the direction of the information flow. It is a dependency relationship.
- *Composition*: as in UML[5], links an element to its components, which cannot exist independently. It is a structural relationship.

[5]UML (Unified Modeling Language `https://www.uml.org/`) is a graphical modeling language for system representation, adopted as a standard by the Object Management Group (OMG) and published by the International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC) as ISO/IEC 19501 standard.

Structural Relationships		**Notation**
Composition	Indicates that an element consists of one or more other concepts.	
Aggregation	indicates that an element groups a number of other concepts.	
Assignment	Expresses the allocation of responsibility, performance of behavior, or execution.	
Realization	Indicates that an entity plays a critical role in the creation, achievement, sustenance, or operation of a more abstract entity.	
Dependency Relationships		**Notation**
Serving	Models that an element provides its functionality to another element.	
Access	Models the ability of behavior and active structure elements to observe or act upon passive structure elements.	
Influence	Models that an element affects the implementation or achievement of some motivation element.	+/-
Dynamic Relationships		**Notation**
Triggering	Describes a temporal or causal relationship between elements.	
Flow	Transfer from one element to another.	
Other Relationships		**Notation**
Specialization	Indicates that an element is a particular kind of another element.	
Association	Models an unspecified relationship, or one that is not represented by another ArchiMate relationship.	
Junction	Used to connect relationships of the same type.	(And) Junction Or Junction

Figure 3.3: Relationships

- *Realization*: indicates that an element plays an important role in the creation, attainment, or operation of a more abstract element. The most common interpretation is that all or part of the source element fully realizes the target element. The element on the side without the arrow realizes the element on the other side, for example an application component realizes a service corresponding to the business layer. It is a structural relationship.

- *Assignment*: associates an element with the one that performs it. It expresses the allocation of responsibility or execution. For example, a business role (active element) is assigned to the execution of a specific process (behavioral element). It is a structural relationship.

- *Serving*: indicates a relationship between a service and another element that uses it. It is a dependency relationship.

3.2.3 Business layer

We now present the elements characterizing the individual layers and the relationships among them. The presentation of the Business Layer is based on illustrating a typical Business pattern (Figure 3.4).

The figure illustrates a *Business Service* (note the elliptical icon that characterizes services in the graphical presentation of elements). A *Business Service* is associated with its interface (*Business Interface*), which corresponds to the channel through which the service is externally provided.

The *Business Service* is realized by a *Business Process* (characterized by the arrow icon), which accesses a *Business Object*.

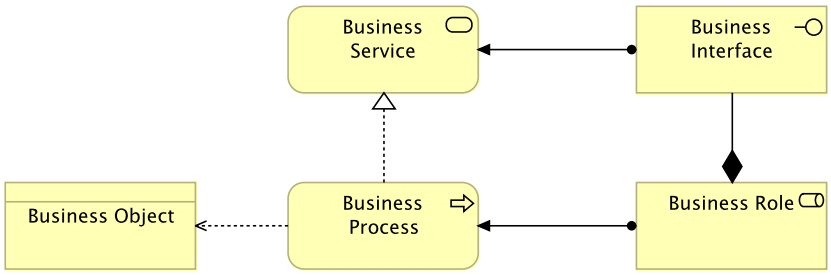


Figure 3.4: Business pattern

The active element that executes the process is a *Business Role* (characterized by a cylinder icon), assigned to the process. The interface of the *Business Service* is a component of the *Business Role*.

In Section 3.3 a complete example with elements from all layers is shown, describing how the activity of writing a document to answer a customer is realized at the application and technology layers using MSWord on a standalone PC. Figure 3.11 illustrates, as a Business-level example in ArchiMate, a document composition service (a generic service, linked to a Business Service not represented in the figure). In the Business layer, we see represented a Business Process (Write Answer), with the Business Role delegated to its execution (active object linked by the Assignment relationship), namely Customer support. The Business Process acts on a Business Object (Answer), connected to it by an access relationship.

3.2.4 Application layer

At the application layer, components have relationships similar to those described above for the business layer. As above, we illustrate a typical configuration at this layer in Figure 3.5. The figure shows an Application service with its associated interface.

The Application service is realized by an Application function (characterized by the upward arrow icon), which uses a Data object.

The active element executing the Application function is an Application component (characterized by the two rectangles on the side), assigned to the function. The interface of the Application service is a component of the Application component.

In Figure 3.11, the Application service is document creation, realized by the Editing function (Application function). The application component assigned to the editing function is a word processing component, with its GUI (Application interface), which is part of it. The Data object is the passive object of the editing function, namely Word Document.

The two illustrated patterns are connected in the model when an application is used by a business pattern. The connection is represented by a *serving* relationship, as illustrated in Figure 3.6.

Returning to the example in Figure 3.11, the Editing function accesses the word

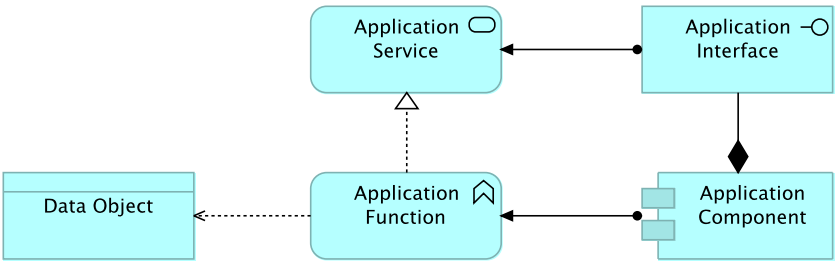


Figure 3.5: Application pattern

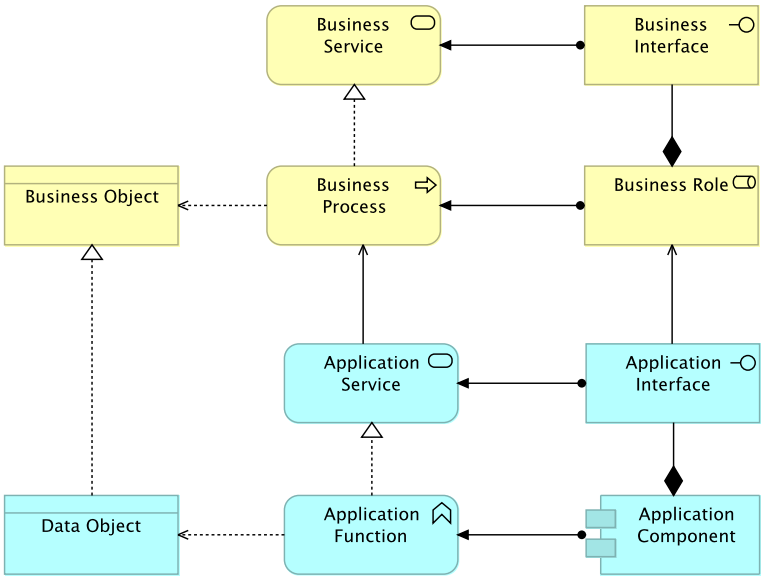


Figure 3.6: Connection of a typical application with a typical business function

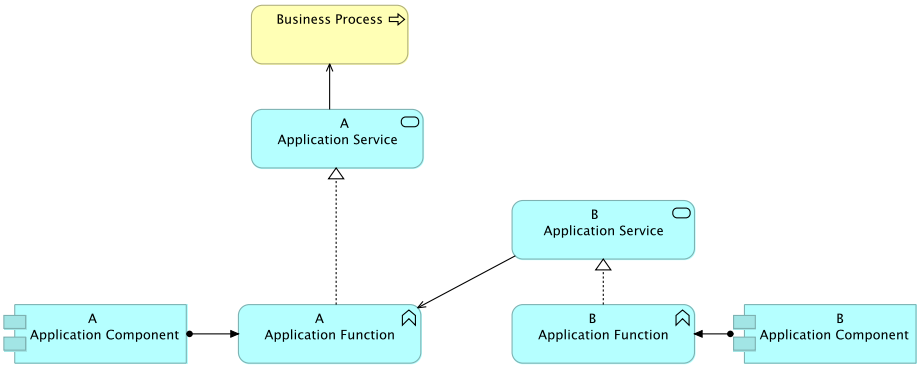


Figure 3.7: Connection between two applications

document (Application object). The active component is the word processing application, which has the editing function assigned, realizing the Document creation service. The application has a GUI interface component, which is assigned to the application service Document creation. The connection to the business layer is represented with the serving relationship between Document creation (Application Service) and the Business Process Write Answer, and the GUI serves Customer support.

The *serving* relationship can also be used to connect applications with each other. Figure 3.7 shows this connection. In the figure, two application functions are realized by two application components, each linked to its corresponding application function. The *application function B* is used as a service by *application function A* as follows: application function B exposes the service *application service B* via the *realizes* relationship. Application function A uses the service exposed by application function B through the *serving* relationship that connects application service B to application function A.

In turn, *application function A* exposes the service *application service A* via the *realizes* relationship, a service that is used (serving relationship) by the Business process.

3.2.5 Technology layer

The technology layer is now described. Figure 3.8 shows a configuration of the technological infrastructure following the same type of pattern used for the business and application layers.

As in the previous cases, we have a technology service with its associated interface and a technology function that realizes it.

The object in this case has a graphical representation to indicate that it is an artifact (text file, executable, or other). A node, with its technology interface, is assigned to the technology function, in the sense that it can execute it.

Figure 3.9 shows the connection between the application layer and infrastructure,

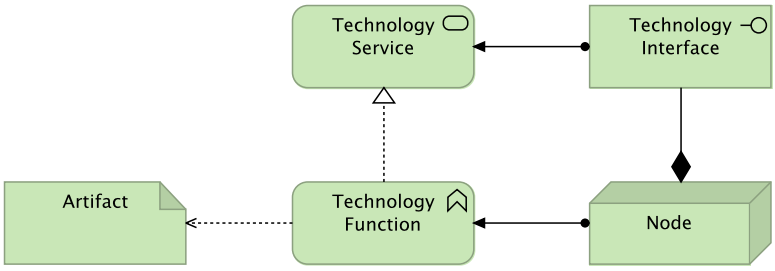


Figure 3.8: Basic infrastructure

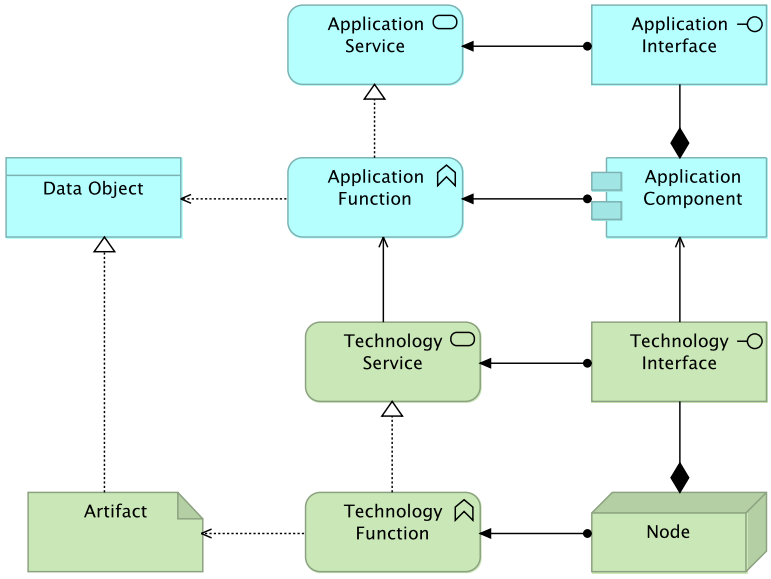


Figure 3.9: Connection between application layer and infrastructure

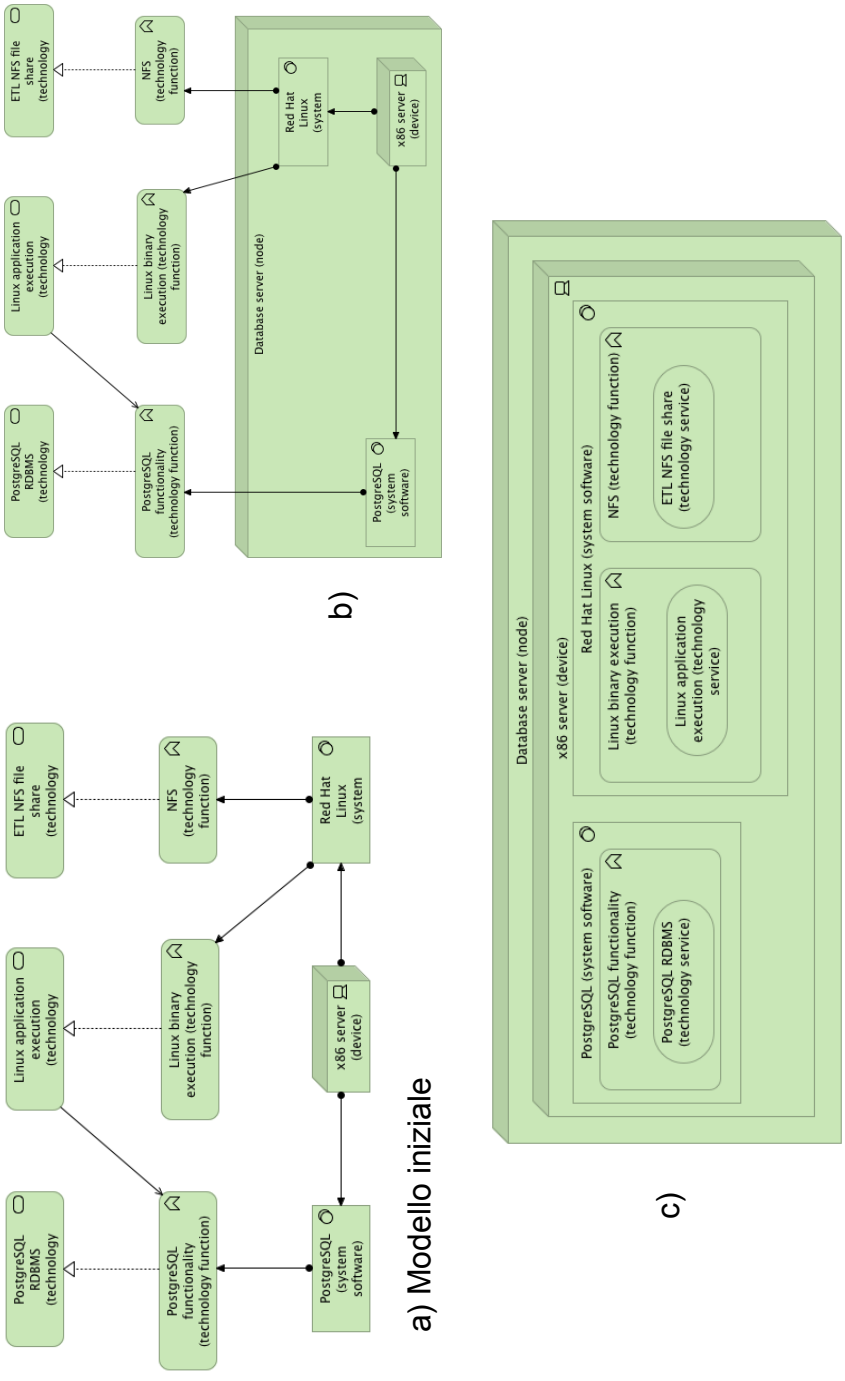


Figure 3.10: Examples of nodes

illustrating how the technological infrastructure provides the necessary services to the application.

Nodes

A node (Figure 3.10) can be used to model hardware and software infrastructure in a more or less detailed way.

A nesting relationship can be defined through composition, aggregation, assignment, and realization relationships. In this way, a node can model different levels of abstraction.

In Figure 3.10, three representations are provided at different levels of abstraction of an infrastructure component containing an SQL server, the operating system, and file-sharing software. The three representations are essentially equivalent, as they represent a database server. What changes in the three representations is the level of detail provided by the representation of the database server in showing the services provided and its hardware and software components and their relationships.

In Figure 3.10a, the relationships between all the individual elements composing the database server are illustrated, while in Figure 3.10b, the elements corresponding to the SQL server software and the operating system on the device are grouped into a node, still highlighting the technology services provided and the relationships among them. Finally, in Figure 3.10c, all components are included in the node, which will then be considered a single element at a higher level of abstraction.

3.3 Examples

3.3.1 Example of representing a document composition system

The model in Figure 3.11 shows writing a response to a customer by a Customer Support role. Two infrastructure services are required for this to work: one to run the application and the other to store the document. In this example, everything happens on a standalone PC.

We have already discussed in previous sections the Business and Application layers of ArchiMate and their relationships. Here we see them also connected with the necessary technological infrastructure. At the technology level, the Word document is realized by the file foo.doc. The editing function uses the technological service Windows application execution, realized by the technological function Windows binary execution, run on the device labeled Desktop or Laptop PC, to which is assigned, besides the binary execution function, also the executable word.exe.

3.3.2 Example of an insurance service

Figure 3.12 illustrates the architecture to provide a claim management service to customers by an insurance company.

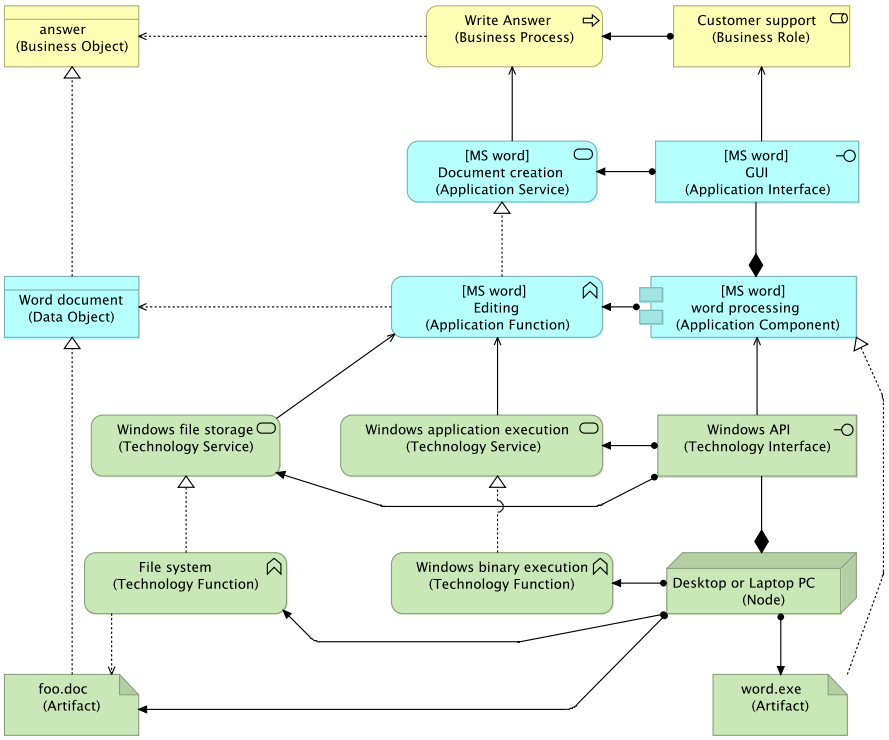


Figure 3.11: Example of a business service for document composition [13]

Note the representation of customers and the insurance company with the person icon, associated with the corresponding roles (the association relationship is here represented symmetrically).

In this figure, it is also useful to examine how the application components (CRM system, Policy administration, Financial application) provide the necessary services to realize the business process.

The technological infrastructure is provided by a mainframe and a blade system.

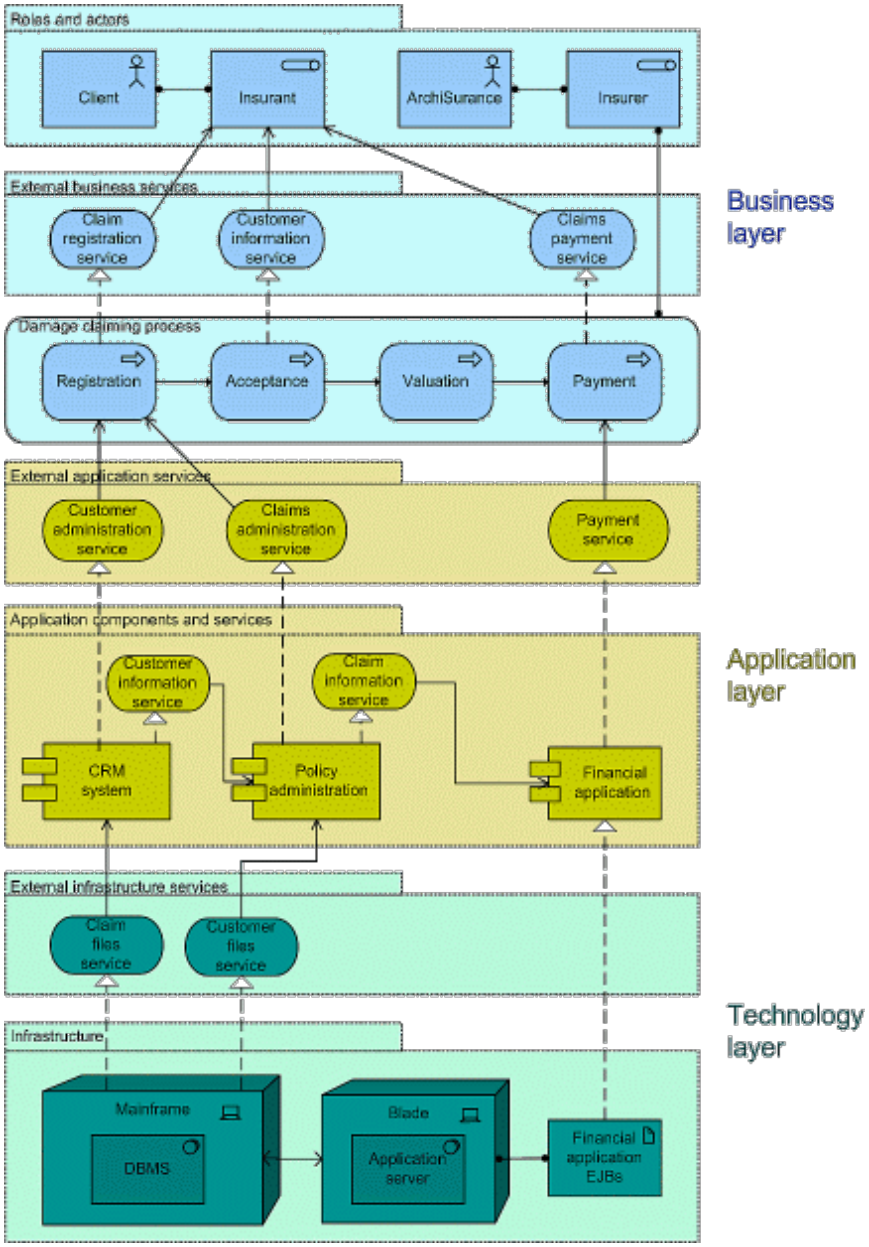


Figure 3.12: Example of a business service for document composition (source: IBM Developers word)

Chapter 4

Process Modelling

4.1 Process Modelling in BPMN

4.1.1 Definitions

As we have seen, the concept of process is central to the representation of information systems, as it allows for connecting the activities carried out for a specific purpose and understanding data flows. In the business-level modelling of ArchiMate, processes are the functions used to describe the realization of a business service, indicating in an abstract and concise way the main steps and the necessary business objects. For example, in Section 3.3, the process in the insurance example is described as a simple sequence of steps. In general, ArchiMate's approach allows for describing the main steps and major alternatives in the process execution, providing an overview of the business-level service realization, but not the detailed aspects of the process.

To describe processes in more detail, it is necessary to model them more precisely than we have done so far, highlighting the structure of interactions, message sequences, and main alternatives and exceptions. To represent these aspects, we need to introduce the concept of a business process and its modelling.

In this section, we first describe the basic principles and the nature of the business process concept. Then, we show how to specify business processes by introducing the BPMN (Business Process Modeling Notation) in Section 4.1.2.

A *business process* (hereafter simply *process*) is a set of *activities* performed in a specific order to achieve a certain *business goal*. A process can address primary goals, such as selling a product, or secondary ones, such as conducting a company's monthly financial analysis—an activity that constitutes a secondary process.

The activities in a process implement the steps needed to reach a goal and are performed by *actors* (also called *agents*) of the organization, who are located in organizational units. In the description of a process, actors are conceptualized as *roles*. A role describes the capabilities required of an actor to perform a certain activity so that an appropriate actor can be selected when the process is executed. Selecting an actor for a role is called *role resolution*. The activities of a process are

part of business functions. Actors that perform activities are often humans, but they can also be machines, which—in an e-business context—are typically automated information systems. For example, consider the goal of achieving fully automated processes.

A process is specified according to a *process model*, which defines the order of steps. Steps may be elementary activities or subprocesses. Elementary activities are atomic units that do not need further refinement in the process specification. Subprocesses are processes at a lower level of aggregation, used to reduce model complexity or reuse parts of a model for different business goals.

The ordering of steps in a model can be simple (e.g., a sequence) or complex (including alternative paths, parallel branches, and iterations). The ordering of steps over time is called the *control flow*. When dealing with processes, the focus is on the control flow rather than on the internal details of individual steps.

A process model specifies the structure of a given type of process—that is, all possible process executions. For example, a sales process model specifies all the ways in which sales should be executed. A single execution of a process according to a model is called an *instance of the model*, or a *process case*. Carrying out a specific sale is thus an instance of the sales process.

4.1.2 Business Process Modeling Notation

To facilitate interaction between different organizations—or even internal management of processes—identified processes must be modeled to define how a particular activity should be performed. Modeling a process allows for defining the exact order in which the various activities must be performed, as well as the possible relationships between internal processes and external processes involving partners (customers or suppliers).

One of the most widespread notations for process modelling is the Business Process Modeling Notation (BPMN). The main elements of a process are:

- *Activities*: the work units of the process. An activity has a sub-goal and a non-zero duration and contributes to the overall goal of the process. In BPMN, an activity is represented by a rectangle with the activity name inside.

- *Gateways*: used to model alternative flows within a process, such as decision nodes (based on conditions) or branches where multiple activities can be executed in parallel. In BPMN, a gateway is represented by a diamond shape with an internal symbol indicating its type.

- *Events*: events are information about the environment in which the process runs; they can be generated by the process or received during its execution and can come from inside or outside the organization. In BPMN, events are modeled as circles with an internal symbol defining their type.

A summary of the basic BPMN elements grouped by category is shown in Figure 4.1. The elements of a process are connected to each other through the *Process flow*. Activities can be simple or can represent a grouping of many activities that

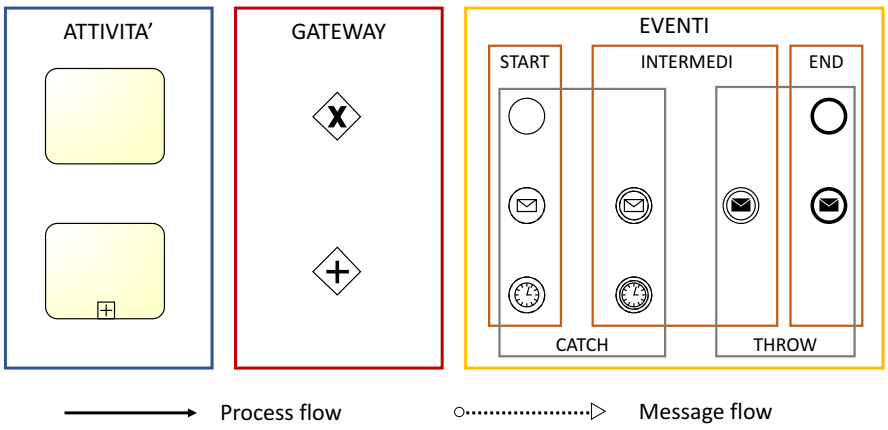
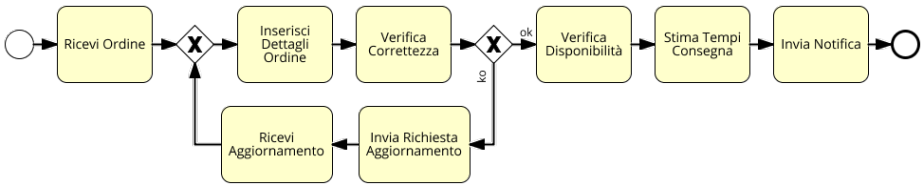


Figure 4.1: The main BPMN elements, grouped into Activities, Gateways, and Events.

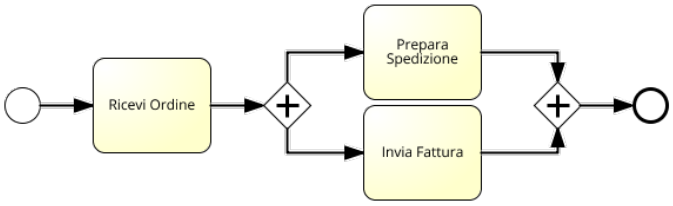
contribute to achieving a sub-goal of the process. In this case, they are called *sub-processes*, represented as an activity with a "+" at the bottom center, as shown in the figure.

There are also different types of gateways. In Figure 4.1, an XOR gateway (represented with an "X") and an AND gateway (identified with a "+") are shown. The XOR gateway allows selecting only one among two alternative flows, based on a condition on the process data. Alternatively, the gateway can also be used to merge multiple flows within a process (as soon as an activity on one of the incoming flows to the gateway is completed, the process continues). An example of a process using an XOR gateway is shown in Figure 4.2a, where an order management process is modeled that, if the entered data is incorrect, requires an update. The AND gateway, on the other hand, allows the activation of multiple branches in parallel and thus does not define a precise order for the execution of the activities found on these branches. When used to merge multiple branches, its behavior is to synchronize branches of the process by waiting for all activities preceding it on the incoming branches to be completed before proceeding. In the example of Figure 4.2b, a process is shown where, upon receiving an order from a customer, the invoice and shipment are prepared simultaneously, and the process ends only when both activities are completed.

Finally, events can be grouped into different categories. Events can occur at the beginning, end, or within the flow of a process. Based on their position, they are classified as *start events* (represented with a single circle — these are the conditions that initiate the process), *end events* (also represented with a single circle — typically the conditions that terminate a process), and *intermediate events* (represented



(a) Order process using XOR gateway



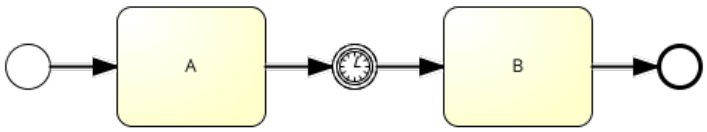
(b) Order process using AND gateway

Figure 4.2: Example of using XOR and AND gateways

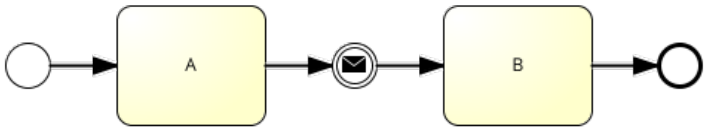
with a double circle). Every process always starts and ends with an event. Within an event, it is possible to find a symbol that identifies its type. Among the most commonly used events is the *timer*, which indicates any time-based event (every Monday, after two hours, upon timeout, etc.). Another frequently used event is the *message event*, represented with an envelope, which indicates an exchange of information between the organization and external partners. Events can also be categorized based on whether they are produced by the process or captured by the process but generated externally. In the first case, they are called *throw events*; in the second, *catch events*. The two types of events are distinguished in notation: throw events are marked with a black-filled icon. For example, sending a message is a throw event represented with a black envelope, while the expiration of a timer is a catch event. By nature, start events are always catch events, while end events are always throw events. Intermediate events can be either catch or throw. In Figure 4.3a, the process starts with a generic start event and, after completing activity A, waits for a timer event to begin activity B, and then ends with a generic end. Similarly, in Figure 4.3b, after activity A, the process sends a message and then proceeds.

BPMN also allows modeling the interaction between multiple organizations, which occurs through message exchanges. Each organization has its own process that must coordinate with the partner's process via message-type events or activities. The exchanged messages are explicitly indicated in the model using the *Message flow*. The message flow can originate from a throw message event and arrive at a catch event, or originate from and/or arrive at activity elements that generate or receive messages.

An example is shown in Figure 4.4, where the processes of a buyer (above) and



(a) Process with intermediate timer event



(b) Process with intermediate message throw event

Figure 4.3: Example of using events in a process

a seller (below) are modeled.

The same interaction, but with a different process on the seller's side, is shown in Figure 4.5. In this case, the seller applies a *business rule* that products are shipped only after receiving payment. As seen, the relationship between the actors may remain unchanged even if the internal process of one of them changes. However, in other cases, a change might also affect the partner's process, altering how the two actors' activities must coordinate.

This is also why process representations must provide a public view of the process to allow verification that the planned interactions are consistent. For example, in this case, if the seller waits for payment before delivering goods and the buyer waits for the goods before paying, a classic deadlock would occur, and the process would not be able to proceed.

This section briefly introduced the main BPMN concepts. For a comprehensive treatment and more examples, refer to [7] or to [32] for a more in-depth discussion on process modeling. The diagrams were created using Signavio, an online tool available with an academic license [23].

4.2 Process Identification

We have discussed how organizational modules (units) implement business functions within organizations. This implementation consists of the detailed definition of business processes, defined as the set of activities through which an organization's resources can achieve business objectives by transforming an input into an output, thus creating value. A business process is therefore focused on an output (a service or product) and describes all the activities and roles involved in obtaining it. For this reason, a process can involve several units within an organization and various roles that collaborate to achieve a common goal. The value resulting from the process

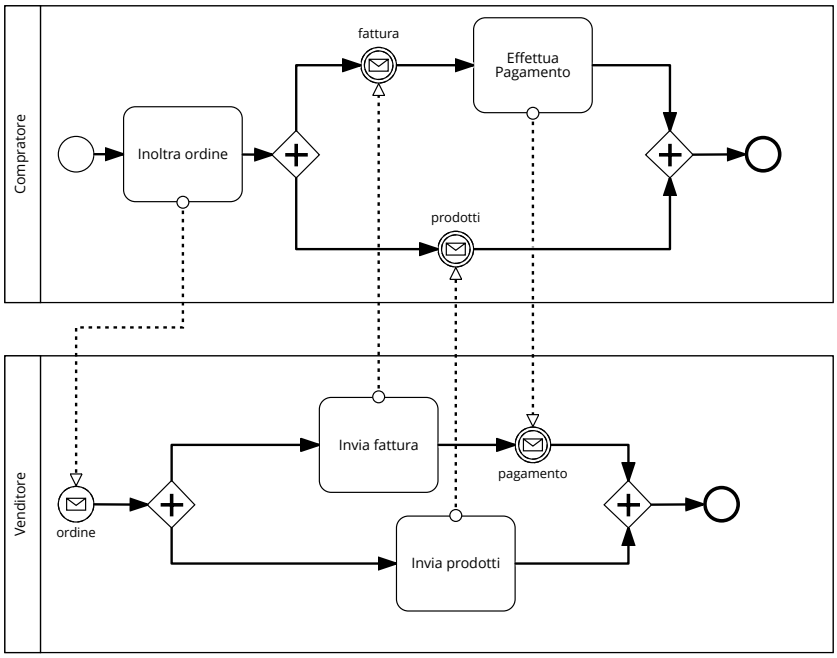


Figure 4.4: Example of a seller-buyer process

depends on the proper functioning of the individual activities and the quality of their coordination. Therefore, although a process is aimed at achieving a business goal (a functionality offered by the organization to external actors it interacts with), it defines all the activities that lead to that result. These activities therefore also include all back-end functions that contribute to the achievement of the goal and are not directly exposed to external actors.

Identifying and describing the processes of an organization is a very complex and costly activity. First of all, processes can be described at different levels of detail. Each process can be hierarchically broken down into phases, activities, and operations. Based on its needs, an organization can establish the level of detail for the description of its processes. Usually, small companies are not interested in describing processes at a greater level of detail (operations), as such an analysis may be excessively costly compared to the benefits obtained. Once the desired level of detail has been established, it is necessary to identify which processes exist within an organization, a necessary step to then proceed to their modeling.

Below, we discuss some approaches that can be used for process identification.

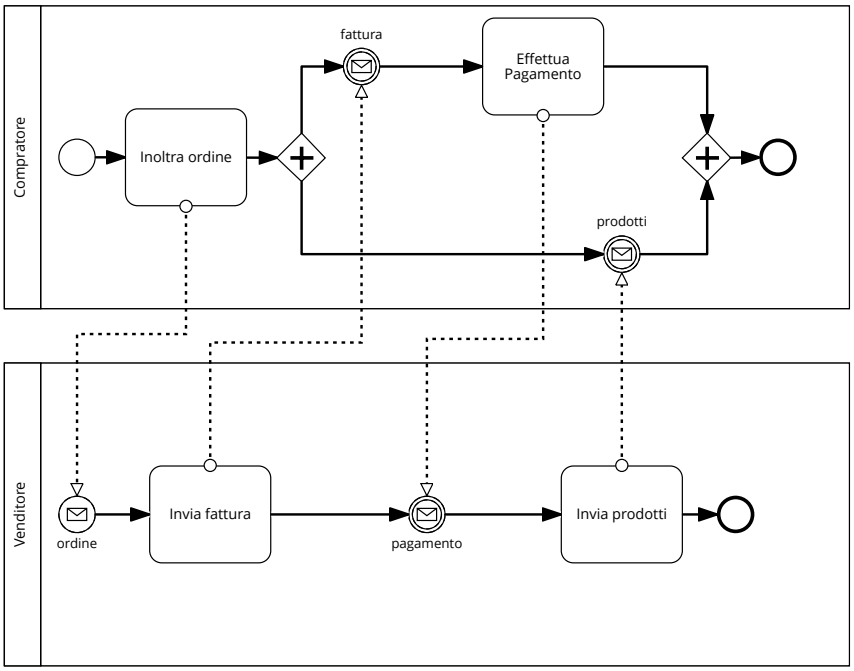


Figure 4.5: Example of a modified seller-buyer process

4.2.1 Sectoral Checklists

A very simple method to support the identification of processes is based on the similarity of organizations operating in the same sector. The method takes as reference a list of processes that have been identified as typical for the sector in which the organization operates, from which it is possible to determine which of these processes are present in one's own organization. This approach is called a checklist because it consists of selecting the processes to be considered from a reference list. An example of a framework that provides a classification of business processes is the Process Classification Framework provided by the American Productivity & Quality Center (APQC)[1].

Although this approach is very simple, it has several limitations. While the method is effective for highly regulated sectors where the processes of different organizations are indeed very similar, it is less applicable when the sector is not heavily regulated. In this case, a complete overlap between the checklist and business processes is not guaranteed. Another limitation of this method is the need to conduct an in-depth analysis of business processes before applying the method, in order to make the comparison. This operation is obviously costly and complex.

[1] https://www.apqc.org/pcf

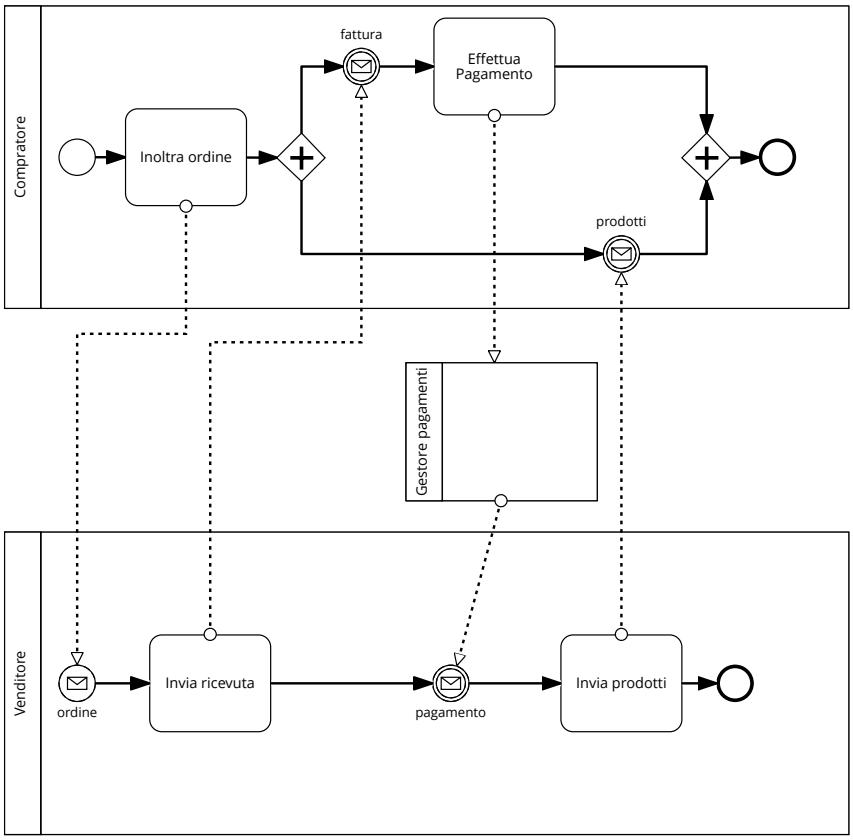


Figure 4.6: Example of a seller-buyer process with intermediary

4.2.2 Porter's Value Chain

In the introductory chapter, we saw how processes can be classified by referring to Porter's value chain. This model divides processes into primary and support processes and defines five categories of primary processes and four categories of support processes. This classification can also be used as a starting point for identifying an organization's processes.

This approach also has some limitations. First, Porter's approach focuses on activities divided by organizational units (marketing, logistics, production, etc.), thereby losing the cross-functional and collaborative vision typical of processes. Moreover, the activities identified by Porter through the nine categories represent only a limited number of an organization's activities, analyzed at a high level of process aggregation. Therefore, starting from this model, it is difficult to define processes at a more detailed level if such analysis is of interest to the organization. Lastly, the model is not a general-purpose model suitable for all types of organizations. It was in fact designed for traditional manufacturing organizations, where

physical production and transport are the most important activities. Consequently, it is not straightforward to effectively apply the model to service-oriented organizations or to small and medium enterprises where some of the activities identified by Porter might be entirely absent.

4.2.3 Analytical Approach

The analytical approach to process identification is based on a top-down analysis. The analysis starts by identifying the recipients, that is, the external actors with whom the organization interacts (suppliers, customers, intermediaries), and reconstructs the process backwards, first identifying the output provided to them by the organization, then the activities that generate this output, the roles involved, and finally the process inputs, both in terms of physical inputs and the information needed by the various roles to carry out their activities. For example, one can start from the definition of the external interfaces that the organization exposes and reconstruct the processes that enable such functionalities.

This procedure allows for the identification of primary processes, i.e., those that lead to the generation of an output in terms of products or services. To fully describe the organization, this procedure must be completed with the identification of support processes, i.e., those whose focus is not on the external organization but on its internal management.

The processes thus identified can be represented using process modeling techniques. Some BPM tools, starting from a process representation using a standard notation, can translate it into software procedures either automatically or semi-automatically.

The analytical approach is the most accurate among those presented, but also the most time-consuming and costly.

Part III

Technological Aspects

Chapter 5

Introduction to Technological Elements

5.1 Introduction

The technological components of information systems can be classified according to the categories identified in Figure 5.1.

Considering the architecture of information systems, Figure 5.1 can be seen as a map of the main technologies usable for the actual implementation of these systems. Technologies are divided into three levels: (i) application level, (ii) platform level, and (iii) physical architecture level. The actual selection of technologies to be used at each level is also guided by the application domain considered (*Application Domains* in Figure 5.1). Each application domain is characterized by different goals and needs that inevitably lead to different choices and implementations.

5.1.1 Application-level technology

Application-level technology includes all software applications that can be adopted to support various business functions in operational and informational systems. The set of components at the application level of a company is defined as the *application portfolio*. The portfolio is usually built by selecting some of the following types of applications:

- **DW - Data Warehouse**: archive containing an organization's data, designed using a multidimensional model to facilitate analysis in support of managerial decisions.

- **BI - Business Intelligence**: a set of modules for collecting and analyzing information useful for assessing the current business situation and forecasting future events.

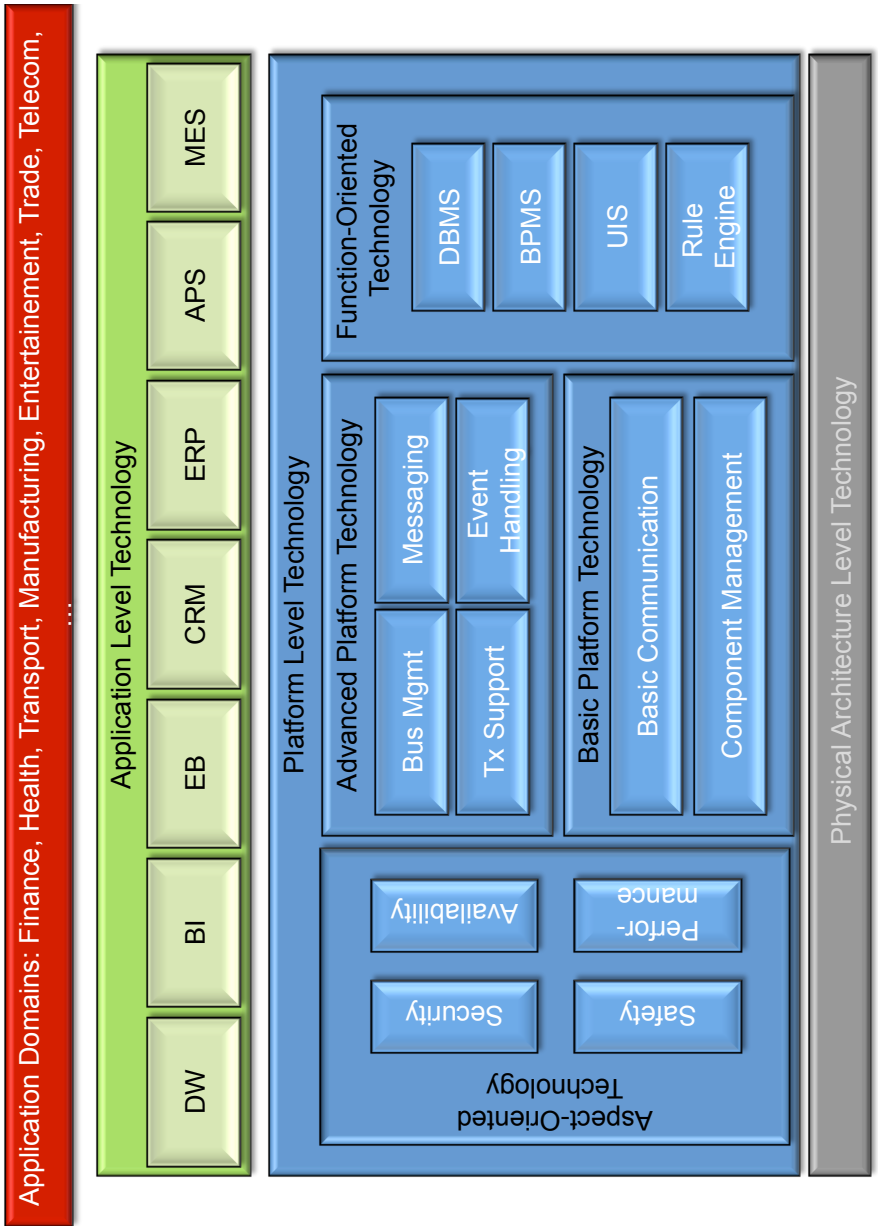


Figure 5.1: Technological components

- **EB - E-Business**: this term refers to applications that support business functions using digital tools for communication, collaboration between companies, and execution of business transactions.

- **CRM - Customer Relationship Management**: a set of modules supporting a strategy for managing interactions with potential and existing customers. These modules are responsible for storing and analyzing customer interactions, and for offering innovative services aimed at customer loyalty.

- **ERP - Enterprise Resource Planning**: software suite composed of modules primarily aimed at supporting a company's operational activities. For example, in a manufacturing company, ERP modules can support all primary and support activities constituting Porter's value chain.

- **APS - Advanced Planning and Scheduling**: set of applications used in manufacturing to optimally manage raw materials and production capacity.

- **MES - Manufacturing Execution Systems**: systems used in manufacturing to track and document the production process from raw materials to finished products.

When deciding which technologies to adopt at the application level, in addition to the type, the best approach for system implementation must be considered, choosing also between Make-or-Buy strategies. For functionalities now widely used across various industries and thus highly standardized, Buy solutions are often used, allowing the adoption of COTS (Commercial Off-The-Shelf) software modules, as is typically the case for ERP suites, based on consolidated and customizable procedures according to business needs. These modules must then be integrated with modules developed using the Make approach to meet the organization's specific requirements (this topic is further explored in Section 11.2.2).

The application modules listed above correspond to application-level components in ArchiMate and may themselves be composed of more detailed functional modules or use services provided by other application components.

5.1.2 Platform-level technology

The implementation of applications requires the use of various platform infrastructure technologies, allowing application components to implement their functionalities by using support components, such as data access, inter-application communication, and security. Technologies at this level can be divided into four categories: (i) Functionality-oriented technologies, (ii) Non-functional-oriented technologies, (iii) Basic technologies, and (iv) Advanced technologies.

Among the **functionality-oriented technologies**, we find:

- **DBMS - Database Management Systems**: systems for managing, controlling, and optimizing data access and manipulation.

- **BPMS - Business Process Management Systems**: systems for modeling, automating, executing, monitoring, and optimizing business activity flows (i.e., processes).

- **UIS - User Interface Systems**: a set of services capable of supporting user interaction and managing different types of interfaces in an adaptive and flexible way, including advanced interaction features and management of graphical and multimedia elements.

- **Rule Engine**: software system that allows defining, testing, and executing business rules at runtime independently from applications.

Non-functional-oriented technologies include technologies that guarantee some non-functional characteristics of applications, such as efficiency, security, or correctness. In Figure 5.1, systems are particularly highlighted for ensuring application and communication security, availability of delivered functionalities, system performance, and safety with respect to physical and environmental risks.

Basic technologies include technologies for managing communication between applications and components. Component management systems allow for monitoring and tracking components and services used within the organization, facilitating both reuse and traceability.

The last category includes **advanced platform technologies** (Advanced platform technology in Figure 5.1). Among these are messaging systems, event and transaction management (Tx support) systems that support integration and communication between applications.

5.1.3 Physical architecture-level technology

As discussed in the introduction, current information systems rely on IT systems that constitute their automated component. IT system architectures have developed and evolved over the years from centralized schemes to distributed models, which better respond to the decentralization and cooperation needs of modern organizations. Technologies at the physical architecture level include paradigms for allocating applications to physical machines, specifying outsourcing solutions such as Cloud Computing. From an architectural point of view, it is important to define who manages the provided infrastructure. It is possible to reason in terms of physical infrastructure management, or at the platform level, or even at the application level. The recent trend is resource virtualization, making them available as *services*. In this case, no physical server will be available, but rather virtual servers (Virtual Machine - VM) with characteristics similar to their physical counterparts. Virtualization of physical resources makes the system more flexible, which becomes a set of virtual resources dynamically corresponding (i.e., changing according to computing needs) to actually available physical resources.

Details on technologies related to physical architectures will be described in later chapters.

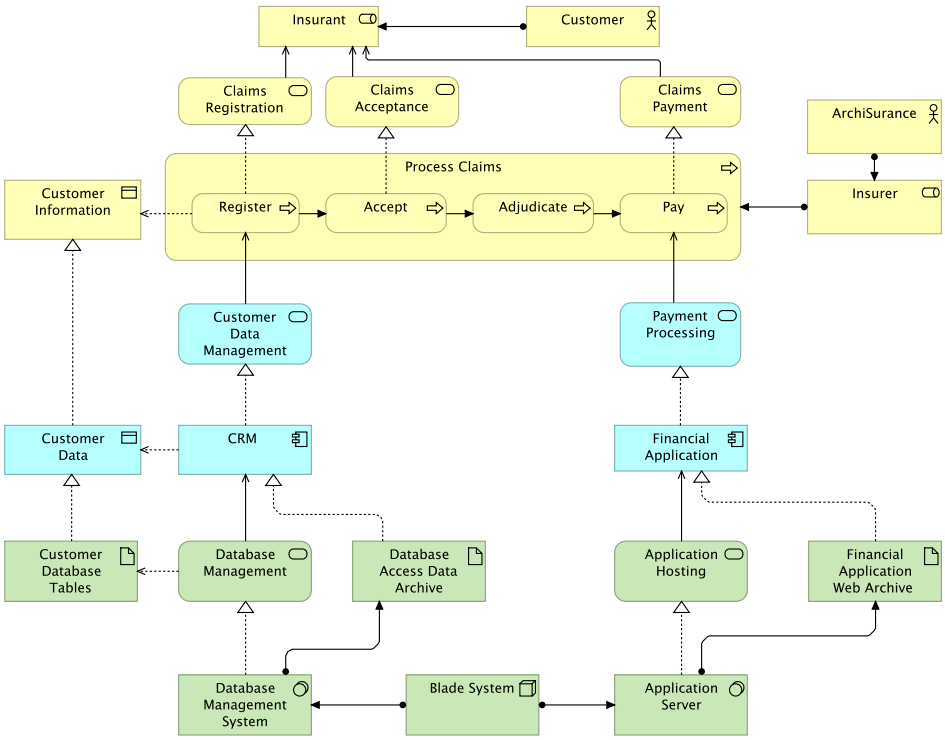


Figure 5.2: Example of component representation across the three layers in ArchiMate (source: The OpenGroup, 2019)

5.2 Application and Technological Components in ArchiMate

Figure 5.2 illustrates an example of the implementation of business-level services using application and technological components with the ArchiMate notation.

At the business level, three services are provided to the insured party (Insurant: the insured is a role, assumed by a Customer, who is an actor, in this case a person): claim registration, claim acceptance, and claim payment. The process that supports claims management is Process Claims, which includes the Register process implementing Claims Registration, Accept implementing Claims Acceptance, and Pay implementing Claims Payment. Process Claims is assigned to the Insurer role performed by the company ArchiSurance.

At the application level, two services—Customer Data Management and Payment Processing—serve the Register and Pay processes.

Customer Data Management is implemented at the application level by a CRM system managing customer data. At the technology level, it uses a platform service for database management (Database Management), which accesses customer data

tables (Customer Database Tables). The Database Management System (DBMS) is a system-level software on the platform that actively executes the Database Access Java Archive object that handles data access. The DBMS runs on a node (Blade System) as an active element, constituting the physical architecture of this system. No additional implementation details of the CRM application are shown in the figure.

The Payment Processing service is implemented by a Financial Application Web Archive object, which realizes the Financial Application.

An Application Hosting service supports both the CRM system and the Financial Application and is implemented by an Application Server system software, assigned to the Blade System node.

Chapter 6

Main Components at the Application Layer

In this chapter, we will analyze in more detail the application-level technologies introduced in Chapter 5. It has already been discussed how the information system must support various business processes that allow for the execution of specific functionalities. These processes can be fully or partially automated through software applications. In this chapter, we introduce the evolution of application components and the main applications that make up the operational-level portfolio of an organization. In Chapter 7, we will instead introduce the main application components that support the tactical and strategic level of Anthony's pyramid.

6.1 Evolution of Application Components

The application components of information systems have undergone significant evolution over time. The first applications were developed using languages close to machine language (assembler or similar) and encoded only basic functionalities. In the 1970s, operating systems began to allow the separation of application logic from hardware interface programming. Each application was independent and used dedicated data saved in files. In the 1980s, DBMSs emerged, enabling data sharing between different applications, avoiding redundancy and decoupling the physical structure of data from the way applications accessed them. Finally, graphical interfaces were introduced to simplify user interaction with applications, replacing previous textual interfaces.

An application component can be broken down into different elements, both from a functional and architectural point of view. The architecture of an application can be very complex. However, in general, applications can be structured into three logical layers, often referred to in the literature as *layers*, which correspond to three software layers into which the applications are partitioned:

- The *presentation layer (P)* is responsible for managing presentation logic, that

is, the methods of interaction with the user. It includes the graphical interface and information rendering. This layer is sometimes also referred to as the *front end* of the application and is accessible to users via a GUI (Graphical User Interface) or UI. This layer is usually web-based.

- The *application logic (A)*, or business logic layer, is responsible for the functions to be made available to the user.
- The *data access logic (D)* layer handles information management, possibly accessing databases or legacy systems (a *legacy system* refers to a system inherited from the past and developed with older generation technologies, such as a COBOL application, which is still very important for the organization), or even information resources available on the internet.

The presentation layer constitutes the *front end* of the application, i.e., the part of the application component dedicated to user interaction. The application logic and data access layers constitute the *back end* of the application and are accessible through methods based on APIs (Application Programming Interface). Unlike GUIs, which are designed for human users, APIs are aimed at other software. In this way, different applications can be integrated. Drawing a comparison with typical programming languages (e.g., the C language), an API for application logic appears as a list of method signatures[1]. Regarding the data access layer, APIs can also take the form of methods that receive an SQL query as input and return the result of that query.

From an architectural point of view, logical layers can be contained within a single *software artefact* in the case of monolithic applications, or split across different *software artefacts* that communicate with each other via appropriate interfaces (Fig. 6.1) in the case of distributed applications. In the latter case, one can distinguish between *closed systems*, in which each logical layer is designed to communicate only with adjacent layers, and *open systems*, where each logical layer can also communicate with external systems (e.g., other logical layers from other applications).

Applications designed according to the distributed model are more flexible and scalable, and for this reason, they are the most widely used in current practice.

6.2 Functional Components

An information system is a complex system composed of several applications, which in most cases are *modular*: a module is defined as an autonomous software component associated with one or more functionalities, and therefore with specific functional and application objectives. The architecture is thus partitioned into functional modules, each of which plays a specific role. Often, the partitioning into modules reflects the functional division of various activities in the company (consider, for example, the functions identified by Porter's value chain at the operational level).

[1] A method's signature consists of the method's identifier and the list of input parameter types it accepts

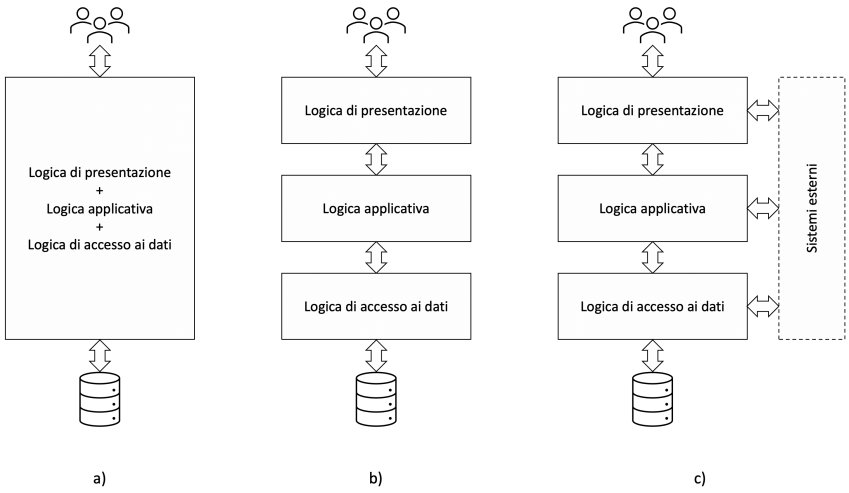


Figure 6.1: a) Monolithic applications b) Closed distributed applications c) Open distributed applications

Modularity is a fundamental characteristic because it improves the maintainability of the architecture. Especially in open systems—i.e., those structured to cooperate with other systems over a network—where interconnection with other firms fosters change and organizational dynamism, internal architectures are often subject to change. Modularity allows a module to be replaced or its functionality changed without affecting the entire architecture, but only the impacted module(s). In such cases, the change might not even be visible at the interface level, and therefore does not alter other modules or their cooperation. Furthermore, information systems in organizations are so complex that building them from scratch is nearly impossible or, at the very least, not economically viable. For this reason, it is usually better to opt for already available modules on the market (i.e., *COTS* components), and only develop specific functionalities. With this approach, when facing the classic Make-or-Buy decision, the Buy option is increasingly favored—at least for standard functionalities—over the Make option.

Two of the applications that companies most frequently acquire using the "Buy" approach are *ERP (Enterprise Resource Planning)* and *CRM (Customer Relationship Management)*, which are described in the following sections. According to what is described in Chapter 5.1, ERP and CRM are application-level technological components.

6.3 ERP

ERP (Enterprise Resource Planning) is a software suite that offers modules to support the operational system and thus the internal transactions within the organization aimed at enterprise management. An ERP system provides functionalities

that fully cover Porter's value chain.

6.3.1 History of ERP

To understand the emergence of ERP systems, one must trace back to the advent of CIM (Computer Integrated Manufacturing) systems, which became widespread in organizations during the 1980s and focused on production activities in manufacturing companies. In fact, starting from activity monitoring, data integration and evaluation from individual machines, and data related to the organization of cells, production areas, and plants, CIM aimed to provide an integrated view of the production system, supporting its various sectors (design, engineering, production, quality control, production planning, and marketing). This was mainly to support decisions related to process efficiency optimization.

Alongside CIM systems, MRP (Materials Requirements Planning) systems also emerged in the 1980s, with the following key functionalities: (i) scheduling material requirements by automating the bill of materials and, based on production targets (e.g., number of products to be produced), automatically defining the list of raw materials to purchase, (ii) scheduling operations, and (iii) inventory control. MRP systems were soon absorbed by MRP II (Manufacturing Resource Planning) systems. These are modular systems designed to replace any other sector-specific system for managing raw materials and operations. Indeed, MRP II systems deal with planning and managing all business resources and processes at the operational level: materials, production, personnel, maintenance, financial and administrative resources.

ERP emerged in the 1990s to further expand MRP II systems. The first version of ERP directly connected accounting management with logistics management (warehousing and procurement). Subsequently, internal relationships were also implemented with other operational level areas (e.g., sales, distribution, production, plant maintenance, project management, etc.).

Since the early 2000s, major ERP solution providers have begun creating vertical IT solutions for various business sectors; thus, specialized applications began to emerge for the automotive sector, retail, logistics, mechanical industry, and construction industry.

In recent years, the ERP domain has also seen widespread adoption of technologies that enable data analysis to extract knowledge and build models, including predictive ones, thanks to Artificial Intelligence and Machine Learning techniques. This chapter primarily examines aspects related to supporting operational activities, while data analysis technologies are treated separately in Chap. 7. These features are currently an important and constantly evolving part of ERP systems. Currently, evolution also focuses on new functional aspects supporting sustainability in organizations, for example, generating reports on a company's environmental impact.

6.3.2 Properties of ERPs

In addition to the functionalities provided by early IT systems, it should be noted that until the early 1990s, information systems in companies followed a design and development cycle known as the *silo approach*: applications were autonomous islands supporting a well-defined business function. These islands contained all software components (database, application logic, and presentation) necessary to perform their intended tasks. This approach worked well as long as the applications did not need to interact. The need to integrate or simply communicate applications highlighted the weaknesses of this approach, such as: (i) the need for significant IT effort to manage information flows between applications, (ii) data redundancy, which caused misalignment and fostered errors and inconsistencies, and (iii) difficulty in obtaining an overall view of data across different subsystems.

These problems encouraged the spread of ERPs, which are characterized by three fundamental properties:

- Uniqueness of information.
- Modularity.
- Prescriptiveness.

Uniqueness of information refers to a single logical representation of data valid across all ERP modules. All modules see the same values: this resolves redundancy and data inconsistency issues.

Regarding *modularity*, the suite is composed of self-sufficient modules that can be acquired and integrated incrementally. Each module consists of functions that generally support elementary activities (e.g., material reception). In addition to benefits in terms of application flexibility and scalability, modularity enables new COTS acquisition strategies. Indeed, one can purchase modules from a single provider (*One Stop Shopping Strategy*) or from different providers (*Best of Breed Strategy*). In the former case, interoperability between modules is guaranteed. In the latter, interoperability is not guaranteed, but there is an opportunity to acquire modules from highly specialized providers, hence of better quality, to perform various functions. Clearly, in the latter case, significant integration effort is required.

The third characteristic of ERPs, *prescriptiveness*, is linked to the fact that these products incorporate the company's "operating" logic, also codified in business rules. For example, production and logistics processes can be regulated by prescribing that a material cannot enter without an order or that a material can be ordered if and only if it has been requested by an authorized business unit. In this way, it is the company that must adapt to the software and not the other way around. This could be an advantage for companies adopting ERP, as the embedded logic is usually optimal, being designed based on best practices or well-known and optimized processes.

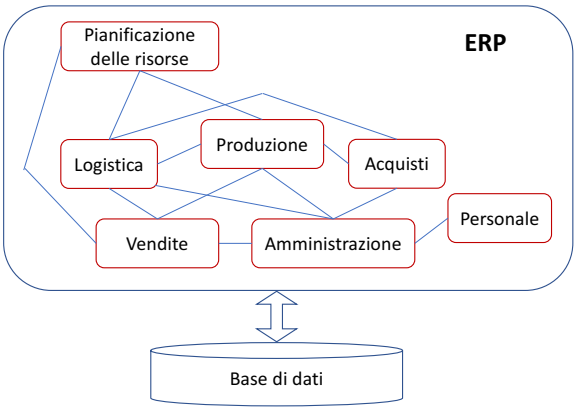


Figure 6.2: Organization of an ERP system (adapted from [19])

6.3.3 ERP Functionalities

Figure 6.2 shows a schematic representation of the organization of an ERP system. The modules represented are some of the main functionalities implemented according to best practices. These modules can be classified into three categories: institutional systems, sector-specific modules (primary activities), and managerial modules (see Figure 6.3). Institutional and managerial modules are horizontal, meaning cross-sectoral. Sector-specific modules are vertical, meaning they are specific to a given business context. Managerial modules provide data processing aimed at offering an overview of the activities carried out at the operational level, including elements such as dashboards, reporting, and advanced economic or business intelligence analysis.

Figure 6.3 represents a so-called *core* ERP, i.e., the main system. Subsequently, the core ERP has been extended to meet new needs, especially in terms of interaction with customers and suppliers. New applications have been developed, giving rise to the *extended* ERP, as shown in Figure 6.4. PLM (Product Lifecycle Management) is an application capable of managing product information throughout its entire lifecycle—from product conception to realization. SCM (Supply Chain Management) is a software for managing the supply chain that provides information to support purchasing decisions and production scheduling. In particular, this information helps companies plan their supply chain (supply chain planning) and execute the steps of the supply chain (supply chain execution). CRM (Customer Relationship Management) supports customer interaction, from initial contact to after-sales, with the goal of providing advanced services to foster customer loyalty and increase satisfaction.

Note that these extension modules may be part of the ERP suite as extension modules, but alternatively, they can also be acquired separately from other vendors and therefore appear in the architecture as external modules with which the ERP system must interact and be integrated.

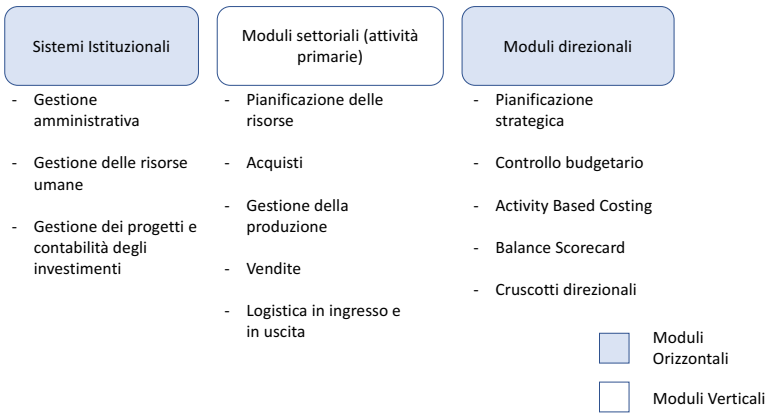


Figure 6.3: ERP Suite

6.3.4 Architectural Aspects

From an architectural point of view, three main approaches can be identified, shaped by the evolution of technology over time (see Figure 6.5). In the 1990s, ERP was installed in organizations using the client-server paradigm. In this setup, all functional modules were installed on one or more servers, and users accessed the functionalities via clients. With the spread of the Web, ERP vendors began offering *Web-based ERP*: ERPs developed as web applications and thus accessible through a browser. This shift led from two-tier architectures to three-tier or more. This innovation enabled companies to access their ERP systems remotely, offering users the ability to use ERP functionalities even outside the organization. For data analysis, modern ERPs have adopted OLAP-based technologies for the analytical part, allowing efficient, complex queries on large volumes of data. They have also developed analytical modules using data mining and machine learning techniques. Another major ERP innovation coincided with the spread of Cloud Computing: many ERP vendors now offer modules in SaaS (Software as a Service) mode. These architectural details will be described in more depth in the following chapters.

In any case, the architectural evolution of ERP has led to decreased costs for acquisition and management. Web-based ERPs allowed outsourcing of IT system management, significantly reducing costs. These costs were further lowered by SaaS ERPs because (i) service-oriented development logic reduces vendor costs and thus sale prices, and (ii) pricing is consumption-based and typically results in lower total costs than traditional acquisition (license and implementation costs).

6.3.5 ERP Implementation

As mentioned earlier, ERPs are COTS applications. As such, they are not designed and built for a single company but for a set of companies in a specific industry (or

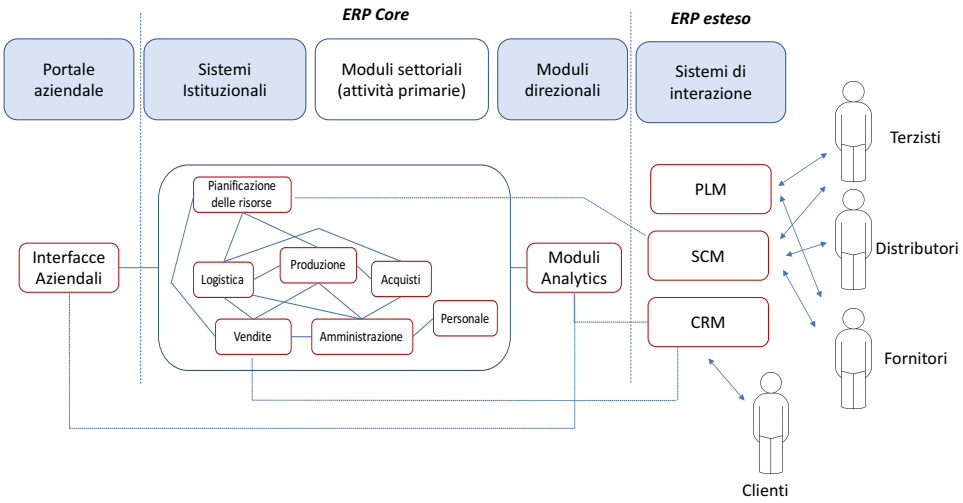


Figure 6.4: Extended ERP

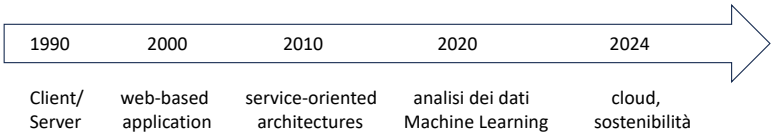


Figure 6.5: ERP Evolution

application domain) or sharing some characteristics. Therefore, different suites are available for different sectors. Suites designed for manufacturing companies offer modules reflecting Porter's value chain functionalities. However, for example, chemical and steel industries follow a slightly different model, and the developed software suite will thus comprise different modules. Figure 6.6 shows a possible segmentation of ERP modules for such companies. Note that Supply Chain Management is not included in ERP as it is usually an extension.

Figure 6.7 illustrates instead a software suite for telecommunications and energy service companies. An ERP supports almost all operational activities; only network planning and design are excluded.

In any case, even though ERPs are verticalized for a specific sector, companies within the sector often have different needs. Therefore, once installed, they are not immediately usable but must be properly configured to meet business requirements as much as possible (see implementation process in Figure 6.8). The first step in this direction is parameterization: by setting certain parameters, functionalities can be enabled or disabled, or a specific version of a function can be selected. This builds an application instance. If the company wants to make further changes to the product, it must proceed with actual customization, which involves changing

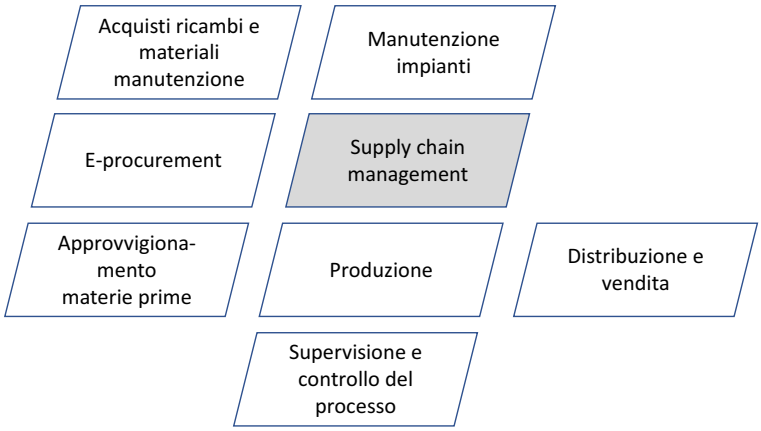


Figure 6.6: Main modules offered for chemical and steel companies (adapted from [6])

the code. This phase can be likened to a software project aimed at modifying an existing program.

6.3.6 ERP Benefits

Adopting an ERP affects not only management processes but also how the company positions itself in the market. In other words, this intervention brings benefits in terms of both efficiency and effectiveness. Efficiency benefits relate to time and cost savings in operational activities obtained through procedure automation and integration. Effectiveness benefits are linked to data uniqueness, the ability to process and transmit information, and IT platform standardization: there is, in fact, the possibility of greater data availability, accessibility, and timeliness. This enables access to the data needed to support tactical and strategic decision-making.

6.4 CRM

CRM (Customer Relationship Management) is a software suite that supports organizations in interactions with customers. CRM places the customer at the center of the business strategy. CRM is typically used to better understand customer needs and behaviors and to improve the use of customer interaction channels. In particular, the goal of CRM is to retain customers by building long-term personalized relationships capable of (i) increasing customer satisfaction and (ii) increasing the value of the company for the customer and of the customer for the company.

CRM is often seen as an extension of ERP because it provides additional functionality for customer management, from the initial contact to after-sales service. Unlike ERP, however, only certain companies can benefit from its implementation.

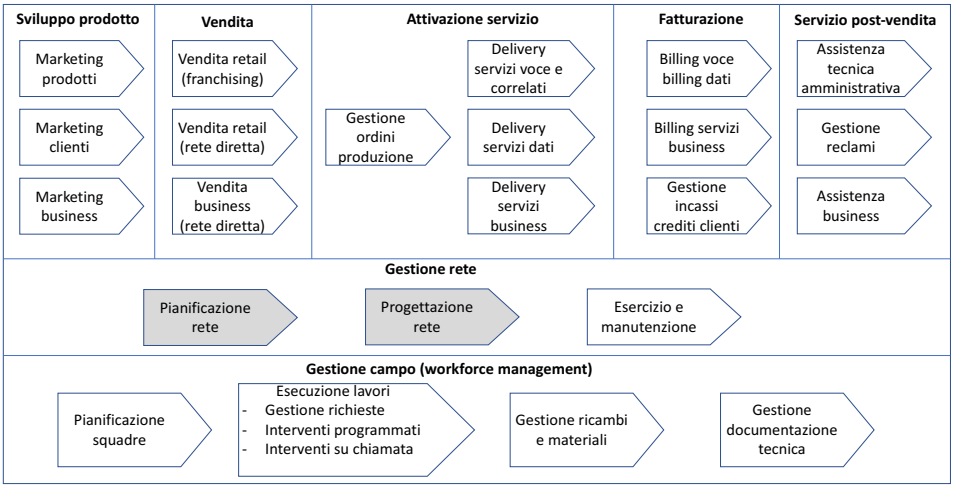


Figure 6.7: Main modules offered for telecommunication and energy service companies (adapted from [6])

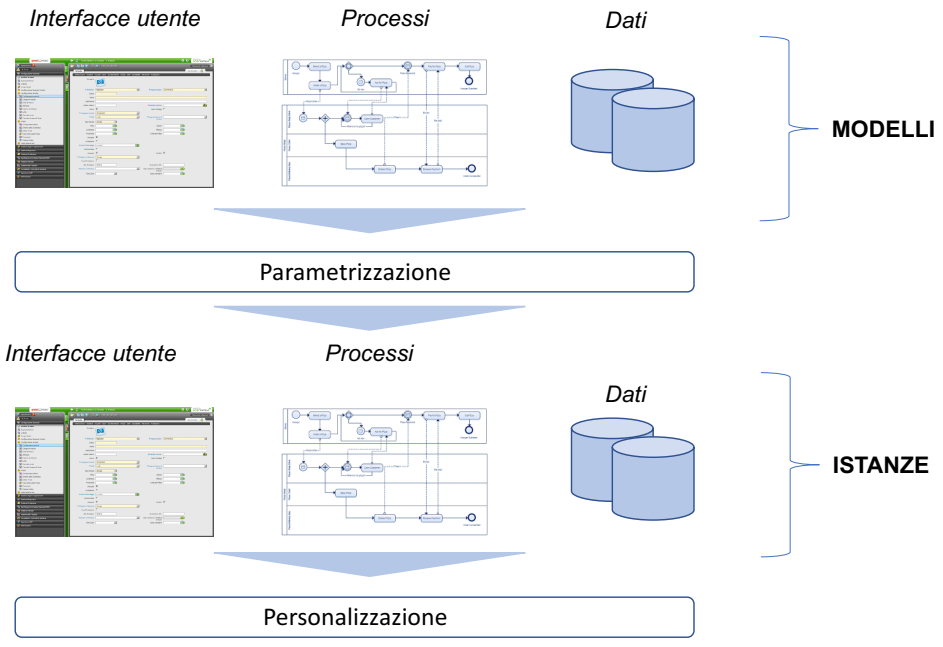


Figure 6.8: ERP Implementation

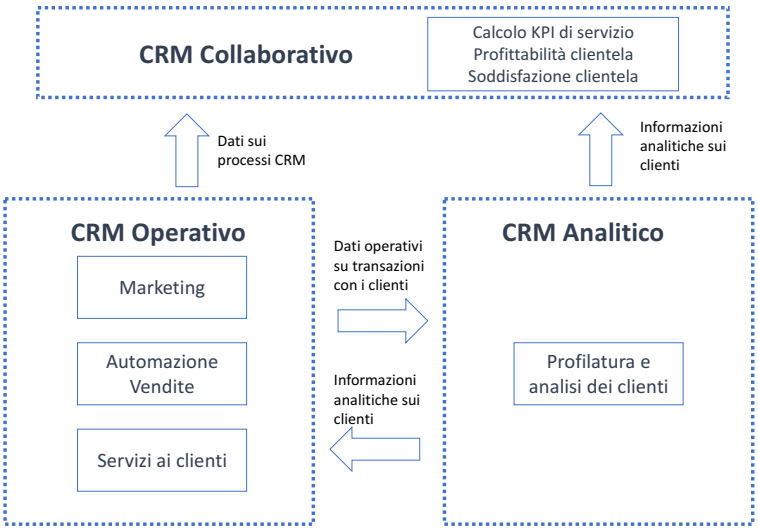


Figure 6.9: Components of an Enterprise CRM

In fact, the opportunities created by CRM are greater in sectors where there is higher relationship intensity with customers (frequency of interaction, duration of the contractual relationship), a sizable customer base, the potential to establish customer loyalty, and multichannel interaction. The latter refers to the fact that a customer can access an organization's services through various channels (e.g., physical locations, web, phone). In short, CRM systems provide more functions than the commercial logistics systems in ERP suites. Key aspects of CRM include multichannel customer relationship support (inbound and outbound contact), post-sales support (service) and loyalty management, and analysis of customer behavior.

It is important to understand that CRM is not just an application, but a true strategy that the organization must pursue at a company-wide level. Although CRM consists of different software modules and technologies, it should be considered a system supporting a process aimed at improving business objectives. Implementing a CRM helps better characterize customers and design personalized marketing campaigns with the goal of increasing returns.

The two main components of CRM are operational CRM and analytical CRM. *Operational CRM* is the front-end module that supports daily customer interactions, while *analytical CRM* is considered a back-end module that focuses mainly on customer data analysis. Figure 6.9 shows the components of CRM and the communications between modules. The figure also shows a third module, *collaborative CRM*, which is responsible for calculating indicators to be shared with various stakeholders within the company (e.g., sales and marketing managers, head of sales agents).

Figure 6.10 expands on the previous figure by focusing only on the interaction between operational and analytical CRM, highlighting especially the components

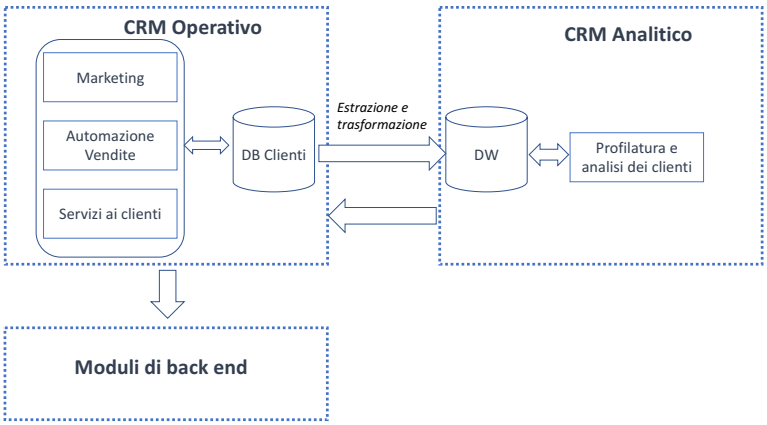


Figure 6.10: General architecture for data management in a CRM

Marketing	Vendite	Servizi ai clienti
Generazione liste clienti	Gestione vendite	Contact center
Gestione campagne	Gestione contatti	Web based self service
Cross-selling e Upselling	Gestione opportunità	Call scripting

Figure 6.11: Components of Operational CRM (adapted from [4])

needed for data exchange and management. Operational CRM records all customer interactions with the company across various channels into a single database. Analytical CRM relies on a data warehouse built from the information extracted from the operational CRM database. The functioning of the data warehouse will be explained in Section 7.1. The figure also shows that operational CRM data also feeds into back-end systems. In fact, customer interactions often generate events that must be handled by back-end systems—for example, a sales request must then be managed by an ERP.

In the following sections, we describe the composition of CRM modules as presented in Figure 6.9.

6.4.1 Operational CRM

Operational CRM deals with customer interaction with the company and, as described in Figure 6.9, it consists of various components to support sales, marketing, and customer service activities. These components rely on other modules, as described in Figure 6.11.

Marketing Company strategy no longer focuses solely on expanding the customer base and selling products to as many customers as possible, but rather on maximizing loyalty among existing customers and thus selling more products to

already acquired clients. This is achievable only through well-designed marketing campaigns targeted to individual clients or small client groups. It should be emphasized that the success of a marketing campaign is usually proportional to the company's ability to acquire and analyze data.

Systems that a marketing department may implement to increase customer satisfaction include:

- *Customer list generation*: produces lists of customers suitable to receive marketing communications. These lists are compiled through customer segmentation based on data related to interactions with the company (e.g., website visits, surveys, call center contacts). The segmentation is provided by analytical CRM.

- *Campaign management*: this module, also known as Marketing Automation, refers to tools designed to automate marketing tasks and processes. Specifically, it guides users in defining, planning, scheduling, and analyzing marketing campaign results. The benefits a company can gain from implementing this module are significant: (i) improved efficiency and reduced errors, (ii) increased contact conversion rates (the percentage of contacts turned into customers), (iii) enhanced marketing productivity and effectiveness, (iv) strengthened customer relationships.

- *Cross-selling and Upselling*: Cross-selling refers to selling more products or services to a customer, while upselling refers to strategies aimed at increasing the value of sales to a customer. For example, a restaurant performs cross-selling by offering an extra dish, and upselling by suggesting a more expensive dish than the one originally chosen. This module helps the marketing department implement these strategies by providing all relevant customer data and analyses.

Sales Automation The sales department was the first to adopt CRM, with one of the earliest components developed: *Sales Force Automation (SFA)*. SFA is a module used by sales agents to support them throughout the sales process: customer identification, contact management, offer definition, and purchase recording. Sales support modules now manage all channels and are divided into:

- *Sales management*: the sales management module automates or supports all the phases of a sales process illustrated in Figure 6.12. In particular, the module handles contact selection, assists the sales agent in drafting the offer and contract, and, by recording all operations performed by agents, enables performance analysis. The web channel provides all functionalities for online purchases: catalog management, shopping cart management, order finalization, and payment.

- *Contact management*: handles customer data and identifies potential customers for future sales. The contact management module also takes into account additional information that can help improve customer relationships.

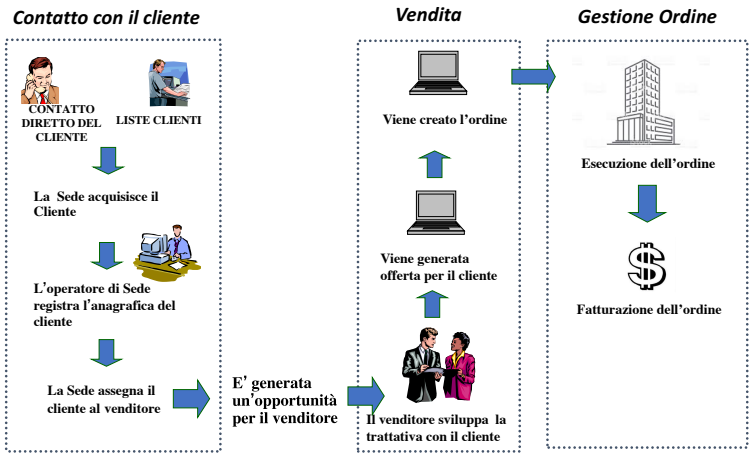


Figure 6.12: Main steps of a sales process

For example, equipping a call center with functions to identify the caller and retrieve notes on previous interactions can allow greeting the customer by name and personalizing the conversation based on past communications—this can make the customer feel valued and further build loyalty.

- *Opportunity management*: this module aims to find new customers or organizations for future sales. Its functionalities are also designed to calculate the likelihood of a sale associated with a customer, saving time for sales agents and marketing on analyzing and contacting unprofitable leads. The main difference between the contact management and opportunity management modules is that the former deals with people and organizations who are already customers, while the latter focuses on new leads.

Customer Service Sales and marketing are the business units that interact the most with customers before a sale. For this reason, the support modules for these activities are considered among the most important in CRM. Unfortunately, many organizations overlook the importance of continuing to nurture customer relationships after the sale has been made. Maintaining relationships in the post-sale phase is often even more important for strengthening customer ties and building loyalty. For this reason, CRM also offers a Customer Service module, which consists of the following submodules:

- *Contact center*: contact centers (or call centers) are responsible for managing inbound calls (customers calling the company) and outbound calls (company calling customers). CRM records all customer interactions with the call center and provides features for caller identification and call handling. Common functionalities used by call centers include (i) automatic call distribution, (ii) IVR (Interactive Voice Response), where users provide or receive information

through phone keypad input, (iii) predictive auto-dialing, which automatically calls customers and selects the right operator to handle the call.

- *Web-based self-service*: allows customers to use the web to find solutions to their problems or to contact the company (e.g., forms or online chat).

- *Call scripting*: this module manages knowledge by providing operators access to a database containing solutions to common customer problems. The system can also provide a list of questions that the operator can ask to identify the root issue and appropriate solutions.

6.4.2 Analytical CRM

Operational CRM records all customer interactions in a unified database. Analytical CRM uses this data to analyze customer preferences and behaviors to extract meaningful patterns. This is enabled through data warehouses and data mining systems, described in Sections 7.1 and 7.2. These systems aggregate, analyze, and disseminate information quickly within the organization to support decision-making. Analytical CRM serves three main purposes: (i) *Reporting*, understanding who the customers are, their characteristics and preferences; (ii) *Analysis*, segmenting customers into categories (e.g., active, dormant, inactive); and (iii) *Predicting*, forecasting customer actions and desires. This helps identify new opportunities and deliver higher quality service (efficiency, responsiveness, attention to needs, etc.).

Analytical CRM sends analysis results to operational CRM. For example, it supports the Marketing module by providing customer segmentation and identifying potential campaign targets. It also supports Sales by recommending products/services based on customer preferences and purchase history.

6.4.3 Collaborative CRM

Collaborative CRM aims to compute metrics relevant to the entire company, such as customer profitability and satisfaction, and to share this data and insights from analytical CRM with internal stakeholders. These indicators help evaluate the CRM's impact (e.g., increased returns) and can be used to assess whether the investment met expected objectives.

6.5 Questions

- Describe the general characteristics of ERP systems and discuss their representation in an ArchiMate diagram.

- What is meant by an ERP system? Describe its operating principles, benefits, and general functional architecture.

- Explain the functional architecture of ERP systems and discuss the pros and cons of make vs. buy approaches.

- Explain the role of data in an ERP system.
- Describe the characteristics of CRM systems and their functional architecture.

Chapter 7

Data Analysis

7.1 Data Warehousing

Data are an important resource not only at the operational level, but also at the tactical and strategic levels to support decision-making processes within an organization. This creates the need to access aggregated data and perform complex queries, such as analyzing correlations between different variables (for example, the relationship between a customer's place of residence and their likelihood of using a certain category of services). As one moves from the operational level to higher levels of Anthony's pyramid, both the type of data of interest and the way they are used change. For tactical and strategic activities to be successful, it is necessary to provide tools capable of extracting relevant information from different sources (both internal and external) quickly and effectively. These tools fall within the application area called *Business Intelligence (BI)*.

The first decision support tools were reports. Reports contain analytical data, obtained from analyzing the operational database, produced in a predefined format at regular intervals. These tools suffer from several limitations, as the data they contain are static and the user cannot interact with them to obtain details or additional information on unforeseen aspects. Additionally, generating reports can be complex and time-consuming, making it impossible to access up-to-date information. Finally, reports provide a limited view of aspects relevant to strategic decisions because they are based only on the organization's internal data extracted and aggregated from the operational database.

A more advanced and in some ways more dynamic tool is the spreadsheet (e.g., Excel). In this case, analyses can be defined directly by the end user, who can modify and execute them as needed. Despite these advantages over reports, spreadsheets are still limiting due to the complex processes required to export data from the database and import them into the spreadsheet. Moreover, each user can create a personalized tool that, although flexible, lacks the benefit of integrity and correctness checks typical of tools shared among multiple users.

A tool capable of supporting activities at the higher levels of Anthony's pyramid

is a database with characteristics different from traditional operational databases:

- The database can include data from operational databases and external sources.
- The data are organized in a structure that facilitates analysis.
- The database structure highlights all and only the relevant data.
- Data sources are integrated and updated.
- Analysis tools are available with relatively short response times, considering the objectives of the analysis.

To meet these needs, the concept of a *data warehouse* [19] was introduced.

7.1.1 Data Characteristics

A data warehouse is an OLAP system (Online Analytical Processing, introduced earlier). This differentiates data warehouses from traditional DBMSs, which are based on OLTP (Online Transaction Processing) technologies with many short online transactions (using INSERT, UPDATE, DELETE operations).

Analytical data used to support strategic and decision-making activities, as well as some planning and control activities, differ from operational data in several ways:

- *Objective*: data are used to make decisions about the organization's future developments and to identify organizational issues or growth opportunities.
- *Users*: data users are high- and mid-level managers who use information to improve organizational performance.
- *Time Horizon*: while operational data mainly reflect the present, analytical data incorporate historical data related to current data to identify issues, trends, and periodic patterns.
- *Level of Detail*: analytical data are usually aggregated from operational data or other sources at different levels of granularity.
- *Access*: unlike users of operational data, analytical data users access the information in read-only mode.

Given these characteristics, the classic relational model is not sufficient for building an analytical database.

To represent data in an OLAP system, the *multi-dimensional model* is used. In this model, information is represented through the concept of a *hypercube*, formed by a variable number of *dimensions* N, where each dimension is an analysis axis for the database. Each element is a database object, that is, a record whose value is defined by the coordinates represented by the analysis dimensions. The components of a multidimensional database are:

- *Fact*: a hypercube element obtained by specifying a value for each dimension.

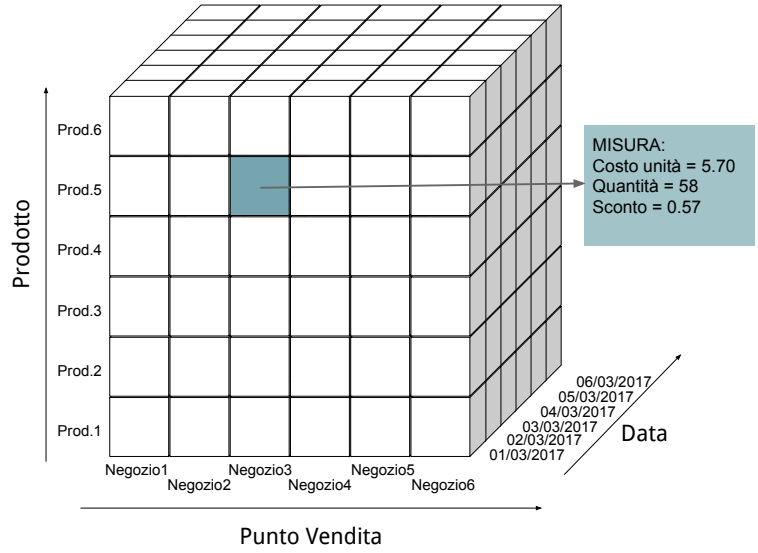


Figure 7.1: Three-dimensional hypercube representing key aspects for sales analysis in a supermarket chain

- *Dimension*: coordinates of each element corresponding to analysis dimensions.
- *Measure*: quantitative value of the elementary fact.

An example of a three-dimensional hypercube for analyzing sales in a supermarket chain is shown in Figure 7.1, where the analysis dimensions are time, product, and store location, while the fact is the sale and its measures include quantity sold, unit cost, and discount applied. Each dimension represents one of the coordinates of the fact and has a discrete value domain that defines the set of relevant facts. When a dimension's values are not discrete, they must be discretized, for example, by applying thresholds to define value intervals. Dimensions can be numerous and organized into hierarchies, where the level of granularity varies from level to level. Dimension hierarchies are organized such that a functional dependency exists between a higher granularity level and a lower one, such that a $1...N$ relationship exists between them. In the example of Figure 7.2, products are divided into several categories. Each category contains one or more product types, and each type includes multiple specific products. Along a hierarchical line, relationships define different aggregations for the dimension considered. Returning to the hypercube in Figure 7.1, the product dimension can be manipulated by changing the level of granularity—for instance, considering product groups by category rather than individual products. In this case, the fact content will aggregate information for all products in the selected category. Similar operations on other dimensions, their granularities, or values allow selection of the desired level of detail for analysis.

Figure 7.2: Example of a hierarchy for the product dimension. Granularity increases from right to left.

7.1.2 Data Warehouse

The data warehouse is a database that collects summarized, integrated, structured, and historical data of interest for an organization. More broadly, data warehousing refers to the entire set of operations involved in populating, maintaining the database, and querying it to extract relevant information from the decision-support database. As mentioned, the data warehouse is an OLAP (On-Line Analytical Processing) system. As described in Chapter 1, the OLAP paradigm is characterized by a few transactions and complex queries requiring aggregation of large data volumes. OLAP systems are evaluated by response time efficiency and effectiveness. An OLAP system stores aggregated data, historical data, and uses multi-dimensional schemas (typically star schemas). Normally, queries access large amounts of data to answer complex questions, such as the net profit achieved by a company in a given geographic area last year.

OLAP systems support complex and ad hoc operations, where each operation may involve many aggregated, historical, and often non-current data. For this reason, the ACID (Atomicity, Consistency, Isolation, Durability) properties are not critical, since operations are read-only.

Instead, OLAP systems adhere to the FASMI properties (Fast, Analytical, Shared, Multidimensional, Informational). Thus, the system must offer a multidimensional view of data (*multidimensional*) containing all relevant information (*informational*), enabling complex analyses (*analytical*) by multiple users with varying access rights (*shared*) and returning answers quickly (*fast*).

Moreover, a data warehouse must have the following features:

- **Subject-oriented**: focuses on key analytical entities. For example, sales, orders, and products may be analytical entities.
- **Integrated**: data are collected from both internal and external sources. Starting from heterogeneous data, they must be made consistent and integrated to provide a complete and clear view of the analyzed entities.
- **Time-variant**: the data warehouse stores historical data. To analyze event timelines, each data point is tagged with a timestamp indicating its reference period.

- **Non-volatile**: once stored in the data warehouse, data are typically not modified. Data are archived in read-only mode.

7.1.3 Data Warehouse Architecture

Data warehousing is the set of activities that lead to the definition, construction, and maintenance of information in the data warehouse, which can be defined as the database that collects all data of interest to the company, synthesizing, integrating, and structuring them. Within the data warehouse, it is possible to identify several databases organized hierarchically. At the first level, we have the *sources*, i.e., the origins from which the data populating the data warehouse must be extracted. These sources certainly include the organization's operational database, but also other databases or external sources that may be structured in various forms (databases, reports, big data, etc.). All these sources undergo a set of operations called *ETL* (Extraction, Transformation, and Loading), which allow for extracting the relevant data from the sources, transforming them according to the desired multidimensional structure, and loading them into the data warehouse. This population phase may use an intermediate database between the sources and the actual data warehouse, called the *staging area*. The data warehouse is the multidimensional database containing all the information of interest accessible by specifying the analysis dimensions. Depending on the presence or absence of the staging area, the DW architecture can be two-level (without staging area) or three-level (with staging area). Downstream of the data warehouse, there can be another type of database called a *data mart*. Data marts are small thematic data warehouses containing an extract of the information in the complete data warehouse. The latter can indeed have very large dimensions that are unlikely to be of interest to all system users. Using data marts, it is possible to define views of the data warehouse that contain a subset of the information. This subset may involve a reduction of the analysis dimensions (only those relevant to the target user category of the data mart are considered), the granularity level of the dimensions, or the selected time horizon. An architecture that uses only the data warehouse is a centralized architecture in which all data is contained in the sole centralized multidimensional database. On the other hand, using data marts enables a distributed architecture in which the different thematic databases can be located on different nodes.

As an example, we can refer to a supermarket chain that uses data warehousing to improve its operations. In the example, the information sources can be various. First, there are operational databases, in this case, the sales data and inventory databases. Also relevant for analysis could be information about advertising campaigns conducted by the chain. Other sources that can influence decisions may come from outside, such as weather data (sales of certain products may be influenced by weather conditions), and data from agencies specialized in sales analysis, such as Nielsen. This information is transformed and restructured using ETL transformations and then stored in the corporate data warehouse. Decisions can be made based on the interaction with the data warehouse, influenced by all the analyzed sources—for example, supply planning. Other areas, such as customer management or logistics planning, use a subset of the data in the data warehouse, which is why

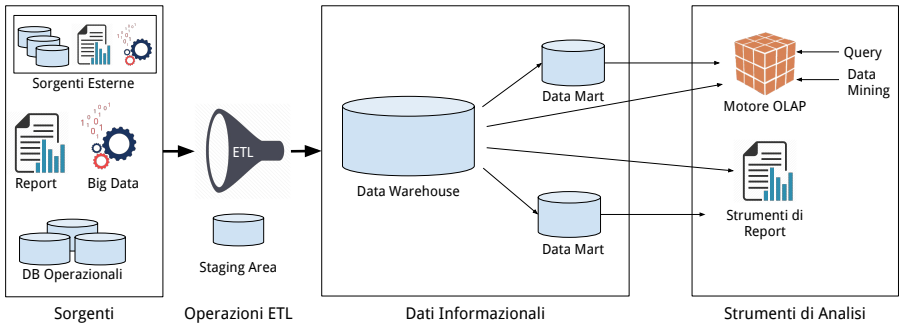


Figure 7.3: Three-level data warehouse architecture

thematic data marts can be created, queried through an OLAP engine, and used to extract automated reports.

ETL

Particular attention should be paid to the tools supporting the ETL process. These, as mentioned above, allow for the extraction, transformation, and loading of data into the data warehouse. As for *extraction*, it defines which data must be extracted and how they should be handled (e.g., whether to aggregate them at the source or extract them at the highest level of detail). Additionally, extraction can be: (a) *Static* if all data present in the sources are considered or (b) *Incremental* if only the data produced or modified since the last update of the data warehouse are considered. Following extraction, data needs *transformation* since it originates from heterogeneous sources and may be represented in different formats, making integration difficult. The main operations performed are:

- *Data cleaning*: Data extracted from sources often contains errors that must be corrected before insertion into the data warehouse. This operation is usually performed first in the transformation phase, though it may also be done after or during. The main data quality issues addressed at this stage are missing data, invalid values, and inconsistency between fields that should be related or have equivalent meanings.

- *Reconciliation*: The integration of different data must be preceded by this operation, which aims to relate data referring to the same "object".

- *Format standardization*: Data must be made homogeneous through standardization operations. These operations refer to (i) Joining or splitting fields: information present in multiple fields is combined into one, or information in one field is split into separate fields; (ii) Standardizing classification codes: different conventions for the same criterion are aligned to a single coding; (iii) Standardizing data formats: data represented in different formats are made consistent.

- *Duplicate detection and removal*: When duplicate information is present, this operation identifies and reduces it to a single instance.

Loading is the phase that transfers the data into the data warehouse according to the multidimensional model introduced in Section 7.1.1.

Metadata

The functions performed by ETL tools are supported and documented by metadata (data about data). Metadata is an important part of the architecture of a data warehouse and is usually stored in a repository that includes:

- *Data warehouse structure*: Metadata describes the structure of the data warehouse (schemas, views, dimensions, hierarchies, decomposition into data marts, and their locations).
- *Operational metadata*: Refers to data history and documents their origin and the transformations they underwent.
- *Metadata mapping operational data to data warehouse data*: Provides information on the sources and their content, transformation rules, and data warehouse update rules.
- *Usage statistics*: Metadata also describes how and how often the data warehouse is used.

7.1.4 Conceptual Model of the Data Warehouse

As mentioned earlier, data warehouses are based on the multidimensional data model, in which facts—elements of analysis—are represented through a set of measures, defined by dimensions that are the coordinates of the fact.

There are several conceptual models to represent multidimensional systems. The most widely used is the Dimensional Fact Model (DFM). In DFM, a fact is represented as a rectangle containing the corresponding measures. Dimensions are represented as labeled circles connected to the fact itself. Dimensions may be simple attributes of the fact or hierarchies, represented as trees, with the base dimensions as roots. Some hierarchy attributes may be optional, which is indicated in the DFM by a slash on the corresponding attribute line.

An example of a DFM representing the sales fact for the aforementioned supermarket chain is shown in Figure 7.4. The fact in this case is sales, with measures such as quantity sold, unit price, and any discounts. The fact is determined by four dimensions: the date the sale occurred, the location of the sale, the product sold, and the customer. Each dimension has an associated hierarchy, allowing for more or less detailed analysis of the facts stored in the data warehouse. For instance, it is possible to analyze the total quantity of sales for a specific product over the entire analysis period for stores in the city of Milan, grouped by customer gender. These insights result from aggregating across the various hierarchies of the

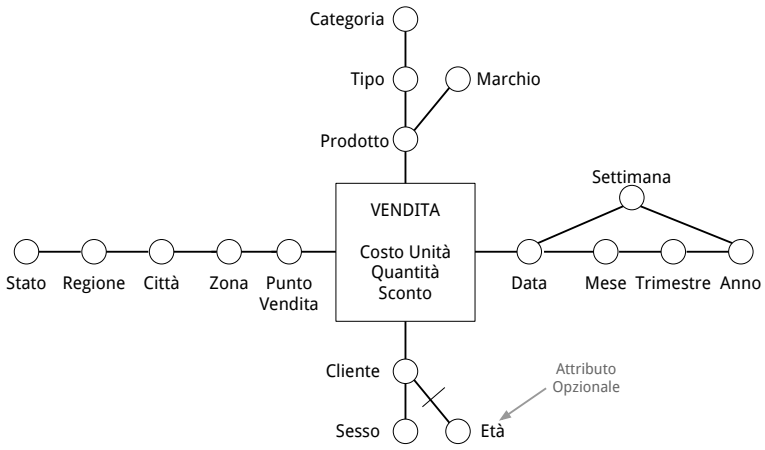


Figure 7.4: Dimensional Fact Model for supermarket sales

considered dimensions. In the modeled DFM, the age attribute for the customer is optional.

The DFM shown as an example corresponds to the hypercube relating to sales for the supermarket chain. However, the data warehouse will not consist of only one hypercube but of many, each modeling the dimensions related to an analytical fact of interest. For example, a company's data warehouse concerned with analyzing sales, customer support efficiency, and supplier interaction optimization will contain three different hypercubes—one for each analysis area: sales, customer service, and supplies. A DFM will be modeled for each of these hypercubes, defining the fact, measures, dimensions, and relevant hierarchies.

7.1.5 Logical Models of the Data Warehouse

Once the conceptual model of the multidimensional database has been defined, it must be implemented by translating the conceptual model into a logical model. It is therefore necessary to choose which DBMS to use to manage the multidimensional information.

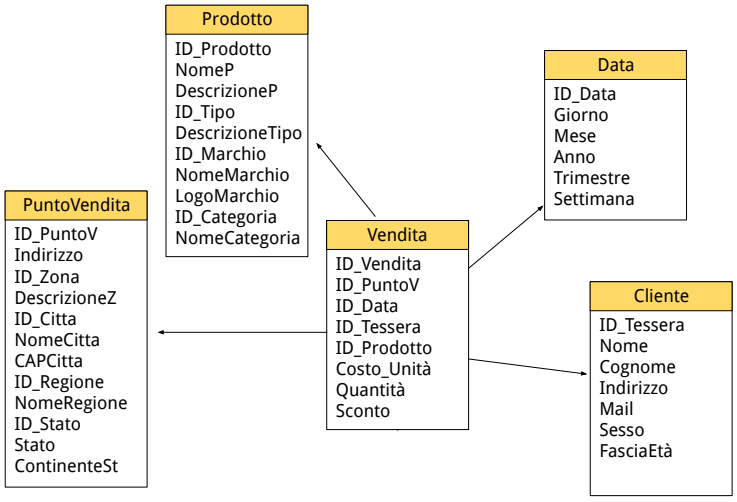
MOLAP Models The most direct choice might be to use a logical model of type MOLAP (Multidimensional OLAP). The MOLAP model translates the conceptual model into a multidimensional database that can be queried using multidimensional engines specifically designed to handle this type of data. This model has the advantage of exactly translating the conceptual model and of leveraging all the benefits of a multidimensional database, making queries efficient and fast. On the other hand, however, multidimensional databases are much less widespread than traditional relational databases and using them requires specific knowledge that the average user does not possess. Moreover, they are mostly proprietary models and languages

for which no widespread standard has been defined. Finally, the multidimensional model is based on the concept of a hypercube, in which a memory cell must be allocated for each possible combination of dimension values to store the corresponding fact, even if this fact did not actually occur and the cell remains empty.

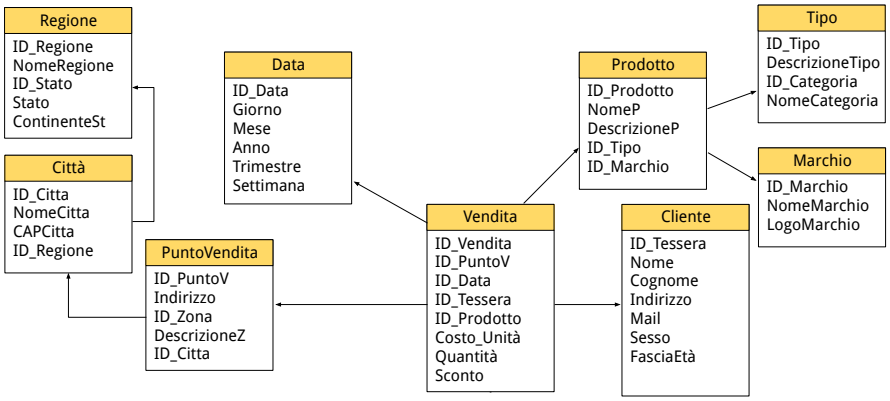
ROLAP Models To overcome these problems, an alternative to the MOLAP model is the use of a ROLAP (Relational OLAP) conceptual model. ROLAP models translate the multidimensional conceptual model into a relational model that represents the data using tables and allows queries to be performed using familiar SQL-based query languages, by performing JOIN operations on the tables representing the fact and its dimensions. The main advantages of this model stem from the familiarity with relational database querying tools, which allow users to interact with the data warehouse without specific knowledge. Furthermore, being based on the relational model, this representation requires less storage space than the MOLAP model, since each fact corresponds to a row in the database and it is not necessary to reserve space for facts that have not been recorded. However, this model has some disadvantages due to the difficulty in adhering to the data's conceptual model and the slowness and awkwardness of queries on a relational engine that is not well suited to handling multidimensional data.

HOLAP Models The HOLAP (Hybrid OLAP) model translates the multidimensional model in a hybrid way, usually using a relational database for the data warehouse, limiting space usage and allowing interaction with familiar tools. Multidimensional databases are instead used for thematic data marts, where the dimensions are inherently limited in number, but which gain the advantage of more efficient queries.

Multidimensional Schemas on Relational Databases In order to map a multidimensional database into a relational schema, it is necessary to identify the tables that will compose this schema. In the literature, two main approaches exist for this purpose: the *star schema* and the *snowflake schema*. In the star schema, two types of tables are used: the fact table and the dimension tables. The fact table contains all the attributes corresponding to the measures of the fact, and each row of this table thus corresponds to a specific fact. The dimension tables, on the other hand, contain for each dimension associated with the fact in the multidimensional schema, all the attributes related to the hierarchy corresponding to the dimension. A query will thus translate into a JOIN among all the involved tables based on the fact of interest. However, in this way, the concept of dimension hierarchy is lost, as it is flattened within a single table. An alternative approach is to use a snowflake schema, in which each dimension is associated with multiple related tables, allowing the preservation of some of the most relevant functional dependencies for users. A comparison between the two schemas is shown in Figure 7.5, where the star and snowflake schemas are presented in relation to the multidimensional model whose conceptual model is shown in Figure 7.4.



(a) Star Schema



(b) Snowflake Schema

Figure 7.5: Comparison between star schema and snowflake schema to represent the Sales fact in a supermarket chain

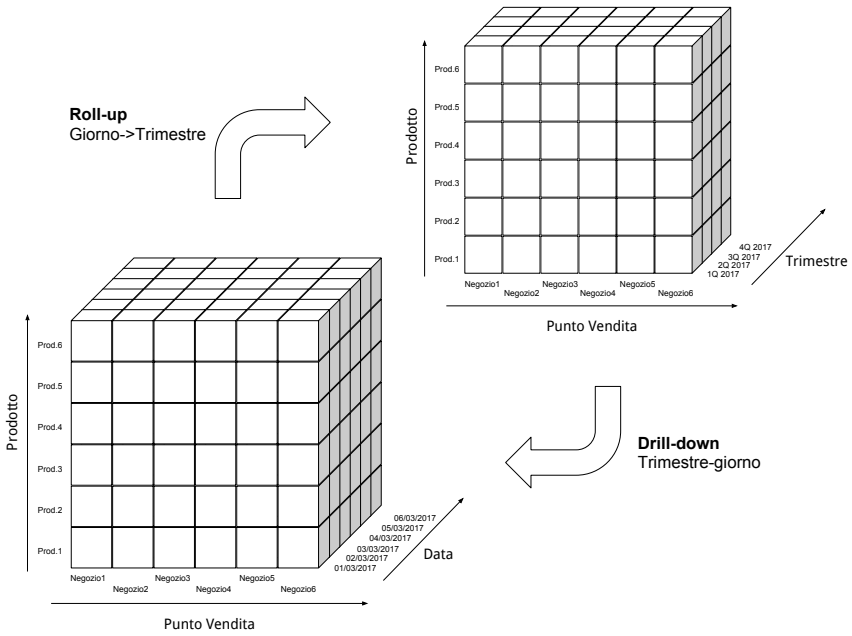


Figure 7.6: Example of roll-up and drill-down with aggregation/disaggregation on the *Date* dimension

7.1.6 Data Warehouse Operations

Data warehousing includes, in addition to a model of managerial data, a set of techniques that can be used to analyze the data. These techniques must allow even non-expert users to interact with the hypercube of facts to obtain more or less detailed information based on their needs, focusing on the dimensions of interest. The interaction usually begins with a hypothesis formulated by a user that leads to the extraction of the subset of data of interest, which can be explored further through successive interactions. Each interaction consists of applying an OLAP operator. The main OLAP operators are: drill-down, roll-up, slice, and dice.

The **drill-down** operation allows obtaining more detailed data by descending along a hierarchy of a dimension, thus moving from a higher level of aggregation to a lower one. Considering the DFM in Figure 7.4, a drill-down operation might go from a quarterly to a daily analysis of sales (see Figure 7.6). The opposite operation to drill-down is **roll-up**, where the level of detail is reduced by moving to a coarser level of granularity along an analysis dimension. An example could be moving from a monthly sales analysis to an annual one, in which monthly values are aggregated. Alternatively, a roll-up operation can be performed by eliminating an analysis dimension that is no longer relevant. For example, from a monthly sales analysis, one might remove the store dimension and aggregate based on the remaining dimensions (see Figure 7.7).

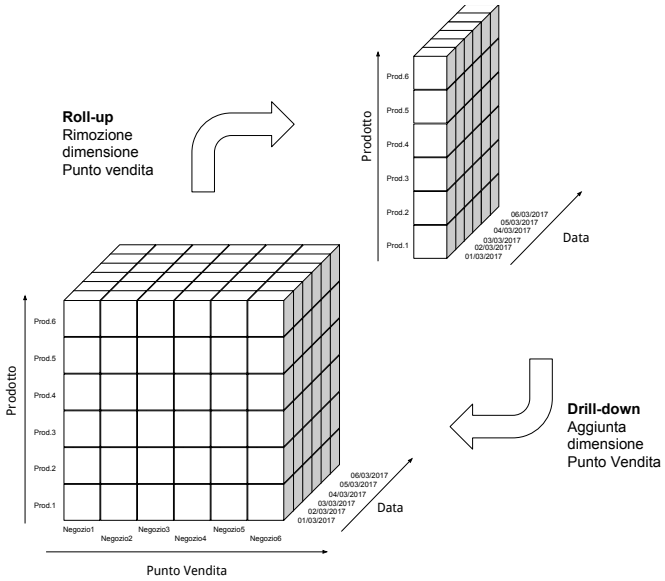


Figure 7.7: Example of roll-up and drill-down with elimination of *Store* dimension

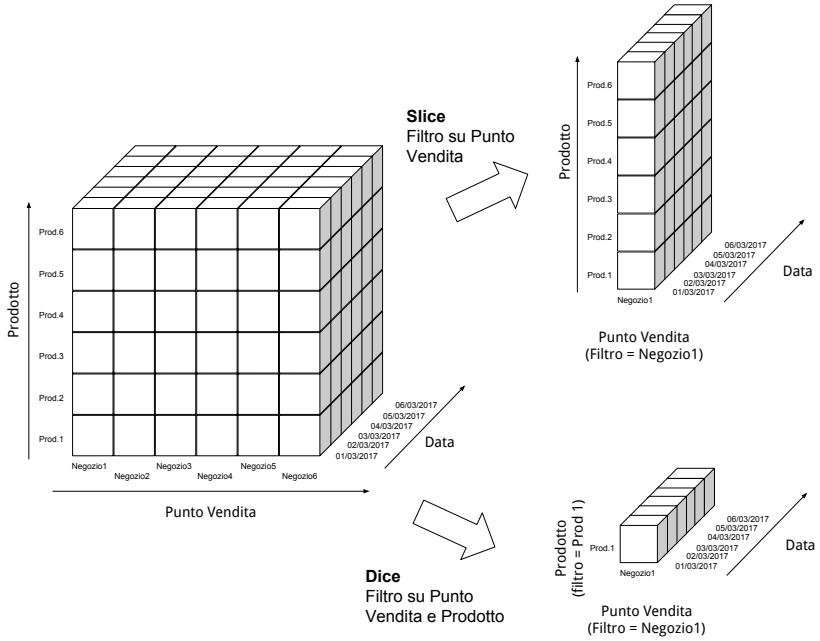


Figure 7.8: Example of slice and dice

The **slice** operator allows focusing the analysis on a portion of the data by fixing the value for one of the analysis dimensions. For example, one can focus on sales related to a single store, while maintaining all the details related to the other analysis dimensions. The result of this operation is a slice of the original hypercube, identified by the fixed dimension (see Figure 7.8). Note that, unlike roll-up, the slice selects a single store and does not aggregate over it. Another reduction operation is **dice**, which identifies a set of coordinates at any hierarchical level for any desired dimension. The result in this case is still a hypercube, but with a reduced set of data. For example, one could analyze sales data for a product (e.g., *product1*) in a specific store (e.g., *store1*).

7.1.7 Life Cycle of the Data Warehouse

As stated in Section 7.1.4, an organization's data warehouse will consist of multiple hypercubes, each containing relevant information for a fact of interest in the organization's managerial oversight.

The construction of the data warehouse thus follows an iterative approach. It begins by identifying the most relevant fact, then the first hypercube is modeled and populated. Subsequently, other hypercubes of interest are added and populated one by one. This approach has a dual advantage: on the one hand, it allows identifying relevant facts later, in response to new organizational needs; on the other hand, it limits the initial investment by developing the multidimensional database piece by piece.

The population phase of the data warehouse is a complex step that requires several operations. As already mentioned, ETL (Extraction, Transformation, and Loading) tools are used to populate the DW (see Section 7.1.3), where the final phase involves *loading* the data into the data warehouse. The data is inserted starting from the highest aggregation level and then descending to the more detailed levels.

During insertion, it is important to consider issues related to data updates, which may concern facts or dimensions. Updating a fact means inserting a new value for a combination of dimensions, or modifying an existing value. Updating a dimension, however, can lead to inconsistency. For example, if a customer's address dimension changes over time, one of the following decisions must be made: (1) update the address value for existing facts so that a customer is always associated with the same address; (2) retain the previous value without updating; (3) adopt a hybrid solution that tracks the different addresses over time and associates them with their respective time intervals. The last solution is the most complex to manage.

The data warehouse life cycle also highlights its limitations. It is important to note that the ETL process can take considerable time to execute. It is not necessarily an automatic task. Especially during the transformation phase, where discrepancies between data from different sources must be resolved, human intervention may be required. Therefore, not only is the creation of a data warehouse a costly operation, but its maintenance and continuous data updating also require significant effort.

This complexity results in an increased gap between updates in the source databases—which occur very frequently—and the moment when those updates be-

come visible in a data warehouse. To reduce this gap, in recent years, alternative approaches to data warehouses have been proposed. These often leverage big data algorithms and techniques that, although they do not require such a detailed design phase as in the case of the data warehouse, are capable of producing real-time analytical results.

7.2 Data Mining

Data warehousing supports the user's deductive processes by providing a set of tools for real-time interaction with the multidimensional database and for data navigation. This interaction allows the user to perform data analysis in search of specific information considered of interest by that particular user. This approach to directional data analysis within an organization can be insufficient, and further analyses aimed at automatically extracting hidden information inside the data warehouse can enrich the set of knowledge that can guide business strategies.

As defined in [19], *data mining* is the activity aimed at automatically recognizing and extracting hidden information from large databases, such as the data warehouse. The data mining process consists of several steps that can be iterated multiple times in order to refine the obtained information. This process, although largely automatic, requires user interaction in several phases. The user can indeed guide the iterative process by indicating the most interesting investigation directions. Moreover, even the final results offered by data mining analyses must be validated and selected by users, since part of the resulting information might prove to be of little interest.

In general, the data mining process consists of the following steps:

1. Selection of the data considered relevant for the analysis.
2. Data cleaning to eliminate inconsistencies and errors.
3. Integration of data coming from different sources to make them consistent.
4. Transformation and structuring of the data resulting from the previous phases.
5. Data mining analysis aimed at searching for noteworthy conditions within the analyzed data.
6. Evaluation of the results and their reduction to only those considered relevant.
7. Presentation of the final results to the user.

Steps 1 to 4 correspond to the steps necessary for the creation of the data warehouse, while steps 5 to 7 are typical of data mining alone. The architecture of data mining is shown in Figure 7.9.

Data mining functions can be classified into different categories [19, 31] briefly discussed in this paragraph. First, considering the possibility of training algorithms with ideal examples, data mining techniques can be distinguished into *supervised* and *unsupervised* learning techniques. *Supervised learning* techniques associate to

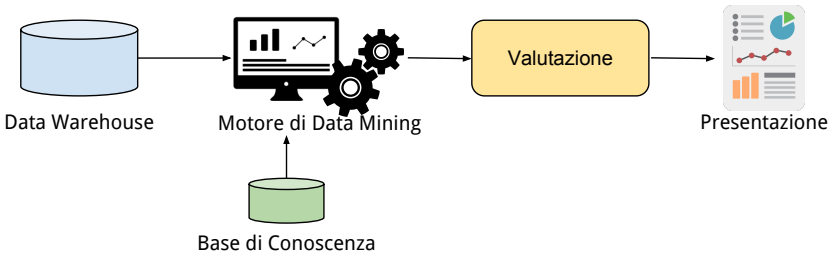


Figure 7.9: Architecture of Data Mining starting from the data warehouse

each considered record a target variable which can be, for example, a classification label of that data set. The operation of associating the record and the classification label is called "labelling." The target variable constitutes the desired output from the analysis of the record that is the input to the system. In *unsupervised learning* techniques, there is no target variable, and consequently, the goal is to identify meaningful recurring regularities or rules.

Another classification can be made considering the analysis objectives, distinguishing *predictive* and *descriptive* techniques. *Descriptive techniques* analyze past data and categorize, filter, and identify patterns by applying mathematical or statistical functions. *Predictive techniques*, instead, analyze past data to support predictions and decisions capable of anticipating events.

Considering the provided functionalities, we can identify several data mining techniques. Those described in this text are association rules, classification, and clustering. According to the categories described, association rules and clustering are two descriptive techniques with unsupervised learning, whereas classification is a predictive technique with supervised learning.

7.2.1 Characterization and Discrimination

Characterization and discrimination algorithms aim to extract relevant information that can precisely and concisely describe the data contained in the analyzed database.

In this category, for example, there is **generalization**, which starts from the principle that the number of elements a user can analyze is limited. The goal is thus to reduce the level of detail to reduce the number of elements to analyze. This reduction can be performed by generalizing the analyzed elements by finding common characteristics within the dataset. The reduction can be performed using aggregation techniques as those used in data warehouses (roll-up), or, if the attribute hierarchy is unknown, by removing attributes that have too high variability. For example, consider a case where a user wants to analyze sales data, considering revenue as the measure. The analysis dimensions could be address, city, and age group of customers, and day of sale. The address will be an attribute with high variability for which the dimensional hierarchy is unknown and can therefore be

removed. For the day of sale and for the city, instead, it is possible to roll up along the dimension until the attribute's variability remains below a certain threshold.

Other algorithms belonging to this category are **characterization** ones. In this case, the interest is to understand the nature of the data, thus studying their trends (maximum, minimum, mean, variance, etc.), the level of differentiation (for example, how many different values exist for a specific attribute of the database), and according to which distribution these values can be described. This type of analysis is often performed upstream to get an overview of the kind of data dealt with and their general characteristics.

7.2.2 Association Rules

Association rules describe the relationships occurring between different attributes of the database, aiming to identify conditions that frequently occur simultaneously. They are usually expressed with the formula $A \Rightarrow B$ where A is called the antecedent and B the consequent. Association rules can be used, for example, for market basket analysis, i.e., to identify products that a user often buys together. An example is the following rule:
$buys(X, cereals) \Rightarrow buys(X, milk)$

The validity of an association rule can be evaluated based on two values: support and confidence. *Confidence* measures the certainty of the found relation, i.e., how many times in the analyzed dataset, if the condition expressed in the antecedent occurs, the consequent also occurs. In probabilistic terms, this relation can be expressed as the conditional probability $P(B|A) = \frac{P(A,B)}{P(A)}$, that is, the ratio between the number of times A and B occur together (a purchase includes both milk and cereals) over the number of times A is observed (a purchase includes cereals). The *support* instead indicates the number of times the rule is verified over the total records in the dataset and is thus equal to $P(A, B)$.

As an example, consider analyzing sales data of products to find relationships between the sale of different products. Consider the following extract of the database:
$t_1 = \{p_1, p_2, p_3\}$
$t_2 = \{p_1, p_4\}$
$t_3 = \{p_1, p_3\}$
$t_4 = \{p_2, p_5, p_6\}$

where t_i represents a timestamp and p_i represents the sale of a product. If we wanted to extract the association rules that have both support S and confidence C greater than 0.5, we would obtain:
$p_1 \Rightarrow p_3$ with
$S = P(p_1, p_3) = \frac{2}{4} = 0.5$ and
$C = P(p_3|p_1) = \frac{P(p_1,p_3)}{P(p_1)} = \frac{\frac{2}{4}}{\frac{3}{4}} = 0.66$

$p_3 \Rightarrow p_1$ with
$S = P(p_1, p_3) = \frac{2}{4} = 0.5$ and
$C = P(p_1|p_3) = \frac{P(p_1,p_3)}{P(p_3)} = \frac{\frac{2}{4}}{\frac{2}{4}} = 1$

Both relations, being symmetrical, occur twice in t_1 and t_3 out of a total of four cases analyzed, so the support $P(A, B)$ in both cases will be 50%. Confidence instead depends also on the number of times the first term occurs in the analyzed cases. While p_1 occurs three times out of four (once independently of whether p_3 occurs), p_3 occurs only twice and always in conjunction with p_1. For this reason, the confidence of the two rules, calculated as $P(A, B)/P(A)$, is different.

Possible applications of association rules include planning effective promotional campaigns (suggesting products based on customers' previous purchases) and triggering preventive maintenance actions on equipment. An example of the first case can be seen on online shopping websites, where the display of a product is always accompanied by a list of other products frequently purchased together with the viewed product.

7.2.3 Classificazione

Classification allows automatically assigning elements of the database to predefined classes. Classification is a guided procedure, starting from an annotated set of elements, i.e., for which the class membership is known, and building a model able to identify the characteristics that allow assigning an element whose class is unknown to one of the analyzed classes. This model is then tested with a dataset for which the class is known to see if the classifier can correctly identify the category. The set of annotated elements used to build the model is usually called the *training set*, while the set of elements used to test the classification results is called the *test set*. The quality of the model is measured by its ability to correctly classify the elements composing the test set. Once a satisfactory model is obtained, it can be used on unclassified data to make a prediction on the class membership. Possible applications of classification include predicting the quality of suppliers or products, and predicting customers' propensity to purchase.

There are different types of classifiers based on various techniques including mathematical functions, Bayesian networks, decision trees, and neural networks. They differ according to a set of characteristics: (i) Accuracy — percentage of correctly classified elements; (ii) Speed — time to build the model and time for classification; (iii) Scalability — classifier's ability to build a model with an increasing number of variables; (iv) Robustness — classification ability in the presence of missing elements; (v) Interpretability, i.e., ease of interpreting the model results.

As an example of a classification model, consider *decision trees*, in which nodes are the attributes of the subject to classify, edges are the possible values of the attribute, and leaves are the classes to which elements can be assigned. An example of a decision tree is shown in Figure 7.10, where the goal is to predict whether a user will purchase an OLED television or not. The considered attributes are the customer's age (divided into age groups), their education level (university degree or not), and their residence (city center or suburb). The tree, built based on data relating to previous purchases of other users, can be used to decide whether to direct specific advertising for the product based on customer characteristics.

The construction of a decision tree proceeds by successive refinements. Starting

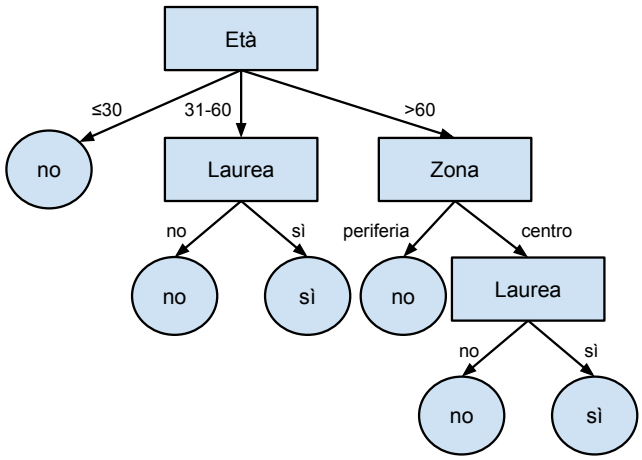


Figure 7.10: Decision tree for predicting the purchase of an OLED television

from a labeled training set, an attribute is selected as the root node (in the figure, age) and the dataset is split based on the different values assumed by that attribute (in this case discretized into three groups: less than or equal to 30 years, between 31 and 60 years, and over 60 years). The choice of the attribute is driven by evaluations related to *entropy*, which measures the amount of information extractable from the attribute, aiming to have a high entropy jump (for example, information gain) between tree levels and minimal entropy, ideally zero, for the tree leaves.

For each subclass, a further attribute is selected and the same operation is repeated. The goal is to obtain, in the leaves, a set of cases all belonging to the same assignment class (in the example, "buy" or "not buy"). The procedure stops when the maximum depth set for the tree is reached or when the obtained subclass has very low variability so that further splitting does not produce a more homogeneous set.

The selection of attributes allowing an effective model with a reduced number of steps is therefore a very delicate phase. The example shown in the figure considers a **general tree**, where each attribute can assume any (but discrete) number of values. However, this condition is not general, and an attribute may have a fixed number of values (if only two values, it is called a **binary tree**). The greater the number of values, the more complex the model construction. In the case of continuous attributes, thresholds can be applied to convert them into discrete attributes. Moreover, the transition from one level to the next can depend on the evaluation of a single attribute (**univariate tree**) or on the combination of multiple attributes' values (**multivariate tree**).

As mentioned earlier, the obtained model can be evaluated based on its ability to classify elements of the test set. Depending on the choices made (order of selected attributes at each step), different decision tree models can be constructed for a dataset. The evaluation allows selecting the one best able to represent the

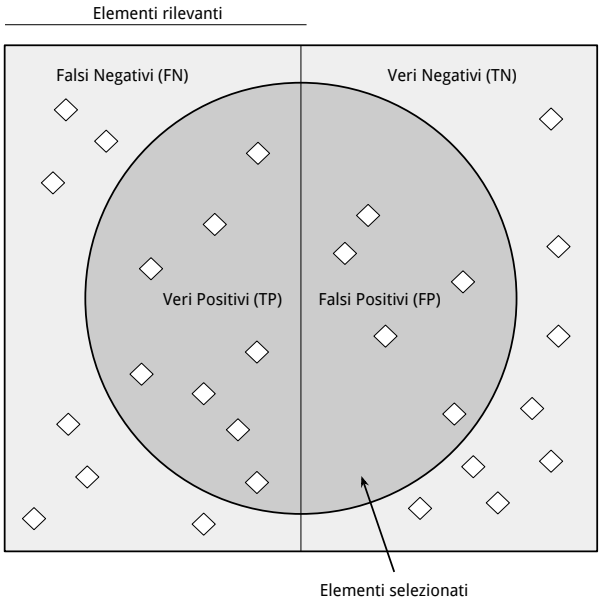


Figure 7.11: Illustration of possible classification outcomes: false positives, false negatives, true positives, and true negatives

considered dataset.

In this regard, the role of the *test set* is fundamental. Indeed, for each element, through the labeling operation, its correct classification is known. Returning to the television purchase example, for each individual, it is indicated whether they intend to purchase the television or not. A decision tree is therefore perfect when it can classify every element in the same class as labeled. Referring to the example in Figure 7.10, for each test set element under 30 years old, purchase of the television should not be predicted, otherwise it would conflict with the labeling information.

In reality, discrepancies always exist, and these are calculated starting from the analysis of so-called *false positives* and *false negatives* (see Figure 7.11). A false negative occurs when an individual is not classified as belonging to a class but should actually belong. For example, the tree predicts that a user does not belong to the class of buyers who will purchase the television, while in reality, in the test set, this individual is labeled as a buyer. Conversely, a false positive occurs if an individual is associated with the class of television buyers but will not actually buy the television.

Similarly, the so-called true positives and true negatives can be defined, which, unlike the previous cases, indicate when an individual is correctly inserted or not inserted into a class.

Once the total numbers of: (i) false positives (FP), (ii) false negatives (FN), (iii) true positives (TP), and (iv) true negatives (TN) for the entire test set are

calculated, *Precision* and *Recall* are defined.

Precision is the percentage of elements correctly classified among all those classified as belonging to a class:

$$Precision = \frac{TP}{TP + FP}$$

Intuitively, Precision indicates how accurately a decision tree inserts individuals into a class. For example, a Precision of 66% indicates that in two-thirds of the cases, an individual identified as a potential television buyer actually becomes one.

Recall indicates how many elements the classification algorithm was able to associate with the class compared to those that ideally constitute the class:

$$Recall = \frac{TP}{TP + FN}$$

Referring to the example above, a Recall of 50% indicates that the elements the decision tree inserted in the group of potential television buyers constitute only half of the actual set of people who will really buy the television.

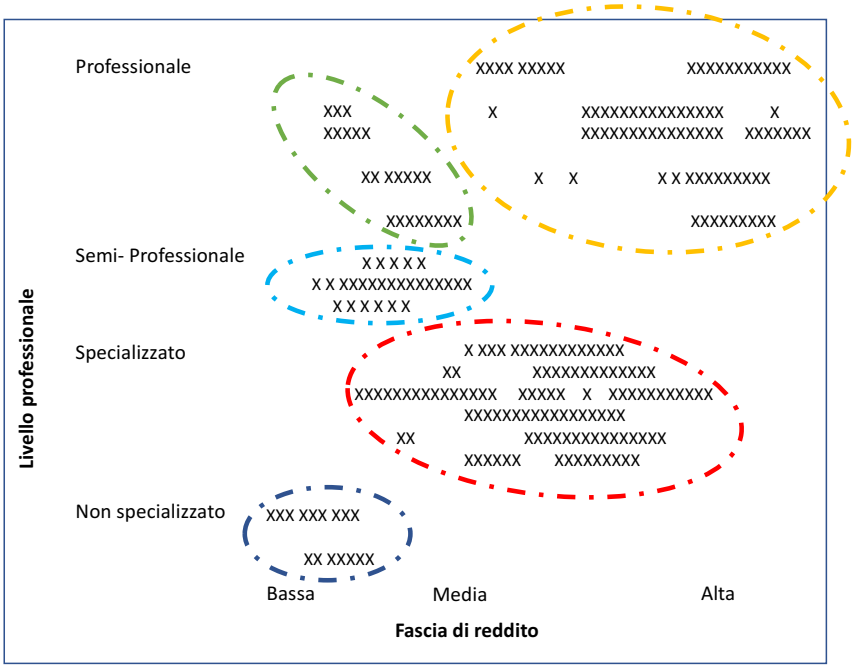
7.2.4 Clustering

Clustering, like classification, allows grouping a set of elements based on their characteristics by assigning them a class membership. The main difference between the two approaches is that clustering does not require knowing the training set class memberships in advance but finds the classifications and sometimes even their number automatically based on the similarity of the attribute values of the dataset elements. The clustering result is thus a set of clusters (groups) composed of elements satisfying the following conditions: (i) similarity among elements within the same cluster is maximized; (ii) similarity between elements of different clusters is minimized.

Clustering methods are applied in various fields such as social sciences, image recognition, and marketing analysis. They are based on the human brain's ability to group elements based on affinity. For example, a set of customers could be grouped via clustering based on their product purchasing behavior.

To calculate the distance between elements in a set, the attributes describing the elements must be numeric. The main requirements by which clustering algorithms are evaluated are: (i) flexibility — ability to analyze categorical as well as numerical elements; (ii) robustness — stability of results in the presence of slight variations in attribute values; (iii) efficiency — time required to obtain the dataset partition into clusters.

The most widespread clustering algorithm is the *k-means* algorithm, an unsupervised technique. It requires (i) prior knowledge of the number k of clusters to identify, (ii) a similarity function indicating the closeness between two elements. The similarity function usually depends, through weighted sums, on the attributes defining an individual in the analyzed set. Thinking of supermarket customers,



Mercato Potenziale (Universo): persone con più di 16 anni
Segmentazione: demografica (variabili multiple)
Variabili: Livello professionale e fascia di reddito

Sorgente dati: ipotetica

Figure 7.12: Example of clustering ($k = 5$) used for market segmentation

their similarity could depend on age, geographic area of origin, and profession. By calculating the difference between these dimensions' values and weighting their importance, one can quantify how similar one individual is to another.

The similarity function is applied iteratively to the set for which clusters are to be identified. The approach starts by identifying the centroids' positions, one for each cluster, representing the point with coordinates corresponding to the mean attribute values for the elements in a set. Initially, centroids are positioned randomly. Dataset elements are then assigned to a cluster based on their distance from each centroid. For each identified set, centroid positions are recalculated, and element assignments are repeated. The algorithm stops when the system converges (centroids have a stable position and the assigned elements to sets do not change from one iteration to the next).

An example of clustering is shown in Figure 7.12, where clustering is used for market segmentation, i.e., dividing customers into "social groups" that constitute market segments based on geographic, socio-demographic, or behavioral characteristics. In the example, customers are grouped based on two attributes: their income bracket (low, medium, high) and their professional level (unskilled, skilled,

semi-professional, and professional). Starting from the attribute distribution, five clusters are identified, each representing a segment. It is important to note that clusters are not assigned a clear category since they are not predefined, but each set represents elements sharing similar attribute values.

Clustering algorithms are classified based on element assignment to clusters. Specifically, an algorithm is **exclusive** if it assigns each dataset element to only one cluster, while it is **overlapping** if the same element can be assigned to multiple classes. **Fuzzy** methods are overlapping methods in which assignment of an element to each class is associated with a weight varying between 0 and 1. Methods are also classified as **complete** if every element belongs to at least one class and **partial** if some elements may not be assigned to any class.

7.2.5 Examples on CRM

Data mining techniques are used by analytical CRM to analyze customer actions and behavior and find significant patterns. For example, association rules are used to analyze different customers' purchases and find products that are often bought together. In particular, rules of the form $X \Rightarrow Y$ can be used to recommend Y to all customers who buy X. For example, consider the recommendation services offered by Amazon: when we visit a book's page, the site, under the section "Customers who bought this item also bought," suggests other books that other users have purchased together with the book we are viewing. This is the result of association rules that link the book we considered (X) to other books frequently bought together (Y). Association rules allow companies to be proactive and anticipate customer desires.

Classification methods (for example, decision trees) allow us, starting from a set of objects, their features, and their classification, to find the method and criteria through which to automatically assign objects to classes. These techniques have often been used in the insurance sector: given the clients, their history, and the risk category, methods have been defined to classify new clients into probable risk categories and determine an associated premium. Classification criteria in this sector include age, area of residence, and the type of insured vehicle. Alternatively, a company manually classifies clients based on their history into different categories such as:

- *Platinum*: reliable users, not price sensitive, willing to try new products, loyal.
- *Gold*: users who make many purchases but look for discounts, buy from multiple suppliers, and are not loyal.
- *Iron*: users who generate low business volume or are inconsistent in purchases.
- *Lead*: demanding users who require special attention but do not show loyalty.

By applying classification methods, it is possible to find customer characteristics to use as classification criteria in order to assign categories a priori and understand which marketing strategies to adopt.

Clustering is used to segment customers and form groups of similar customers. Consider a telephone provider that wants to segment its customer base by categories

of service usage. If customers can be classified by usage, the company can offer more attractive subscriptions to its clients. Variables are given by the use of the following services: Multiple lines, Voice mail, Internet, Caller ID visualization, Call waiting, Call forwarding, 3-way calling, Electronic billing. Applying clustering techniques leads to identifying three clusters:

- *Cluster 1*: customers who spend a lot and buy many services.
- *Cluster 2*: customers who tend to spend moderately on calling services.
- *Cluster 3*: customers who spend very little and buy few services.

Segmentation provides information for marketing campaigns and the offer of advanced services tailored for a specific category of customers.

7.3 Process mining

Business processes are a significant part of an organization's information system and provide a structured approach to achieve business objectives efficiently. However, it is necessary to ensure that processes are correctly modeled, guaranteeing the effective execution of business activities. Moreover, it is essential to monitor that the way activities are actually carried out within the organization reflects the behavior expected and formalized through process models.

Process Mining [29, 30] is an approach that allows the analysis of business processes based on monitoring data generated by their execution, helping organizations understand how processes are actually performed and highlighting any deviations from the original design. Process Mining discovers inefficiencies, bottlenecks, deviations, and opportunities for improvement in workflows, enabling data-driven decision-making for process optimization.

Process Mining is used to discover, monitor, and improve real processes. The essential ingredients of Process Mining are:

- the *monitoring data* related to the execution of activities composing a business process;
- the *process model* describing the order in which activities should be executed.

Based on these inputs, analyses can be performed oriented both to verify the quality of business processes (why are some activities delayed? what are the bottlenecks? which resources are over-utilized?) and the compliance of their execution (which activities are skipped? which exceptions are not properly handled?).

Process Mining can support all phases of a process's lifecycle. Process mining techniques can be classified as follows (Fig. 7.13):

- **Process Discovery**: a process model is generated from an event log without using any a priori information;

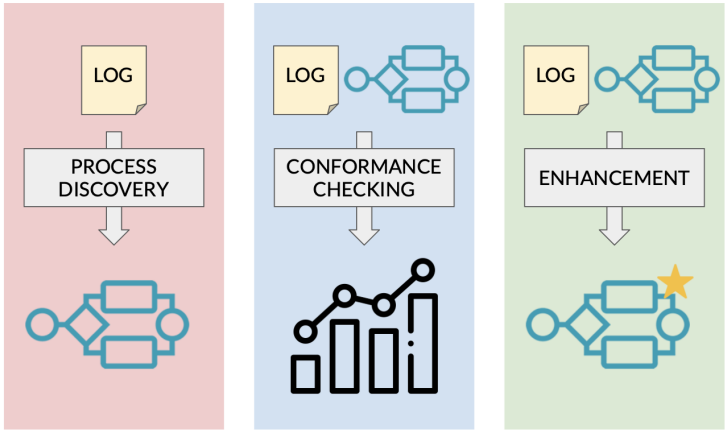


Figure 7.13: Main Process Mining techniques

- **Conformance checking**: a model is compared with an event log to verify whether reality conforms to the model (and vice versa);
- **Enhancement**: an existing model can be improved or extended using information provided by an event log.

7.3.1 Event Logs

One of the fundamental elements of Process Mining consists of event logs generated by processes. An event log is a collection of data recording individual events or activities occurring during the execution of a business process. Each event in the log is characterized by some fundamental information:

- Case ID: a unique identifier for the process instance;
- Activity: the name of the process activity to which the event refers;
- Timestamp: the date and time when the event occurred. It usually represents the moment the activity started but may also record the completion time.

Activities require time to be executed. The event can also record the status of each activity (start, completion, suspension, resumption, cancellation). Additionally, each event can also record information about the resources involved in carrying out the activity (personnel, system, manager, etc.) and additional information useful for analysis. An example is shown in Figure 7.14, where the additional information indicates the involved customer (Customer).

A log can also be organized as a sequence of traces. A trace is the list of activities executed for a specific process instance (Case ID), ordered by execution timestamp. In this case, the log is a set of traces corresponding to different executions of the

Case	Activity	Timestamp	Resource	Customer
...	...	...	...	...
pizza-56	buy ingredients (bi)	18:10	Stefano	Valentina
pizza-57	buy ingredients (bi)	18:12	Stefano	Giulia
pizza-57	create base (cb)	18:16	Mario	Giulia
pizza-56	create base (cb)	18:19	Mario	Valentina
pizza-57	add tomato (at)	18:21	Mario	Giulia
pizza-57	add cheese (ac)	18:27	Mario	Giulia
pizza-56	add cheese (ac)	18:34	Mario	Valentina
pizza-56	add tomato (at)	18:44	Mario	Valentina
pizza-56	add salami (as)	18:45	Mario	Valentina
pizza-56	bake in oven (bo)	18:48	Stefano	Valentina
pizza-57	add salami (as)	18:50	Mario	Giulia

Figure 7.14: Example of an event log related to a pizzeria process

process. This representation is very convenient to group together similar traces (characterized by the same set of activities in the same order) and analyze the most common and rare traces.

7.3.2 Process Discovery

In the section on process identification, we analyzed existing approaches that allow the organization to identify the processes characterizing its activities and highlighted how this activity is very complex and requires a high expenditure of time and resources to be carried out. Process Discovery is a Process Mining approach that supports the identification of processes through automated tools. The algorithms supporting Process Discovery allow reconstructing a process model exclusively from an event log, without relying on predefined models or hypotheses.

Process Discovery focuses on the control flow to transform the event log into the process representation considering only the activities and their sequence in the various traces. The resulting process model must allow all behaviors present in the log (each trace must be executable through the model).

Process Discovery starts from the analysis of traces in the event log from which it extracts the existing relations between pairs of activities. Possible relations are:

- Direct succession ($>$) – an activity is immediately followed by another ($x > y$) in at least one trace in the log;
- Causality ($\rightarrow$) – an activity can follow another but not vice versa ($x \rightarrow y$ if $x > y$ and $\sim (y > x)$);
- Parallel ($||$) – two activities are executed in no specific order ($x||y$ if $x > y$ and $y > x$);

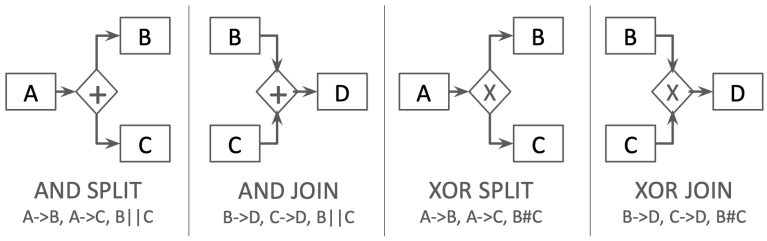


Figure 7.15: Process discovery: types of relations and BPMN gateways

- Choice (#) – the two activities are never executed in sequence ($x\#y$ if $\sim (x > y)$ and $\sim (y > x)$).

Without going into the details of Process Discovery algorithms, we can use the relations just described to translate them into the typical gateways introduced in BPMN notation, as shown in Figure 7.15.

7.3.3 Conformance Checking

Conformance checking is a process mining approach that consists in detecting any deviations between the intended behavior of a business process (expressed through a model) and its actual execution (recorded via event logs).

The comparison is performed via *replay* activity, which consists in attempting to execute the traces observed in the logs in relation to a business process. Different issues can be detected:

- deviations of log events (which traces cannot be replayed by the model?);
- deviations of the model (which activities are not conforming?);

Fitness is defined as a measure of correspondence between model and log.

Conformance checking can thus be used for two main objectives: i) verify whether actors within the organization behave as expected, and ii) evaluate the quality of process discovery by comparing different process models to find the one that best fits the event log.

Deviations are not always to be considered negative. They can be due to operators' reactions to some unexpected problems. In these cases, a deviation could prevent serious damage. Moreover, deviations can also indicate inadequacy of the process model. Users might find a better way of doing things than prescribed. It is therefore important both to detect deviations and to study and learn from them.

7.3.4 Process Improvement

Process mining can also be used to analyze process behavior in detail. From a process model and its event logs, it is, for example, possible to analyze how frequently the activities composing the process are executed, which are the most frequent

paths, how many execution variants exist, and what the average, maximum, and minimum execution times of different activities are. All this information can be very useful to find inefficiencies and possible improvement areas in business processes.

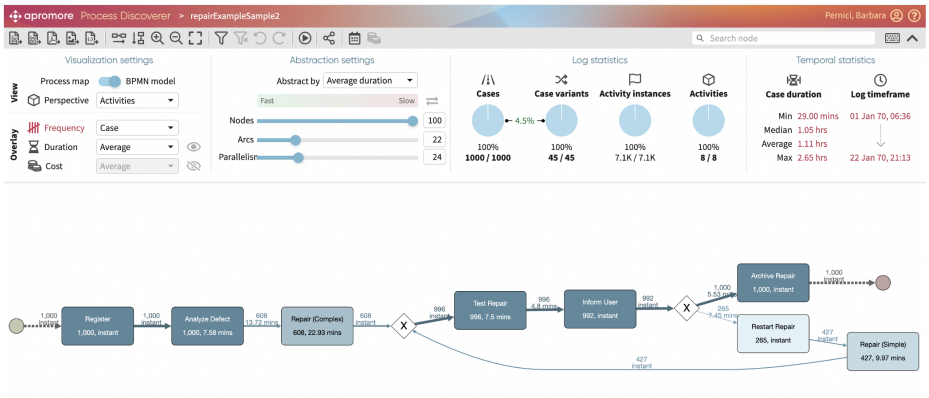


Figure 7.16: Process improvement: example of an analysis dashboard with Apromore

Figure 7.16 shows an example of a process analysis dashboard using the tool Apromore[1]. The process shown models the repair activities of machinery in a factory. The dashboard shows both the frequency and execution times of single activities as well as summary metrics related to the entire process. The most frequent paths are highlighted with thicker arrows between the activities composing the process.

7.4 Questions

- Illustrate the architecture and main properties of a Data Warehouse. Exemplify some problems related to data extraction and loading.
- Describe the components of a data warehousing system. Then briefly explain how companies can use the outputs of such an application.
- Illustrate classification algorithms and discuss their usefulness in analytical CRM.
- Illustrate the criteria for validating learning models.
- Describe the structure of logs in process mining and explain their usage.
- Illustrate the concept of conformance checking in process mining and discuss a simple example of a process model in BPMN with examples of logs with and without deviations.

[1] `https://apromore.com`

Chapter 8

Platform-Level Technologies

In an information system, platform-level technologies provide the infrastructure, tools, and services that enable the development, deployment, integration, and management of applications and data. These technologies form the backbone of an information system and are often classified into different types according to their role. A classification of the various existing platform technologies was discussed in Chapter 5 and is illustrated in Figure 5.1. This chapter focuses on the platform technologies that support the integration of components at the application level. Aspects related to physical architecture are discussed in Chapter 9. For security aspects of information systems, we refer the reader to Chapter 10.

8.1 Integration Architectures

As described in the previous chapters, the main elements of an information system are the data and the applications that manage them. The way in which applications handle data is defined by business rules, which can be represented as processes indicating the set of activities to be performed to achieve a specific goal. An information system can therefore be seen as a set of application modules that implement the functionalities the system must support. These application modules do not operate in isolation but are connected by information flows (data and process synchronization) that enable collaboration. The different modules must therefore operate in a coordinated way.

As discussed in Chapter 2, an organization's information systems are developed incrementally and are subject to continuous evolution. This has led many organizations to face the challenge of integrating modules within their own systems and with external systems they collaborate with. The adoption of information management systems typically follows an incremental logic driven by cost constraints or opportunities. Usually, the process begins with the development/acquisition of systems to support operational processes, and only later extends to managerial processes. A significant amount of time may pass between the two phases, leading to differences in technology, or the initial system's vendor may lack the expertise to

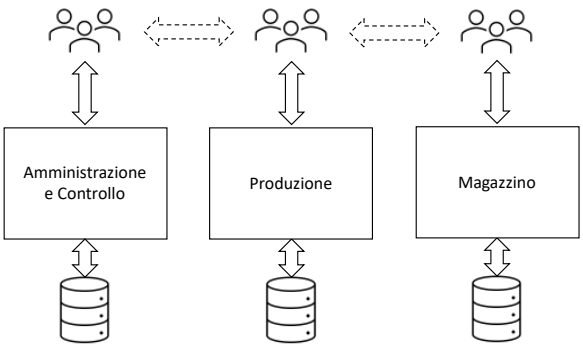


Figure 8.1: Information (or data) silos

provide the second system, forcing the organization to turn to other vendors who may not guarantee full compatibility. It is also possible, particularly in medium and large organizations, that IT systems were implemented in an uncoordinated and fragmented manner.

As can be seen, there are several reasons why one might end up with an information system composed of siloed applications, i.e., applications that do not communicate with each other, forcing operators involved in various processes to communicate manually in order to obtain information managed by other nodes. For example, as shown in Figure 8.1, since information related to production is also of interest to those managing inventory and orders, it is necessary for people in these roles to speak with production staff to obtain information about job status, processing times, and material delivery. As a result, the internal information flow — which is essential for the proper and efficient execution of business processes — is limited, if not hampered, by ineffective IT-level data management. The consequences of this situation are multiple:

- Business functions can only see data managed by applications within their own domain. Information exchange between different functions occurs indirectly through human operators, leading to limited performance and a high risk of error.

- Information relevant to multiple functions must be replicated across different silos. This creates data management challenges and potential inconsistencies in data representation. In fact, updating data in one database is not automatically propagated to others.

- Some activities may be performed by multiple functions; therefore, the supporting applications may be duplicated across different silos, resulting in wasted resources. Waste also occurs in system management, as different silos may rely on different technologies and policies. This requires specialized teams whose roles may overlap across silos.

- The siloed structure of the information system strongly influences the company's organizational structure, where the lack of application cooperation becomes an obstacle to cross-functional collaboration. While highly hierarchical organizations may not suffer significantly from this limitation, more agile organizations may find it to be a serious constraint.

Additionally, the organization must cooperate with the information systems of external entities (customers, suppliers, partners, etc.). Each of these entities has its own information system, designed according to its own rules and business needs. It is therefore likely that the information systems of two cooperating organizations are built using different technologies and are not natively capable of efficient and effective collaboration.

Integration architectures [32] are technical and organizational solutions aimed at ensuring the coordination of intra-organizational and inter-organizational activities. The objects of integration are: *data*, used and created by various modules of the application portfolio; *application components*, which must interact to ensure a correct information flow; and *processes* that involve multiple inter-organizational and intra-organizational activities.

8.2 Application levels and integration techniques

As explained in Chapter 6.1, applications are divided into several logical layers that communicate with each other to provide a result to users. Integration techniques can act on the different logical layers of an application. In this section, we will review the different approaches.

Data-level integration To solve issues related to the integration of different application components, it is essential to enable a proper flow of data between various business functions. This is only possible if data is accessible from a single shared source. This does not mean that there must be a single database, but rather that the complexity of data management—including the use of multiple databases—can be made transparent so that each application can access the necessary data directly, not through manual operations.

One option, certainly radical, effective, but at the same time not very feasible in most cases, is to completely rethink the information system and replace it with one that enables a single shared database. This data, which encompasses the entire informational assets of the company, can be read and written by the various applications across the different business functions. Every operation on the database (e.g., completion of a production batch) is immediately visible to other applications, allowing for the execution of subsequent activities (such as billing or storing in the warehouse awaiting shipment). This kind of radical approach is rarely feasible since organizations are usually reluctant to abandon applications that have proven reliable, and often lack the resources or desire to invest in costly and disruptive change programs. Nonetheless, this goal should be seen as the ideal target of any integration effort and can be pursued through different strategies.

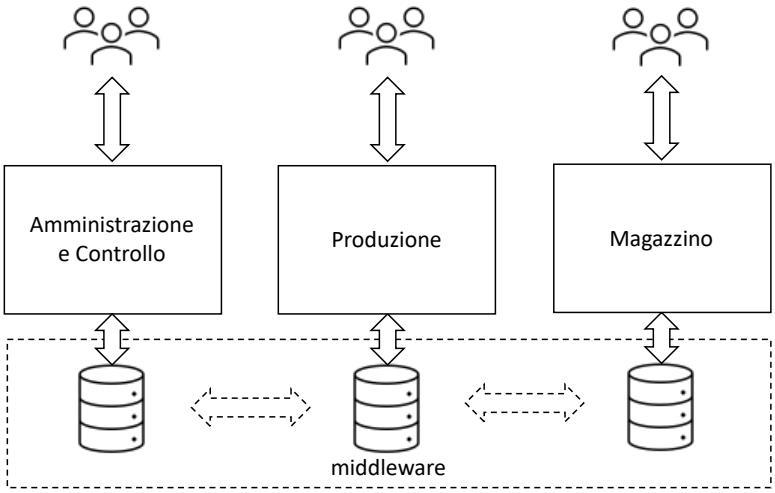


Figure 8.2: Database integration

An example is integration at the database level. In this case, as shown in Figure 8.2, a software layer called middleware ensures data consistency across different databases in case of duplications. For example, assuming an order is stored in all databases, when one application modifies the order, the change is propagated to the other databases. The middleware is appropriately configured to manage the propagation of changes by detecting modifications to data in the various databases.

The middleware also handles the normalization of the format and semantics of the data contained in the different databases. Similar to the ETL tools introduced in Chapter 7.1.3, the middleware makes data from different databases compatible in terms of both *format* and *semantics*. The *format* describes how data is stored and displayed. Integration issues related to format include:

- *Representation*: the same information may be represented differently by various modules. For example, an address may be represented as a record <`street, number, ZIP code, city`>, or as a single text string containing those elements. Address representations can vary by country and even within a single country (e.g., various terms for "street" in Italy such as campo, calle, etc.). Using a single string allows for a more general representation, but it then becomes necessary for applications to extract individual fields (e.g., ZIP code and city) from that string.

- *Structure*: the same data may be represented with the same fields but in different order. For instance, the "address" record may be ordered as <`ZIP code, city, street, number`>, or structured hierarchically as <<`ZIP code, city`>, <`street, number`>>.

- *Presentation*: the same data may be displayed to the user in different formats. For example, a "surname" field of 15 characters may be insufficient to represent

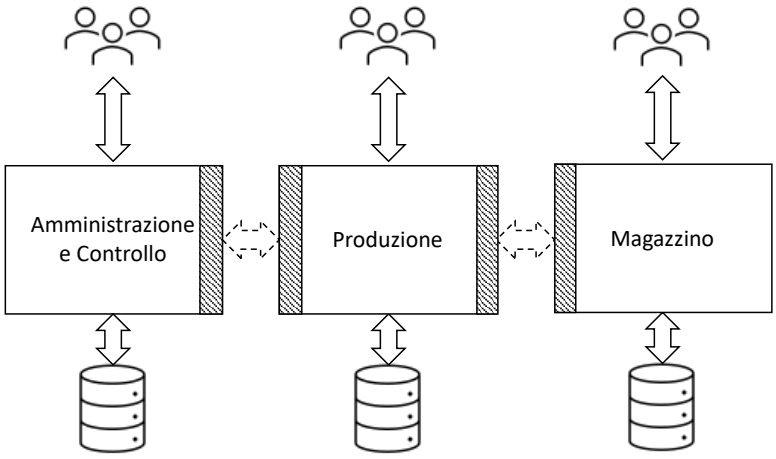


Figure 8.3: Application-level integration

some surnames. As a result, different application modules may use 15, 20, or 30 characters. The compatibility between systems is not guaranteed if the surname exceeds the minimum length.

Semantics refers to the meaning attributed to the data. For example, if we are managing orders and have a field "price," we can associate a numeric value. In an Italian system, this may be interpreted as a price in euros. However, this interpretation is not universal. If we need to connect the systems of an Italian and a British company, the currency must also be specified. Questions also arise such as: should conversion occur at the time of ordering or at payment? Who provides the exchange rate? Similar issues can occur with units of measurement: are we ordering 10 kilograms or 10 pounds? These problems are handled by the logic embedded in the middleware.

Application logic-level integration Another class of approaches involves integration at the application level—that is, allowing an application to communicate directly with other applications (Figure 8.3). This type of solution is not always possible. Its feasibility depends on the software architecture of the application. In the case of applications built using open-layered architectures, integration is easier because the software is designed for external communication. If, instead, the architecture is layered but closed, integration may require modifications that are not always possible to implement. The situation becomes extremely complex with monolithic software composed of a single application.

Presentation-level integration In applications that, from a modularity standpoint, at least separate the front-end and back-end, another possible integration path involves modifying only the client to call server-side functions exposed by

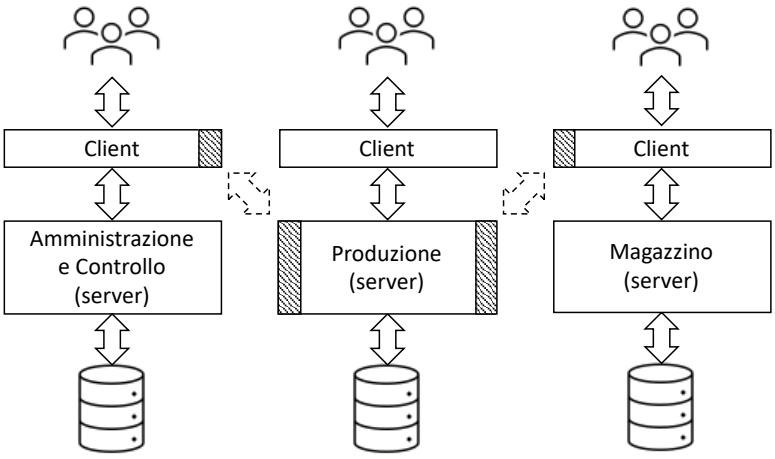


Figure 8.4: Client-level integration

other applications, as shown in Figure 8.4.

Without delving into implementation details for each of these solutions, it's important to remember that ex-post modification of existing applications is never trivial. In some cases, there are limits imposed by the technology used to build the applications. For instance, some programming languages may not support network communication adequately. In other cases, an application's design may not support simultaneous connections. So, while an operator is active on an application, another application—perceived by the first as an external agent—cannot access it. Additionally, the consistency of data managed by an application may be compromised if external agents are allowed to modify that data. For example, if a production management system is natively designed to notify users of a batch closure before updating the database, then allowing an external software to declare the closure must still guarantee that the closure and update operations occur in the expected sequence and manner.

8.3 Application Integration Models

After introducing the different categories of approaches, in this section we discuss some examples of models commonly adopted to achieve integration between multiple application components, either within the same organization or between applications belonging to different organizations.

8.3.1 Point-to-Point Integration Architecture

One possible solution to the problem of application integration is the implementation of a *point-to-point architecture*. In this model, each application is directly

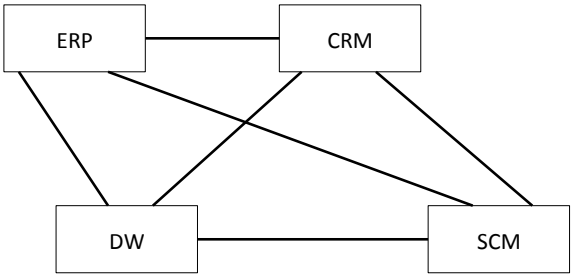


Figure 8.5: Point-to-point integration

connected to the others, typically via custom connectors or APIs. Each pair of systems communicates independently, without a centralized intermediary (Figure 8.5).

In both cases, the systems are connected through integration applications that become part of the information system and, if needed, also provide the transformation mechanisms necessary to make data compatible across multiple systems. This model primarily operates at the application level and requires software modification for each pair of applications to be integrated.

The drawback of such solutions is that the integration logic is hardcoded within each system, making it very difficult to replace a component—either for an upgrade or to switch to a different component. In this model, the number of interfaces to be implemented is on the order of N^2, where N is the number of applications involved. Therefore, this solution is not scalable, since modifying one application requires changes to $N - 1$ interfaces. Thus, it is suitable only for small-scale systems where the number of interacting applications is very limited.

8.3.2 Hub-and-Spoke Integration Architecture

The *hub-and-spoke* paradigm is based on a central hub, to which systems are connected via adapters (spokes). In this way, the connections are not direct as in the previous model, but the responsibility of integration is assigned to a dedicated component (Figure 8.6).

The application component that requires access to information from another component does not need to explicitly define which component the request will be sent to. Instead, the request is sent to the integration hub. The hub serves as a centralized controller to mediate communication between the various applications. This model focuses on coordinating the flow of business logic and application services between different systems. The hub handles request routing, data transformations, and processing. This too is an application-level approach.

It is clear that with this approach, application components can be more easily replaced or added. In this solution, the number of interfaces to be implemented is reduced to N, that is, one interface per application. This makes the solution more scalable, but the complexity of implementing the adapters and the hub remains.

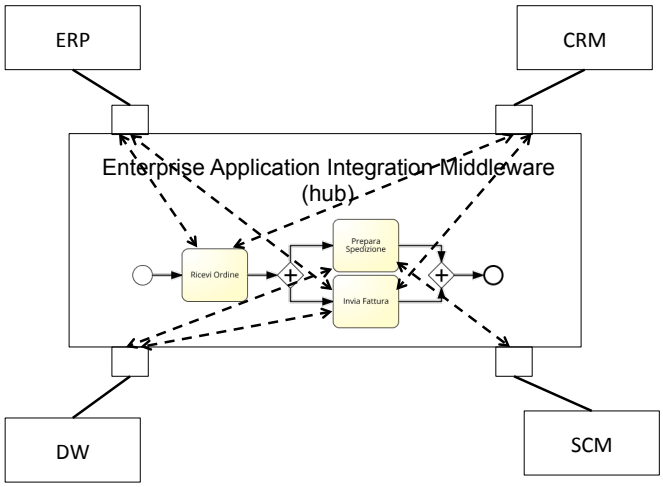


Figure 8.6: Integration using a hub-and-spoke architecture (adapted from [32])

Additionally, the application logic is hidden in the rules, and any modifications can be complex.

8.3.3 Integration with Business Process Management System

In the hub-and-spoke integration architecture, integration also includes the implicit definition of the order in which operations are executed through rules. When applications are numerous and it is necessary to coordinate the functionalities provided based on formalized processes that are subject to change over time, it is possible to use application platform-level components to support process automation. These are separated from the application logic and not embedded within the applications themselves.

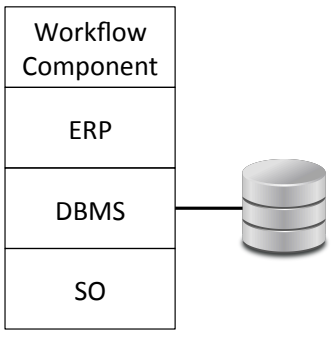


Figure 8.7: Integration using a Workflow Component

A Business Process Management System (BPMS), also called a Workflow Component, allows the management of process states and information flows between applications based on a formalized description of the processes. The Workflow Component helps simplify modifications to the application module resulting from changes or reorganization of business processes, since it separates applications from the logic that composes them into a business module.

It is possible to define ad hoc integration processes using a process automation component, which allows message exchange among various functionalities in an integrated way. For example, a BPMS (Workflow Component) may be part of an ERP suite (in Figure 8.7, the Workflow Component is highlighted within an ERP system), enabling coordination of functionalities provided by different functional modules.

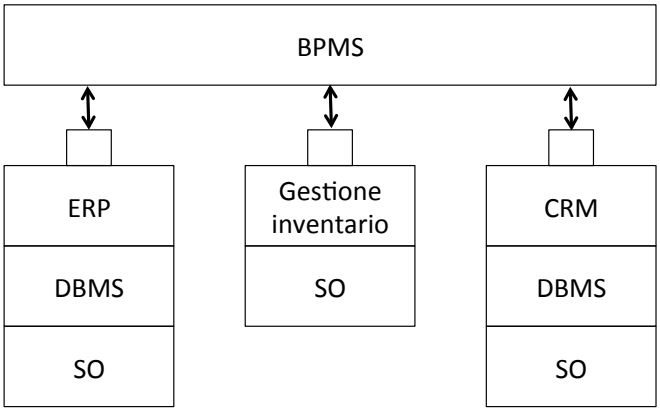


Figure 8.8: Integration through a BPMS

When integrating systems provided by different vendors, an external workflow management system can be used (Figure 8.8). In this case, the BPMS provides the integration infrastructure based on a coordination logic defined, for example, using BPMN.

The systems illustrated above enable integration but do not solve the problem of system interfacing. When integrating different systems, adapter components are always required to allow systems to communicate with one another.

8.3.4 Service-based Models

An alternative software design paradigm is that of Service-Oriented Architectures (SOA). In this paradigm, functionalities are modularly implemented through services that can be remotely queried via the HTTP Internet protocol.

In SOA, the concept of service is a key aspect of the organizational perspective. A service represents the implementation of a business functionality module that can be invoked by users (consumers) of the service, either internally or externally

to the organization. Thus, the service defines a business function with a well-defined interface for interaction with other components.

In [32], services are defined as computational tasks with a high degree of decoupling that communicate over a network, which plays a crucial role in business-to-business interactions. To support services, an SOA focuses on how services are described and organized to enable dynamic usage.

By its nature, a service must provide a self-contained functionality, independent from other related services[1]. Furthermore, for its consumers, a service is a black box, of which only the *interface* is known, used to invoke its functionalities. The interface defines the *operations* through which the service can be used by its users. The characteristics of a service are shown in Figure 8.9 [14].

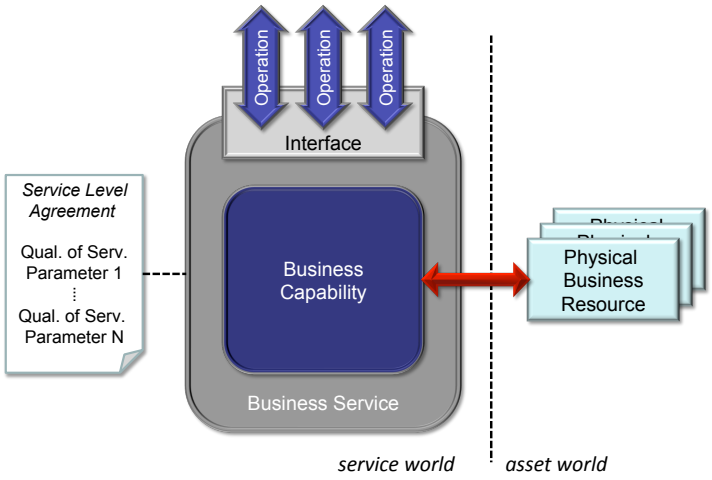


Figure 8.9: The service paradigm

Figure 8.9 also illustrates the physical resources available to the service (asset world). These resources are not visible to the service user, who only sees the interface and the agreed service levels, but not the underlying physical resources, which may also vary.

To ensure that a service is capable of providing a certain functionality correctly, a quality agreement is established with the consumers, called a *Service Level Agreement (SLA)*, which is fundamental in the service paradigm. The SLA defines the non-functional characteristics guaranteed by the service, in terms of *Quality of Service (QoS)* parameters. Each QoS parameter specifies one or more metrics that can be used to monitor service execution and verify valid responses. Examples of QoS parameters include response time (time taken by the service to provide a response), availability (fraction of time the service is available), and operational cost to invoke the service. An example of an SLA is shown in table format in Figure 8.10. The table lists three quality parameters, each of which may have one or more associated

[1] http://www.opengroup.org/standards/soa

SLA Business Service	Manage Shopping Cart	
Response Time	Maximum	0.25 sec
	Average	0.10 sec
Availability	Minimum	99.99 %
Cost	Average	€ 0.02

Figure 8.10: Service levels

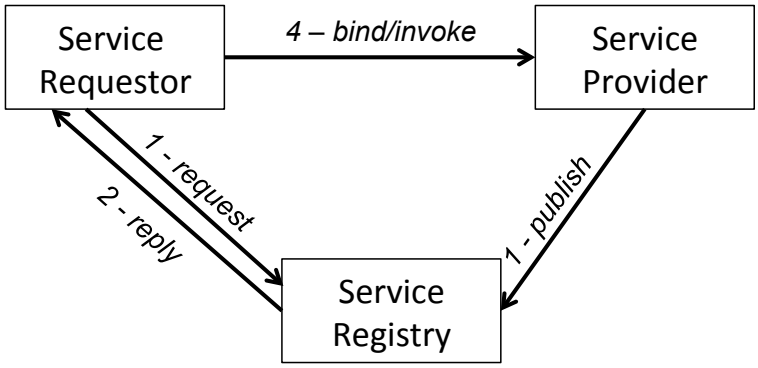


Figure 8.11: Service Oriented Architecture

metrics. For example, response time is measured both as maximum elapsed time and as average time.

The SOA (Service-Oriented Architecture) evolved also due to the possibility of exchanging messages on the Internet using standard protocols defined on top of HTTP and XML encoding (the *SOAP* protocol). In SOA (Figure 8.11), each service provider (*Service Provider*) publishes the information about the services offered in a *Service Registry*, describing the services in terms of operations and exchanged messages. This description is done using the standard language *WSDL (Web Service Description Language)*. Anyone who wants to interact with the service (*Service Requestor*) must query the Service Registry to discover available services that provide the desired functionalities, and from their description will learn which messages to exchange, which protocols to use, and how to access the service. The service is then used directly by the Service Requestor, who binds to the service and invokes the operations provided by the Service Provider (Figure 8.11). This enables dynamic interactions, where the participants can be selected at runtime.

In addition to using WSDL and SOAP to invoke services, there are currently lighter alternatives for building service-based systems, using direct invocation of operations via HTTP, for example.

In the architectural style called *REST - Representational State Transfer*, we refer to *RESTful services* when requests are made directly to a resource via its URI

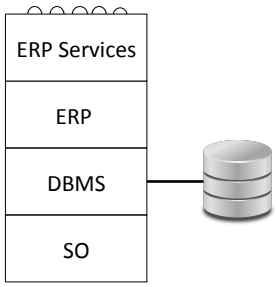


Figure 8.12: Functionalities as services in an ERP system

(Uniform Resource Identifier), and to *REST web-based* systems when the response is returned in a predefined format (XML, HTML, JSON, or others). When HTTP is used (which is most common), the available operations correspond to predefined HTTP methods (GET, POST, PUT, DELETE, etc.). This service invocation style offers benefits in terms of performance, modifiability, and simplicity. The previously mentioned requirements still apply: the formats and methods of information exchange must be defined to enable true integration.

8.3.5 Service Composition and Orchestration

The service paradigm facilitates integration by predefining the messages to be exchanged. Each component defines its own interface, specifying which services it offers and which input and output messages must be exchanged. Integration applications can therefore be built using this paradigm as a foundation.

Several scenarios can be outlined. First, existing systems may expose their functionalities as services, as shown in Figure 8.12—an approach that characterized the evolution of ERP systems in the 2000s.

Different services can be used together to provide the functionalities required by a single software application. For example, to implement an online shopping application, it is necessary to use a service to manage the shopping cart, one for catalog search, another for payment, and so on. To deliver the required functionality, all these services must be composed into a more complex business service. SOA architectures do not focus on software modularization but on integrating independent software components. This is enabled by technologies and standards that allow communication and cooperation between components over the Internet.

Composing services means treating them as elements in an orchestration. A *service orchestration* defines the sequence in which the individual services are invoked, using a representation similar to that used for processes (Figure 8.13).

In inter-organizational collaboration within a service-oriented context, flexibility is critical as it allows organizations to meet diverse customer requirements. Flexibility is also necessary to respond to market changes, which can be highly dynamic for services.

Flexibility can be achieved by dynamically composing services provided by var-

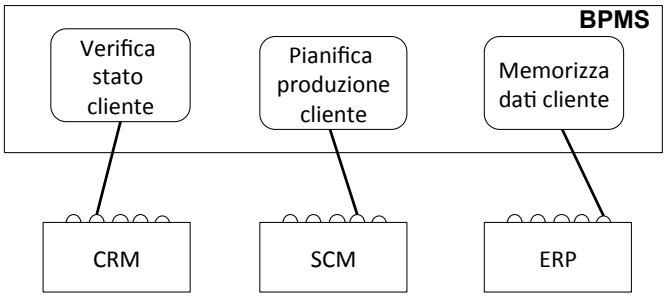


Figure 8.13: Service composition

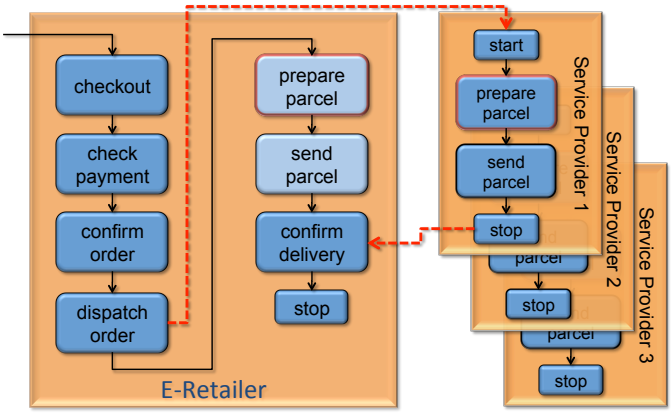


Figure 8.14: Dynamic service invocation

ious organizations in the network (Figure 8.14). The components of the complex service are selected at runtime, using the service registry, based on the offered functionalities and SLAs. To facilitate this operation, the service market includes Service Brokers—organizations that help align offered and requested services. For instance, as illustrated in the figure, a system can dynamically invoke services from different providers, choosing the provider to use each time, or replacing a provider when needed (e.g., if a response is not received within a specified time).

8.4 Questions

- Describe the integration architectures of information systems, discussing the evolution from point-to-point integration and highlighting its advantages and disadvantages.
- Integration architectures of software components: differences between adapter-based approaches and service-based systems.

- Illustrate the common characteristics and differences between point-to-point application integration and other approaches.
- Explain the motivations for integrating the applications of an information system. Describe how applications can be integrated within a service-oriented architecture.

Chapter 9

Technologies at the Physical Architecture Level

9.1 Description

In this chapter, we focus on the organization of hardware and software for managing information resources. As we will see, depending on the information needs of the organization under consideration, the architecture of the information system can vary significantly. In particular, we will describe the possible relationships between hardware and software. This chapter presents some aspects related to the various alternatives for implementing physical architectures on which to deploy the software components previously discussed.

9.1.1 Evolution of Physical Architectures

In the context of information systems, the term architecture refers to the set of technical and organizational choices that influence the development and use of IT-based technological resources.

To begin with, it is useful to distinguish two macro-categories of architectures at the physical level: *centralized* and *distributed*. Centralized information systems, that is, systems based on a centralized architecture, are characterized by the presence of a single processing node where all the software required for information processing is installed. This architecture represents the typical solution of early information systems, when networks had limited diffusion and the cost of purchasing and maintaining a single node was rather high. Connected to the central node were one or more terminals: processing nodes with very limited capabilities, often restricted to the presence of one input device (e.g., keyboard) and one output device (e.g., screen or printer). Through direct network connections to the central node, terminals could interact with the installed software but were not capable of running any application locally.

More specifically, we refer to a *centralized information system* (or simply *cen-*

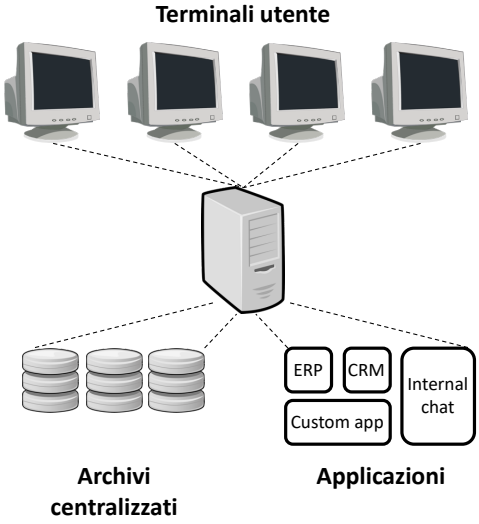


Figure 9.1: Centralized information system

tralized system) when both data and applications reside on a single processing node (Figure 9.1).

Centralized systems emerged with modern computing in the 1950s and developed throughout the 1960s and 1970s thanks to the evolution of mainframes, the introduction of time-sharing operating systems, and the development of centralized database management systems. The emergence and development in the 1970s and 1980s of new, more affordable technologies—both in terms of hardware and versatile, user-friendly data management structures (such as mini and microcomputers, personal computers, local communication networks, client/server architectures, graphical user interfaces, and advanced data modeling techniques)—led to the decline of the centralized model and the rise of systems based on distributed architectures (Figure 9.2). A system is defined as distributed if at least one of the following two conditions is met:

- The cooperating applications reside on multiple processing nodes (distributed processing).
- The unified information base is hosted across multiple processing nodes (distributed database).

Nevertheless, in the early 1990s, the distributed model faced strong criticism due to its design and management complexity. This issue became even more pronounced when the Internet began to be widely used to enable communication between various nodes. In order to address these limitations, more complex architectures were developed based on the distribution of logical levels—linked to application structure—across physical levels, related to the physical structure of the processing systems. As illustrated in the following sections, such architectures took into account

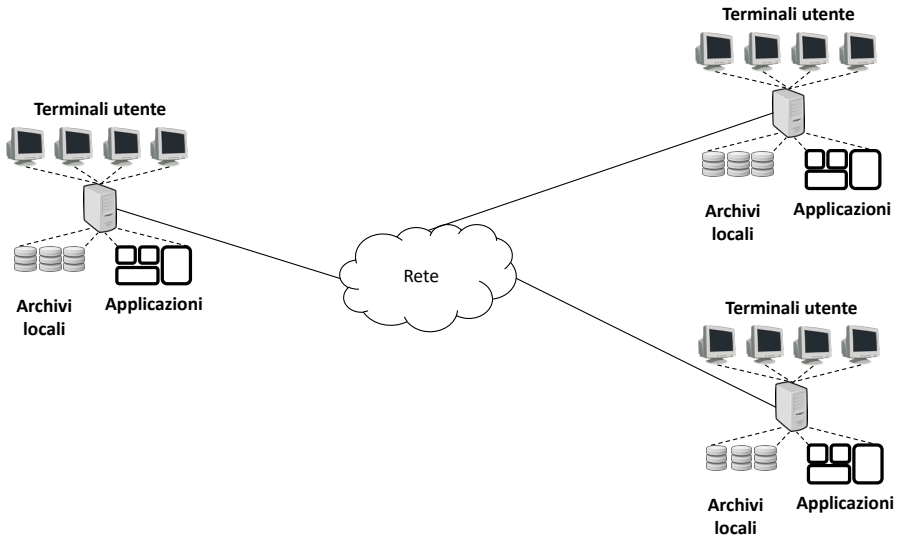


Figure 9.2: Distributed information system

the problems associated with the fact that an application—when available on the Internet—might need to serve a large number of users.

9.2 Distributed Applications

In distributed systems, the software logical layers, called layers (Chapter 6.1), are installed on hardware levels (called *tiers*), where each level represents a machine (or a set of machines), each with its own computing capacity. Depending on how the layers are distributed across the tiers, an application can be configured in various ways.

Single-Tiered Architecture. The first architecture involves a single level of distribution and is called *Single-Tiered*. In this case, the three software layers are assigned to a single machine. This is the classic terminal-host configuration that characterized mainframe-based systems. Although representing a centralized architecture tied to an outdated approach, it still holds some value. In fact, mainframe systems, the quintessential Single-Tiered architecture, offer high levels of performance, reliability, and security. Additionally, maintenance is simplified since it only involves a single node. On the downside, this architecture lacks flexibility. The installed application is monolithic, and an upgrade to one layer directly impacts the others. Furthermore, the need to migrate to graphical user interfaces has highlighted the limitations of this setup, as it would require the graphical processing — a relatively demanding task — to occur for each terminal directly on the mainframe.

Two-Tiered Architectures. An evolution of the single-tiered architecture involves two distribution levels and is known as *Two-Tiered*. Here, the logical layers are split between two computing nodes. This configuration was implemented in the first generation of client/server systems in the 1980s. Six different configurations (Figure 9.3) can be identified, based on how the three logical layers — Presentation (P), Application Logic (A), and Data Management (D) — are allocated. These configurations differ in terms of network traffic, performance, and workload on the workstation, resulting in varying levels of flexibility and ease of modification.

Depending on the functions assigned to the client node, we refer to *thin clients* when the client only handles the presentation logic. An example is a client with a Web Browser (Web Client), which does not require significant computing power, as it merely interprets and displays HTML pages. This solution, which works even with limited resources, requires a stable connection between the client and the server to respond to all user requests.

On the other hand, we speak of *thick clients* (also known as *fat clients*) when, in addition to the presentation logic, part or all of the application logic is present. This requires that the node be powerful enough to perform both display and computation tasks. For example, consider smartphone apps. In this case, the app qualifies as a thick client because it has all the necessary tools locally to handle user requests and interacts with the server only for specific functions.

Although thick clients do not require a constant connection between the client and the server — only during functionality or data retrieval not present on the client node — they introduce some disadvantages. These stem from the need to manage multiple client machines rather than just one server, as was the case in centralized systems. If an application is frequently updated, in thick configurations the updates must be applied to all client machines, which can lead to high management costs.

Nowadays, systems supporting automatic updates of client-side applications are widely adopted, making the distribution and update of software more manageable and facilitating the deployment of application modules to clients such as apps.

Referring to the various Two-Tiered architecture configurations shown in Figure 9.3, the "Distributed Presentation" and "Remote Presentation" configurations correspond to thin clients, while the others refer to thick clients.

Three-Tiered Architectures. A third architecture involves three levels of distribution (*Three-Tiered*): the three application layers are divided, in principle, among three dedicated machines, namely: a user workstation, an intermediate server (also called middle tier), and a data management server. Figure 9.4 shows the possible distribution of the three architectural layers across three tiers:

- Configurations 1 to 5 refer to *thick client* scenarios, more suitable for internal organizational contexts (Intranet systems).
- Configurations 6 to 13 refer to *thin client* scenarios, suitable for service distribution to an open set of users (Internet systems);
- Configurations 4, 5, 7, 8, and 10 to 13 implement physical separation between users (*tier* 1) and data (hosted solely in *tier* 3), via the introduction of an

intermediate machine (*tier* 2). These configurations thus offer improved data access control and security.

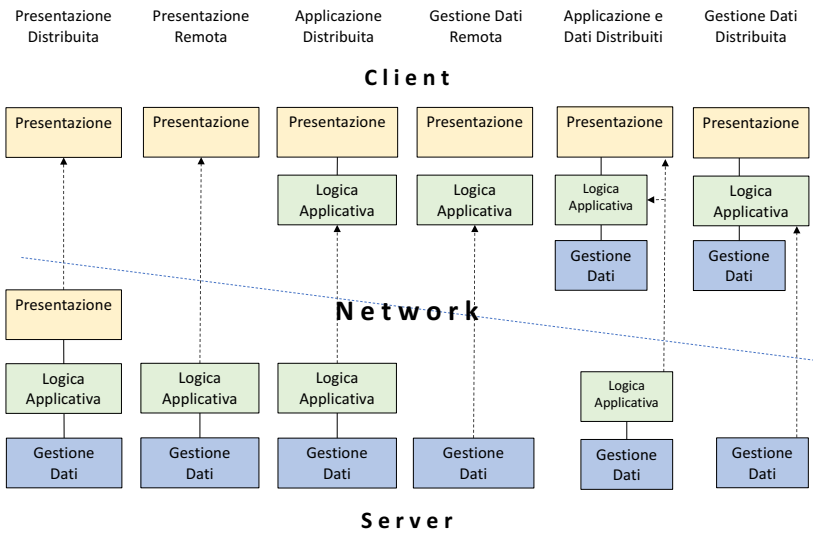


Figure 9.3: Classification of 2-tier architectures

Config. /Tier	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
Tier 3	D	D	D	D	D,A	D	D	D,A	D	D	D,A	D,A	D,A, P
Tier 2	D	D	D,A	A	A	D,A	A	A	D,A, P	A,P	A,P	P	P
Tier 1	D,A, P	A,P	A,P	A,P	A,P	P	P	P	P	P	P	P	P

Figure 9.4: Classification of 3-tier architectures

A three-tier system provides IT infrastructures with increased scalability and flexibility. Thick configurations with three tiers are not ideal when the distributed application logic is particularly resource-intensive in terms of CPU or RAM. In such cases, it is more efficient to centralize the application logic on an intermediate server. This way, ensuring the robustness and performance of the intermediate server — which is shared among various clients — is sufficient. The server is dimensioned to ensure a predetermined performance level for a given number of concurrently active users, which is generally lower than the total number of users accessing the system.

The intermediate server also reduces the load on the DBMS server. The middle tier maintains persistent connections with the DBMS. Furthermore, the number of sessions between the middle tier and the DBMS is generally lower than the number of client-to-middle tier connections.

An example of a 3-tier system (configuration 7) is shown in Figure 9.5, where each layer is allocated to a separate tier.

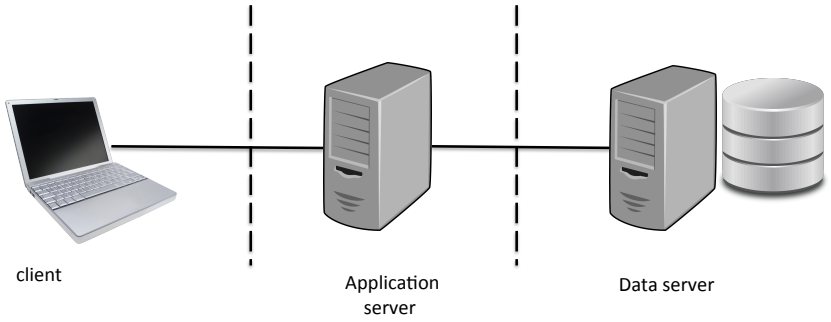


Figure 9.5: Example of a Three-Tiered architecture (configuration 7)

N-Tiered Architectures. While the transition from Single-Tiered to Two-Tiered architectures reduced mainframe load by shifting some computing power to client nodes (which became more common as costs dropped), Three-Tiered architectures aim to optimize server load by distributing it across two nodes. In recent years, this trend has evolved into architectures where application layers can be distributed across a generic number n of tiers (*n-tiered architecture*), with different functionalities handled by dedicated machines: for example, gateway servers, authentication servers, transaction servers, database servers, and so on.

Web-based information systems often rely on 5 tiers. In particular, the presentation layer spans 3 tiers: the user's browser, the web server (which ensures access to static pages and objects), and the script engine (which generates dynamic pages). Tiers 4 and 5 correspond to the application server and the DBMS, respectively.

An example of a 5-tier architecture for a Web-based information system is shown in Figure 9.6.

The number of tiers can further increase if servers are introduced to support the communication layer — i.e., the middleware. For example, when multiple information sources are integrated and queried in real-time, a database gateway can be placed between the data sources and the application server (see Figure 9.7).

The application server performs application functions based on a unified view of the data. The database gateway provides transparent access to the network's DBMSs and translates the application server's API calls into the syntax of the remote data source. This spares developers from having to know the details of the remote databases and their access methods and languages. This location transparency also leads to transparency with respect to *migration* issues: if a data source

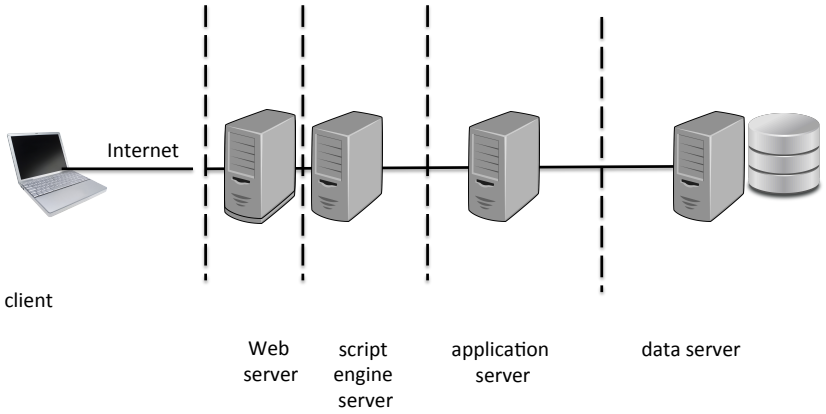


Figure 9.6: Example of a 5-tier system for Web-based information systems

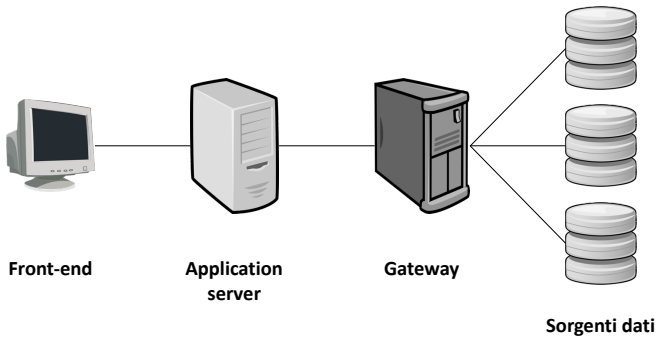


Figure 9.7: Example of a 4-tier system

is moved or converted to a new structure, only the database gateway needs to be updated — not the application server or the system's front-end logic.

It should be noted that even when more physical tiers are added, the number of software logical layers (*layers*) remains 3, since database gateways are typically considered part of the application logic layer.

9.3 Scalability

Scalability of an IT architecture refers to the ability of the infrastructure to meet increasing user demands (or a larger number of users) through appropriate upgrades.

In general, we can identify two types of *scalability*:

- *Vertical scalability* (scale-up): performance improvements are achieved by maintaining the same number of processing nodes while increasing their processing capacity. For example, a server is made more performant by adding CPUs, RAM, or mass storage capacity. In this case, the scaling operation is fairly easy to implement because it does not alter the distribution of applications. On the other hand, increasing computing power does not always result in a proportional performance improvement (e.g., doubling the CPU does not automatically double the number of executable processes), and physical limits exist beyond which it is not possible to go (e.g., limited number of available slots for CPUs and/or memory). This results in architectures that are not scalable beyond a certain level.

- *Horizontal scalability* (scale-out): capacity is increased by replicating the nodes that represent potential bottlenecks. Thus, processes that a single node was initially unable to handle are distributed among the replica nodes. In this case, upgrading the system is more complex, as it impacts the structure of the tiers. In fact, adding replica nodes requires the introduction of a *load balancing* system to distribute the processing load among the nodes. The advantage lies in the possibility of applying the downsizing principle, which allows scalability at low cost. Proposed in the early 1980s and still valid today, the downsizing principle asserts that low-end servers are cheaper than high-end servers or mainframes, given the same total installed computing power.

Regardless of the approach taken to make a system scalable, it is always difficult to properly size the system. In the case of *vertical scalability*, for example, sizing means determining how much additional memory to install, whereas in the case of *horizontal scalability*, it means determining how many replica nodes to introduce.

The problem arises in both over-dimensioning and under-dimensioning. In the case of over-dimensioning, upgrading the architecture might lead to unnecessary costs, since the added resources may not be used if user growth doesn't match. Conversely, under-dimensioning could make the upgrade insufficient after a short time.

Especially in the case of Web applications, the sizing problem is hard to solve because the number of users is a time-dependent variable and often hard to predict.

Consider, for instance, an e-commerce website for the Italian market. Assuming that during the day the number of users remains constant (with some peaks during certain hours), the system will be sized based on this number to estimate resource needs. However, this not only leads to resource waste at night (when few users access the system) but also to under-dimensioning during specific periods like the holiday season, when the number of users could increase significantly. For this reason, especially in the Web context, we no longer talk about scalability but rather about *elasticity*: the property of a system not only to increase resources in response to spikes in demand but also to reduce them when demand falls. Elasticity, which we will see is a key principle of Cloud Computing solutions, can be viewed as a reversible form of *horizontal scalability*, where nodes can be both added and removed.

9.3.1 Server Farm

Following the scale-out approach, physical tiers can also be implemented as a *server farm*, that is, a set of computing units sharing the processing load, applications, and depending on the configuration, data. A server farm is seen by other levels as a single resource.

Systems based on server farms are highly *scalable* because it is possible to meet increasing requirements by adding more machines, without being subject to the upgrade limits of a single machine, and thanks to the downsizing principle, this can be achieved at a lower cost considering the number of users served.

Server farms can be implemented following two design principles: *Cloning* and *Partitioning*.

Cloning means installing the same software applications and data on each server. Requests are then routed to different clones via a load-balancing system. A set of clones dedicated to providing a particular service is called a *RACS (Reliable Array of Cloned Services)*, and each clone implements the same set of functionalities. Cloning services offers both scalability and fault-tolerance advantages in terms of service availability. If one clone fails, another can continue providing the service.

RACS can be implemented in two configurations: shared-nothing and shared-disk, depending on whether storage is shared.

In the first configuration (*shared-nothing*, see Figure 9.8): the stored data is replicated across all clones and resides on a local hard drive at each clone. Thus, any data update must be applied to each clone. Consequently, this configuration is problematic for "write-intensive" services, while offering better performance for "read-only" applications, such as access to static pages or downloading files/images from Web servers.

In the *shared-disk* configuration, often called a *cluster*, clones share a storage server that manages hard drives (Figure 9.9).

The *partitioning* technique duplicates hardware to distribute application execution across nodes (Figure 9.10), also partitioning the data. Each node performs a specialized function; for example, a company's web sales system can be split by customer types or product lines, each managed by a different node.

Partitioning is completely transparent to applications, and requests are sent to the partition that holds the relevant data. For instance, if partitioning is done by

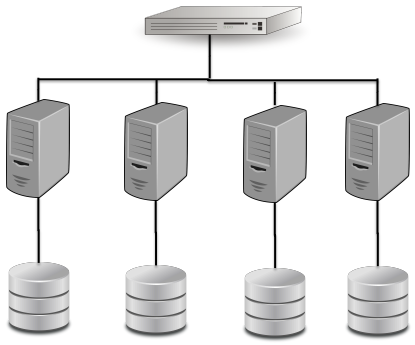


Figure 9.8: RACS: Shared-nothing configuration

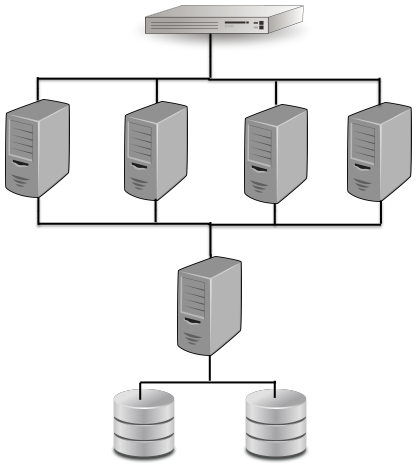


Figure 9.9: RACS: Shared-disk or cluster configuration

product lines (A, B, ...), requests for product line A are directed to the server with access to data for product line A. However, this technique does not improve availability, as the data resides on a single server and a failure makes the corresponding service portion inaccessible. This characteristic is known as *graceful degradation*: unlike centralized systems, in case of failure, not the entire system becomes unavailable, but only specific functionalities.

To address the unavailability issue of some services in case of failure, the individual partition servers are often cloned, creating *packs*. This is referred to as *RAPS (Reliable Array of Partitioned Services)*, a solution ensuring both scalability and service availability (Figure 9.11).

An example of a 5-tier architecture in a server farm for a web-based information system is shown in Figure 9.12. Web servers and script engines are usually cloned, while the application server is partitioned or both partitioned and cloned, depending

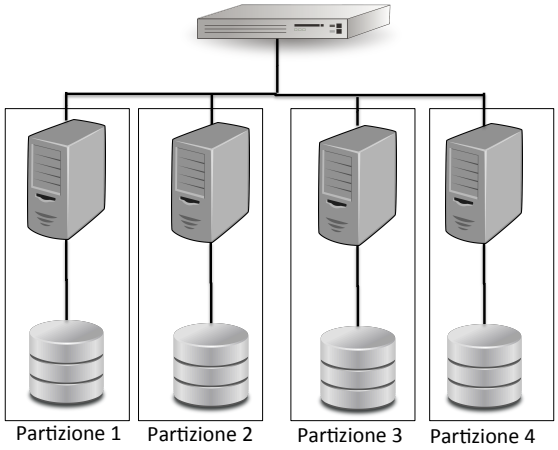


Figure 9.10: Partitioning

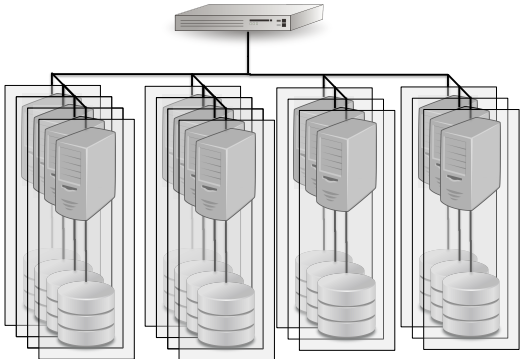


Figure 9.11: Redundant and partitioned architecture (RAPS)

on the applications.

9.4 Virtualization

The creation of server farms has made it possible, as described, to build infrastructures capable of supporting the computational load required when the number of users increases—often due to applications being designed to be accessed via the Web—to such an extent that it compromises the application's performance. However, it should be emphasized that building server farms is often economically expensive: both in terms of the investments required to purchase and install new resources, and in terms of operation, since maintenance and management of such resources is proportional to the number of machines to be handled. This has therefore limited the ability to offer Web applications to only those organizations capable

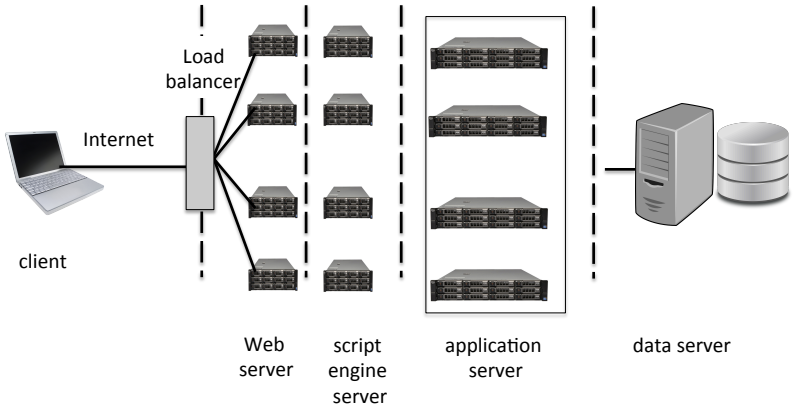


Figure 9.12: Example of a 5-tier system for web-based information systems

of bearing such costs, leading to the creation of large data centers and effectively excluding all small and medium-sized organizations.

At the same time, a software technology known as *virtualization*, developed since the 1960s, has gained new momentum. It allows physical components of a computer (e.g., CPU, memory, disk) to be abstracted and made available as software resources. A virtual CPU is a software component that, from the user's perspective, provides the computational capabilities of a physical CPU. The difference is that requesting a computation from a virtual CPU means calling an application (the virtual CPU), which in turn translates that call to the associated physical CPU. While this indirection causes a performance degradation, it has several advantages:

- Physical resources can be decoupled from the application that uses them. Therefore, by keeping the virtual CPU unchanged, a change in the physical CPU is completely transparent to the application.
- A single physical resource can be split into multiple virtual resources. For example, the same physical memory can be divided into n virtual memories, each assigned to a single application.
- The same physical resource can be shared among several virtual resources, while still giving the illusion to the application of exclusive access. For instance, the same CPU can be accessed concurrently by different virtual CPUs. From the application's perspective, it still appears to have exclusive access to the CPU. In reality, other virtual CPUs—and thus other applications—access it concurrently. This enables optimal use of the resources.

The set of virtual resources required for an application's operation is called a *virtual machine (VM)*, which, to be executed, needs a physical machine capable of providing the real resources. The mapping between real and virtual resources is managed by a software layer called the *hypervisor* or *virtual machine monitor* (see Figure 9.13), which can be installed directly on the physical machine (in this

case, it's a *bare-metal* installation) or on top of the physical machine's operating system (in which case, the OS is called the *host* operating system). Among the most commonly used hypervisors are: XEN, VMWare, and VirtualBox. The hypervisor's goal, regardless of its configuration, is to enable the execution of one or more virtual machines by providing the assigned virtual resources to the applications running on them.

Since the virtual machine, from an application's perspective, is a full-fledged machine, an operating system (called the *guest* operating system) can be installed on it, which may differ from the *host* OS. In this way, using a *hosted* hypervisor, a machine running, for example, Mac OS X as its operating system can execute a virtual machine running Linux or Windows.

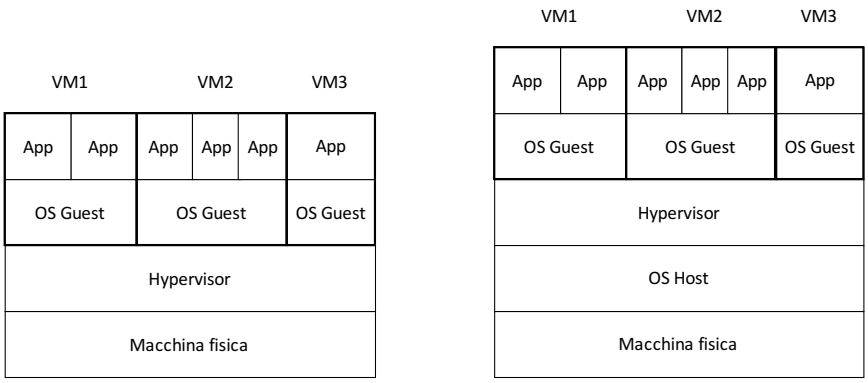


Figure 9.13: Bare-metal (left) and hosted (right) hypervisor

Virtualization has, in recent years, provided the means to improve and optimize resource management, especially in server farms. In fact, one of the problems with server farms—and data centers in general—is resource waste when demand for a given application drops—sometimes only temporarily—below the system's capacity. This type of waste becomes apparent when each application is assigned a specific physical machine. Statistically, the average utilization of a server is about 5–10% of its potential. To address this, virtualization has enabled *server consolidation* strategies. This means that a physical machine, through an installed hypervisor, can host multiple virtual machines that correspond to what were previously separate physical machines, allowing multiple applications to run and better utilize the available physical resources.

In this context, virtualization provides an additional benefit: since a virtual machine is not tied to a specific physical machine, it can migrate from one physical machine to another (*VM migration*). Currently, the most common migration techniques are *cold*, requiring the VM to be powered down, moved, and then powered back on. However, *live* or *hot* migration solutions are under development, where the virtual machine can be moved even while the application is running.

9.5 Cloud Computing

The paradigm shift toward service-oriented architectures (SOA), along with the flexibility provided by virtualization, has created fertile ground for Cloud Computing. It is defined by NIST (the U.S. National Institute of Standards and Technology) as: "a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction."

Cloud Computing is therefore a paradigm for building application architectures that, relying on virtualized systems, allow access to these resources from any network-connected device, from any location. The defining feature of these resources is that they can be requested *on demand*, only when needed. The Cloud platform will provide services to make them available as needed and release them when no longer required. Thanks to its characteristics, Cloud Computing allows cost-effective management of applications with high and variable request loads over time, and which handle significant amounts of data.

Cloud Computing is based on the concept of *utility computing*, where computing resources are perceived by users like a utility service, such as electricity. Users pay based on resource usage and (as with electricity) have the illusion of infinite resources. This illusion is possible because Cloud providers optimize the use of available resources by combining applications with very different computational load profiles. It is highly unlikely that, in a platform hosting many heterogeneous applications, all applications will experience peak load simultaneously.

In detail, the Cloud Computing paradigm includes:

- Five essential characteristics.
- Three service delivery models.
- Four deployment models.

Each of these elements is described in more detail in the following three paragraphs.

9.5.1 Characteristics of Cloud Computing

NIST defines Cloud Computing as a system characterized by the following:

- *On-demand self-service*: users can autonomously obtain resources such as processing time or storage.
- *Ubiquitous network access*: users can use standard networks and internet-connected devices to access Cloud resources.
- *Resource pooling*: resources are distributed across multiple servers and users typically don't know their exact physical location.

- *Elasticity*: resources can be quickly allocated, increased, or reduced to meet demand.
- *Measured service*: Cloud resource usage is monitored and billed based on actual consumption.

9.5.2 Service Models

The three basic service models (delivery models) of a Cloud-based system are:

Infrastructure as a Service (IaaS) The Cloud platform provider offers users physical resources (servers, storage, networking devices). These are perceived by users as virtual elements that can be used to host their applications within virtual machines (VMs), which define the required amount of physical resources. The resources are virtual in the sense that the user has no awareness of the physical machine hosting the VMs. Users can install their own platform software and applications on the VMs. Since the VMs are controlled by the users, they can be flexibly configured and managed, with computational resources dynamically acquired or released. Examples of IaaS providers include Amazon AWS EC2 (computing resources), S3 (storage), and Microsoft Azure Virtual Machines.

Platform as a Service (PaaS) In this model, the Cloud provider offers not only physical resources but also platform-level technologies. For example, a provider may offer Cloud-based DBMS services. APIs are provided to access these services, and users are responsible for implementing their applications. Users control only the application, but can configure certain platform technology parameters. Examples include IBM Bluemix (now IBM Cloud), Amazon AWS Simple DB, and Microsoft Azure Cloud Services.

Software as a Service (SaaS) In the SaaS model, the Cloud provider offers full applications to users, in addition to infrastructure and platform technologies. The customer does not control the service but may configure some application-level parameters. These systems allow access to the same application from multiple devices and support data sharing among users. Common examples include Gmail, Google Drive, and Dropbox.

Everything as a Service (*aaS) Extending the virtualization paradigm, many other components at the application, business, and physical levels are offered as services. Examples include DaaS (Data as a Service), BPaaS (Business Process as a Service), and others.

9.5.3 Deployment Models

Four Cloud Computing deployment models are defined:

- *Private Cloud*: The infrastructure is provided for the exclusive use of a single organization and may be managed by the organization itself or by third parties. This model offers high security and customization, but involves higher costs and limited scalability.
- *Community Cloud*: The infrastructure is shared by a community of organizations with common goals or needs. It may be managed internally or externally. Security and customization are still high, and costs and scalability issues are somewhat mitigated.
- *Public Cloud*: The infrastructure is made available to the public by a private or public provider. Services can be free or paid. This model is low-cost and highly scalable, but offers less security and customization.
- *Hybrid Cloud*: The infrastructure combines two or more previous Cloud deployment models (public, private, community) that cooperate while remaining distinct entities. This model benefits from the strengths of each deployment model. To collaborate effectively, the involved infrastructures must use shared, standard technologies.

Like all enterprise applications, the ERP can be managed internally (on-premise) or externally (outsourced) (this topic is further explored in Section 11.2.2). Cloud also enables ERP delivery as SaaS, where the ERP provider offers a SaaS version of its software, as opposed to on-premise deployment.

9.6 Questions

- Explain cloning and partitioning techniques.
- Describe the characteristics of Cloud computing.
- SaaS opportunities for businesses: advantages and disadvantages.

Chapter 10

Information Systems Security

Security in information systems is the set of organizational and technological measures aimed at ensuring that an *authorized user* has access to all and only the services and resources intended for that specific user, at predetermined times and in predetermined ways (for example, read-only), and that the system's data and applications are protected from unauthorized use. From a security perspective, an information system is seen as a set of entities (organizational areas, groups of users or resources, applications, data) that collaborate to pursue internal objectives (for example, administrative) and external ones (for example, providing services to customers and suppliers). From this perspective, security is the set of policies and mechanisms that ensure that cooperation among the component entities occurs according to predetermined access rules, guaranteeing mutual trust among the cooperating entities in the use of resources and in information exchange.

In this scenario, this chapter illustrates the categories of *attacks* on the system and the techniques of *protection*. In particular, the chapter describes the techniques underlying security, mainly *cryptographic techniques*, the user management techniques comprising *authentication* and *access control* to databases, and the *architectural components* capable of preventing (when possible), detecting and handling attacks with appropriate countermeasures.

Cybersecurity, namely security for digital systems with physical components (for example, in information systems that include IoT systems) is then presented. Finally, the evolution of attacks and techniques is overviewed.

10.1 Security Properties

A formal definition of information systems security is provided by ISO [10], which considers it as the set of measures aimed at protecting the requirements the system is expected to fulfil. Such requirements are defined according to the *security properties* described below:

- *Integrity*: the system must prevent the direct or indirect alteration of information, either by unauthorized users and processes or as a result of accidental

events.

- *Authenticity*: the recipient of the information must be able to verify the sender's identity, and information must remain intact. The author of the information item will not be able to deny its origin (non-repudiation).

  Authenticity is based on *Authentication*: every *agent* (an entity capable of undertaking actions in the system, such as users, programs, machines, and so on) must be identified before interacting with the system. Authentication may also be mutual (for example, the server authenticates the browsers and vice versa the browser must be able to authenticate the server it interacts with before starting a session of information exchange). The simplest form of authentication is the requirement to enter a password.

- *Confidentiality*: no users should be able to obtain or deduce from the system information they are not authorized to know. We refer to *privacy* when confidentiality concerns data about individuals.

- *Availability*: the system must make information and resources available to each authorized user, within the expected times and modalities.

Security must be considered a non-functional requirement of the information system. To treat it as such, it is necessary to integrate its aspects into all stages of the design and implementation of an information system, from planning to coding the software solution and deployment, in order to assess the actual protection implemented by the system. In this way, the system will not only be able to identify and/or prevent attempts of external intrusions or internal abuses (*active protection*), but also to respond effectively to an attack by limiting its damages (*reactive protection*).

10.2 Threats, Breaches, Vulnerabilities, Attacks

The *threats* to an information system can be distinguished as:

- *Physical*: damaging facilities and infrastructures. Examples include thefts, intentional damages, accidental events (air conditioning system failure), or natural disasters (floods, fires, earthquakes).

- *Logical*: stealing or altering information and resources. Examples include data theft (e.g., stored credit card numbers), creating hidden access points (backdoors) to the system, and, in general, the illegal use of computing and communication resources.

- *Accidental*: referring to software and network configuration errors, program malfunctions, data entry mistakes, and so on.

Considering that this text mainly focuses on logical threats, it should be emphasized that the *breaches*, i.e., the consequences of threats carried out against

information systems, exploit vulnerabilities and may lead to various undesired or unlawful situations. Among these, beyond unauthorized access to system resources, is also the inability to use resources one is entitled to access (denial of service, called *Denial of Service - DoS).*

A vulnerability presupposes the existence of one or more weaknesses (*exploits*) that represent a potential risk factor because they allow anyone to infiltrate and damage the system. An exploit is a program, or more specifically a technique, that leverages the vulnerabilities of a system, a program, or a protocol to cause abnormal and unexpected behavior. In general, exploits have a limited scope because they are strictly tied to the presence of a specific operating system and/or network service or particular configurations. Most *vulnerabilities* in a system derive from poor design quality, programming or testing bugs, or inadequate physical and logical access controls (for example, through wrongly-managed passwords) to sites and/or machines and networks.

In the context of information systems, *security attacks* are any action, carried out by individuals or organizations, that targets databases, infrastructures, communication networks, and/or personal electronic devices through malicious acts aimed at theft, alteration, or destruction of specific elements (defined as attack targets), violating system access rules. This is particularly relevant in so-called critical systems (for example, government, healthcare, or military systems).

Among the most frequent *network-level attacks*, we mention:

- *Sniffing*: interception of messages exchanged among agents, leading to unauthorized access to information.
- *Spoofing*: sending packets with a forged source address to avoid identification or impersonate another agent by stealing its identity.
- *Hijacking*: also called Man-in-the-middle, combining sniffing techniques with spoofing techniques to divert communications to a malicious agent, who can thus intercept and modify traffic invisibly to the compromised systems.
- *Flooding*: overwhelming the network with unwanted traffic, leading to the blocking or unreachability of a service (Denial of Service - DoS).

At the *application level*, we find attacks such as:

- *Phishing*: an attack that usually occurs via email by sending messages that appear to come from a trusted organization (e.g., a bank) with the aim of stealing confidential information (e.g., login credentials).
- *Malware*: software created with the intentional purpose of causing more or less severe damage. The main types of malware are listed here. (i) *Virus* – a software fragment that, once executed, is able to infect files in order to reproduce by creating improved and more powerful copies of itself. (ii) *Worm* – a particular category of malware able to replicate itself automatically; unlike a virus, a worm is itself an executable program and does not need to infect other programs to spread. (iii) *Trojans* – named for their hidden functionality

inside an apparently useful program; the user, by installing and running a program, may unknowingly also install and run hidden code. Thus, unlike viruses or worms, Trojans do not spread autonomously. Trojans are often hidden in programs downloadable from the Internet. (iv) *Ransomware* – a type of malware that acts on the infected device by limiting access until a ransom is paid.

- *Backdoor*: the designer may leave a secret "door" in a program to bypass some of the system's security procedures. More often than not, these doors are intentionally left open by system administrators to facilitate infrastructure maintenance operations. However, backdoors can also be exploited to compromise the system. Through these doors, an external user can take remote control of the machine without authorization.
- *Spyware*: software used to collect information from the system on which it is installed and transmit it to an interested recipient. These programs are used to gather information about a user's online activities without their consent. The term spyware is also used to define a wide range of malware with various functions, such as sending unsolicited advertisements (spam), modifying the browser homepage or favourites list, or engaging in illegal activities such as redirecting to fake e-commerce sites or installing diallers that call premium-rate numbers.
- *Zero-day vulnerability attacks*: defined as software vulnerabilities unknown to developers (or known but not properly addressed). A zero-day vulnerability means the software vendor, unaware of the flaw, has "zero days" to fix the problem, with resulting security implications. Famous zero-day attacks include the Stuxnet virus/worm, Operation Aurora (2010), and the SecurID hack carried out in 2011 against RSA SecurID.

Finally, it is also necessary to consider *who* to defend against. Attacks may come from both *third parties* (people outside the organization to be protected) and *internal users* of the system, who use their rights to access information beyond their competence. The access points to a communication network are potential attack points. Moreover, users—both end-users and system or network administrators—who control machines and routers and have privileges to delete/alter network traffic are internal users who must be monitored. Often, attacks come from unauthorized external personnel, but even more often from legitimate users.

After this overview of the security properties to be guaranteed in an information system and the most common attacks, the rest of the chapter will describe some of the basic techniques used to counter such attacks to ensure the aforementioned properties.

The general framework adopted foresees the presence of a *subject* (whether a person or an application) needing to access an *object* (whether a database, an application, or a network service). It is therefore necessary to guarantee subject authentication, to verify that the subject has the right to access the requested object, and to prevent third parties from intercepting communications between subject and object.

Among the fundamental techniques, cryptography plays a primary role. Thanks to cryptography, it is possible not only to protect the confidentiality of transmissions (confidentiality property) but also to provide solutions for authenticity and integrity. Below, the basic principles of commonly used algorithms are illustrated. Next, the possible authentication techniques are described, which, as noted earlier, may rely on—but are not limited to—cryptography.

Finally, access control issues are addressed, both for databases and for network aspects (examining the architectural components Firewall and Intrusion Detection Systems - IDS).

It should be emphasized that *designing security* means defining *rules* that establish which users may access which system data and in what ways (read-only, read and write, delete, and more). A system of rules constitutes a *security policy document*. It is then necessary to define *security mechanisms* capable of implementing such policies, such as security and authentication mechanisms, security views for databases, cryptography for communications, or physical protection architectures (firewalls, VPNs, tunnelling systems, etc.). Policies must be based on the definition of general rules that define a system as *open* (actions not explicitly forbidden are allowed) or *closed* (only explicitly permitted actions are allowed), or as *discretionary* (users are owners of their resources and can delegate privileges to others) or *mandatory* (fixed and immutable access rules, as in military systems). Other policies are defined later in the text.

10.3 Cryptography for Data Protection

Encrypting means encoding information, transforming the original message into a ciphertext message. Cryptographic mechanisms consist of an algorithm (*cryptographic function*) and a *key* (or more keys). Only the secrecy of the key (or one of the keys) can guarantee the confidentiality and authenticity of messages. Moreover, the key must be chosen from a vast number of combinations (key space) and changed frequently (for example, session key or One Time Password - OTP).

Cryptographic security depends on the length of the key and the size of the range of possible values that a key can take: the more possible values, the more difficult it becomes for attackers (*attacker*) to recover the key and decrypt a message.

There are two classes of algorithms:

- *Symmetric key algorithms (or secret key)*, which include substitution algorithms, transposition algorithms, and algorithms derived from their combination. For these algorithms, there is only one key, called secret or symmetric, which must be securely shared between the transmitting parties. Examples of this class of algorithms are DES (Data Encryption Standard), used in the 3-DES (Triple-DES) version, and AES (Advanced Encryption Standard), a block cipher algorithm used as a standard by the government of the United States.

- *Asymmetric key algorithms (or public key)*, which are based on complex mathematical problems and on the fact that each subject owns a pair of keys,

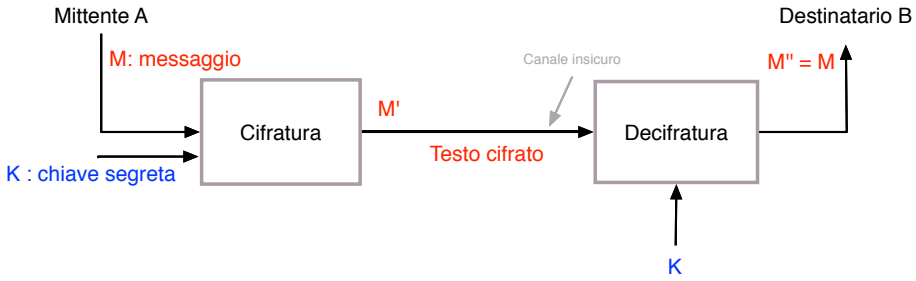


Figure 10.1: Symmetric cryptography

private key and public key, used in a combined way for encoding and decoding. The most widespread and well-known algorithms of this class are RSA, Diffie-Hellman, and El-Gamal.

10.3.1 Symmetric Cryptography

Figure 10.1 shows the operating scheme of symmetric cryptography, where K indicates the symmetric key (secret key) shared between sender and receiver. In the case of sniffing attacks on the insecure channel, attackers, being unable to capture the plaintext M, can only collect a large number of messages on which to perform cryptanalysis with the intent of obtaining the key K. To render cryptanalysis useless and ensure communication security, the key K must change frequently (possibly at each session). The problem is that A and B must find a secure channel alternative to the one used for transmission in order to frequently exchange the secret key K.

The two main techniques underlying symmetric cryptography are substitution and transposition (or permutation).

In Figure 10.2 two fixed *substitution* mechanisms are illustrated, with numeric key and with monoalphabetic substitution. In general, longer text mappings are required to make reconstruction of the original text more difficult. A simple symmetric key algorithm based on the fixed substitution mechanism (mentioned here only as an example) is the Caesar cipher. In this scheme, the message is encrypted by replacing each letter of the message with the letter of the alphabet that is k positions ahead, where k constitutes the secret key to be shared between sender and receiver. For example, if sender A wants to send a message M="ciao" and $k = 4$, the message M' will be equal to "goes" (considering the Italian alphabet).

In *transposition*, the key indicates how to permute the letters of the plaintext during encryption. In the example shown in Figure 10.2, the key indicates into how many columns (7 in the figure) the plaintext is written. The columns are ordered according to the lexicographic order of the letters of the key. Thus, the first letters of the ciphertext correspond to the column associated with the letter A of the key: "fne", followed by the letters corresponding to the column of the letter I: "rib", and so on.

Figure 10.2: Symmetric cryptography: substitution and transposition

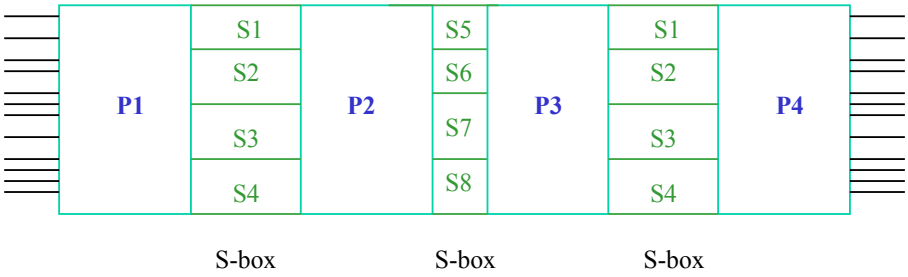


Figure 10.3: Symmetric cryptography: composition of ciphers

As mentioned earlier, substitutions and transpositions, which operate on *blocks* of text, are the basic operations of symmetric algorithms. As shown in Figure 10.3, the ciphertext is obtained by applying to the plaintext a series of substitutions — S-box blocks — and transpositions — P-box blocks (Permutation box).

The standard algorithm is AES, in which the message is encrypted using a key of length 128, 192, or 256 bits. For encryption, the message is partitioned into 128-bit blocks and each block is encrypted through several stages that include xor combinations of the message bits with the key bits, substitutions, transpositions, and combinations among bits in intermediate stages. The key must be long enough to prevent a "brute force" (i.e., exhaustive) attack. In theory, for perfect encryption the key should be as long as the message to be encrypted, as demonstrated by Shannon. In practice, fixed-size, much shorter and more manageable keys are used, while still ensuring a high level of security.

The critical points of symmetric cryptography are related to key management: (i) the key must be changed often to avoid discovery, (ii) the key must be generated and distributed with maximum security.

The main advantage of symmetric cryptography is the speed with which encoding and decoding operations are performed, since these operations can be performed in hardware using dedicated processors.

10.3.2 Asymmetric Cryptography

Asymmetric cryptography (or public-key cryptography) requires that each agent possess a pair of correlated keys: a *public key* K_P and a corresponding *private key* K_{pr}. The correlation between the two keys concerns the principle of an asymmetric cryptography algorithm: everything that is encrypted with one of the two keys can be decrypted with the other key of the pair. Thus, assuming that A's key pair is $\{K^A_{pr}, K^A_p\}$, everything encrypted with K^A_{pr} can only be decrypted with K^A_p and vice versa. An important property of key pairs is that it must not be possible in any way to deduce one of the two keys starting from knowledge of the other. Usually, asymmetric cryptography regards human actors, rather than programs or applications.

The public key can be distributed, while the private key must remain secret, and the agent that owns it must keep it securely.

Since, by the property mentioned earlier, given the public key it is not possible to derive the private key, even though the former is available to anyone in public repositories, the security of the latter is not compromised. This avoids the need to transmit secret keys between agents on the network, who only need to share their public keys, either directly or through a Public Key Distribution Service.

In asymmetric cryptography, two key pairs are used, $\{K^A_{pr}, K^A_p\}$ and $\{K^B_{pr}, K^B_p\}$, owned by A and B, to ensure confidentiality and authenticity. In particular, a first way to use public-key algorithms is shown in Figure 10.4. In this case, the sender A encrypts the message with B's public key, and therefore the message can only be decrypted with B's private key. This ensures the property of confidentiality since B will be the only one able to decrypt the message (being the only one in possession of K^B_{pr}), and consequently, the sender has the guarantee that the message will not be read by unauthorized agents. The weak point of this mechanism is that everyone can obtain B's public key and consequently pretend to be A within the communication, leading to Man-in-the-middle attack problems.

The sender A could also encrypt the message with his private key. In this case, the receiver B must decrypt the message with A's public key (see Figure 10.5). In this way, the exchange is authenticated, since by encrypting with his private key the sender A certifies his identity.

However, the message written by A is easily readable also by third parties in possession of A's public key. To guarantee both the property of authenticity and the property of confidentiality, the two approaches described above are combined with a double encryption phase, as shown in Figure 10.6.

The principle on which asymmetric cryptography is based is the use of a simple function f, whose inverse f^{-1} is complex. For example, calculating the product

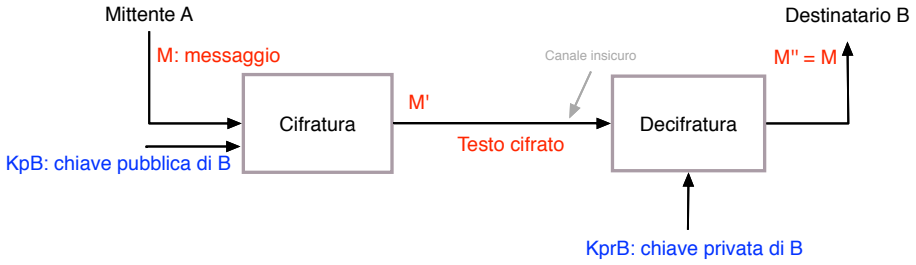


Figure 10.4: Asymmetric cryptography (confidentiality)

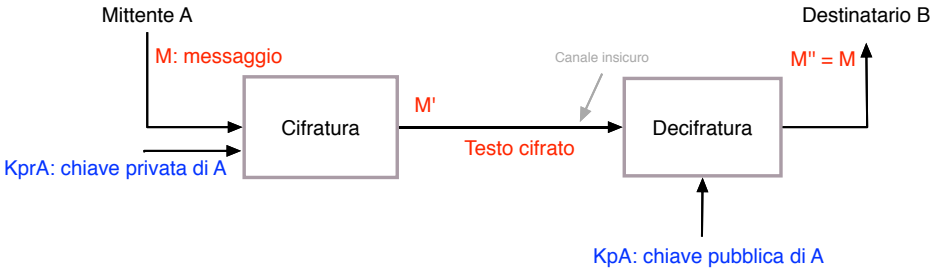


Figure 10.5: Asymmetric cryptography (authenticity)

of two prime numbers (f) is simple, while f^{-1} (finding two prime numbers that, given a starting number, are its factors) is complex, especially if the number to be factored is very large ($> 10^{100}$).

The asymmetric cryptography scheme was proposed by Diffie and Hellmann in 1976, based precisely on the product of two very large prime numbers. Another of the most famous algorithms, the RSA algorithm, invented in 1977 by Ronald Rivest, Adi Shamir, and Leonard Adleman, is also based on the difficulty of finding the prime factors of large numbers, and the construction of the key pair occurs according to the following steps:

1. two prime numbers P, Q of high value are chosen;
2. compute $N = P \times Q$ and $Z = (P - 1) \times (Q - 1)$. N, as we will see, will be the common component of the two keys;
3. now two values E and D must be chosen to compose the rest of the keys;
   - E (with $E < Z$) must be chosen among the numbers that have no common factors with Z (E, Z are said to be relatively prime);
   - D must satisfy the following equation $(E \times D)\ mod\ Z = 1$, thus $(E \times D) - 1$ must be divisible by Z

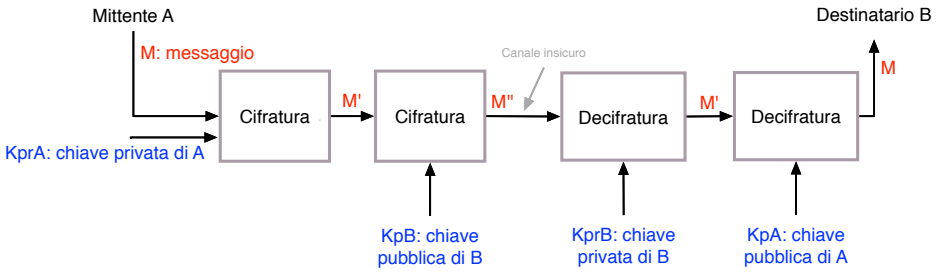


Figure 10.6: Asymmetric cryptography (confidentiality and authenticity)

4. the public key K_p and the private key K_{pr} are then defined as: $K_p = (E, N), K_{pr} = (D, N)$

For example (with small numbers), suppose the starting values are $P = 5$ and $Q = 7$. By definition, $N = 35$, $Z = 24$. A suitable value for E is 5 since E and Z have no common factors and $E < Z$. Finally, a valid value of D is 29 since $5 \times 29 = 145$ and the division $145/24$ has remainder equal to 1.

From the obtained data, it is possible to state that $K_p = (5, 35), K_{pr} = (29, 35)$.

Having the keys available, let M be the message to be encrypted and C the encrypted message:

- encryption: $C = M^E \ mod \ N$
- decryption: $M = C^D \ mod \ N$

The disadvantage of the RSA algorithm, and asymmetric key algorithms in general, is its computational slowness. In fact, the computation of the exponentiation of large numbers requires significant computational power for very long messages. For this reason, asymmetric encryption is used to encrypt the keys that must be exchanged between agents in private-key algorithms, while symmetric encryption is used for encryption of the exchanged messages or texts.

10.3.3 Integrity and hash function

Symmetric and asymmetric cryptographic algorithms guarantee the properties of confidentiality and authenticity. To also provide a guarantee in terms of integrity, one must rely on *hash functions*. A hash function is a function that transforms a message M of arbitrary length into a fixed-length output called the *digital fingerprint*, *hash*, or *digest* of the original message. *Hash* algorithms are public and well known. The hash function is used in this context because it is characterized by the following properties:

- Consistency: identical messages must be associated with the same digest.
- Uniqueness: the probability that two different messages (even differing by a single bit) are associated with the same digest must be nearly zero.

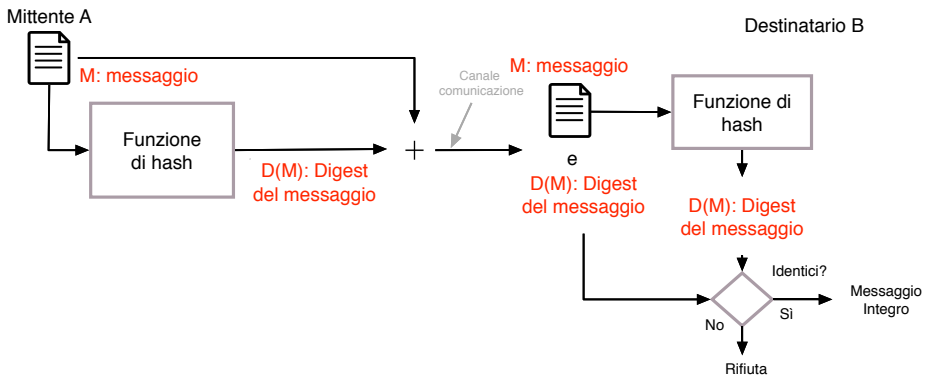


Figure 10.7: Use of the hash function

- Non-invertibility: the function must not be invertible. It must be impossible to derive the message from the digest.

Hash functions thus generate digital fingerprints of the message, which can constitute proof of the integrity of the message itself, i.e., proof that it has not been tampered with by unauthorized agents during communication. In fact, as shown in Figure 10.7, typically a sender who wants to send a message can compute its digital fingerprint through a hash function and send the message concatenated with the fingerprint. Once the message is received, the recipient must decouple the message from the fingerprint and recalculate the fingerprint of the received message, generating the "fresh" digest. If the received fingerprint and the one calculated from the received message are equal, then one can say that the message is intact; otherwise, the message has been tampered with by unauthorized agents during some stage of transmission.

Figure 10.7 shows the principle underlying the use of the hash function, but for safe use of the function one must also protect the fingerprint from external attacks. Consequently, the fingerprint must be used in combination with public-key systems, as shown in the next section.

10.3.4 Digital signature

The purpose of creating and appending a *digital signature* (or electronic signature) to a piece of data/information is to guarantee the authenticity of the data and to reliably identify its creator [12]. The digital signature scheme is shown in Figure 10.8. The digital signature is defined as the encrypted digest of a document (or message), obtained using the sender's private key. The signature is attached to the document; then, signature and document are sent, also separately, since the document encryption or non encryption is a mechanisms which is decided apart, independently on its signature. For the signature, before transmission, the signature is encrypted with the recipient's public key, who must use their private key to

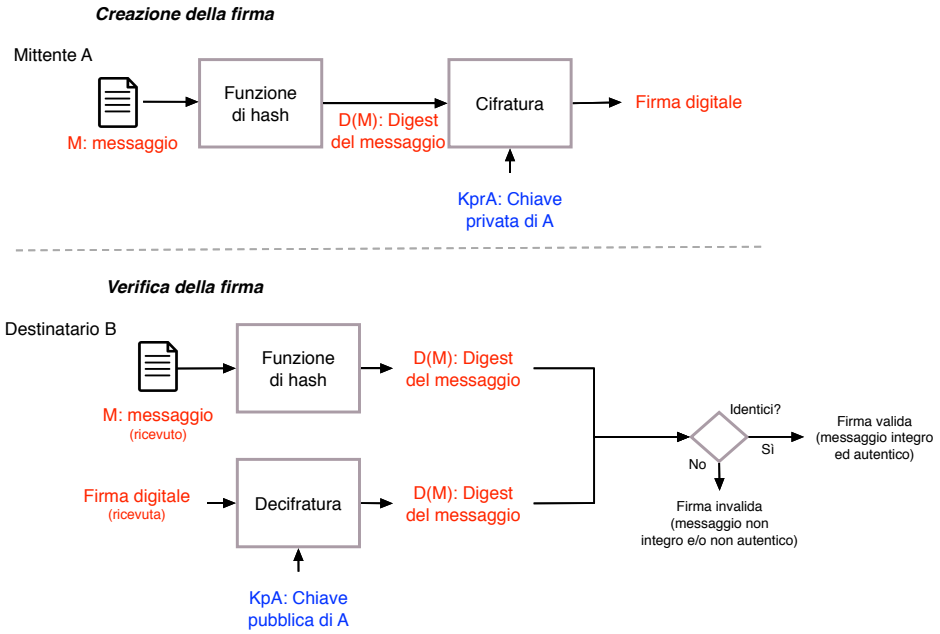


Figure 10.8: Digital signature

decrypt what was received. The document is sent in encrypted or in clear, depending on the desired level of security.

Once the signature and the document are received, two actions are performed in parallel: (i) recalculation of the digest of the document; (ii) decoding of the signature with the sender's public key and retrieval of the original digest of the document. At this point, the two digests can be compared to determine whether the document has been tampered with by unauthorized agents. In case the document has been encrypted, it is decrypted apart.

The digital signature requires a long processing time and is generally not used in network processes that require high performance, but only in applications where a human agent is directly involved (using SSL, TLS, and S/MIME protocols). Moreover, the digital signature, since it depends not only on the signer's private key but also on the data through the digest, is different for every distinct set of data that is signed. The signature affixed to an electronic document therefore cannot simply be copied and affixed to a different document. Finally, it not only identifies who generated the data, but also proves that the data have not been modified after being signed, and is therefore stronger than a handwritten signature (property of *non-repudiation*).

10.3.5 Aspects of key management and digital certificates

In order to apply cryptographic algorithms, and sometimes even digest algorithms, it is necessary that the parties involved possess and share the keys. This aspect is called *key management* and requires solving various sub-problems, such as key generation, key storage, and key exchange (key exchange or key distribution).

Key generation must be carried out directly by those who performs the cryptographic operations; in exceptional cases (e.g., in trusted high-security environments like military applications), it is possible that keys are generated by an entity different from the one that will use them. Cryptographic keys should be random bit strings; Random Number Generators (RNG) can be used for their generation. The quality of the RNG determines the quality of the security system in terms of secrecy, integrity, and other provided security properties (e.g., level of trust obtained for the keys).

Regarding *storage* of cryptographic keys, especially private ones, software or hardware solutions can be used. In the former case, keys can be stored in an encrypted file protected by a password. As for hardware solutions, keys can be stored inside removable hardware devices (e.g., external hard drives) or in "active" devices capable of storing keys and autonomously performing cryptographic operations (e.g., smart cards, USB tokens, or PC cards).

Key exchange is particularly critical in symmetric cryptography, since knowledge of the key would entirely compromise communication. One of the most widely used techniques for key exchange is the OOB (*Out-Of-Band*) technique, in which the key is distributed through a channel different from the one used for data transmission. Alternatively, if the recipient of the encrypted data holds an asymmetric key pair, the sender can send them the secret key by encrypting it with the recipient's public key. Finally, one can also rely on a third party (*key distribution center*).

If the key to be exchanged is public, secrecy is not important: the problem is correctly associating it with the identity of the owner. Public keys are *distributed* through a data structure called a *Public Key Certificate* or PKC, of which there are several formats, including X.509v3.

Systems that support key generation and management are called PKI (*Public Key Infrastructure*). PKIs are infrastructures that provide methods and tools to implement basic cryptographic functions for a certain community of users, such as:

- issuance of public key certificates, after performing the necessary technical and procedural checks;
- revocation of public key certificates (for example, following theft of the private key associated with the public key);
- distribution of public key certificates and information about revoked certificates.

Optionally, a PKI can also provide support for validating a digital signature through functions identifying the time at which the signature was affixed (timestamp), the role held by the individual who signed (role certification), and the reason why the signature was affixed (signature policy).

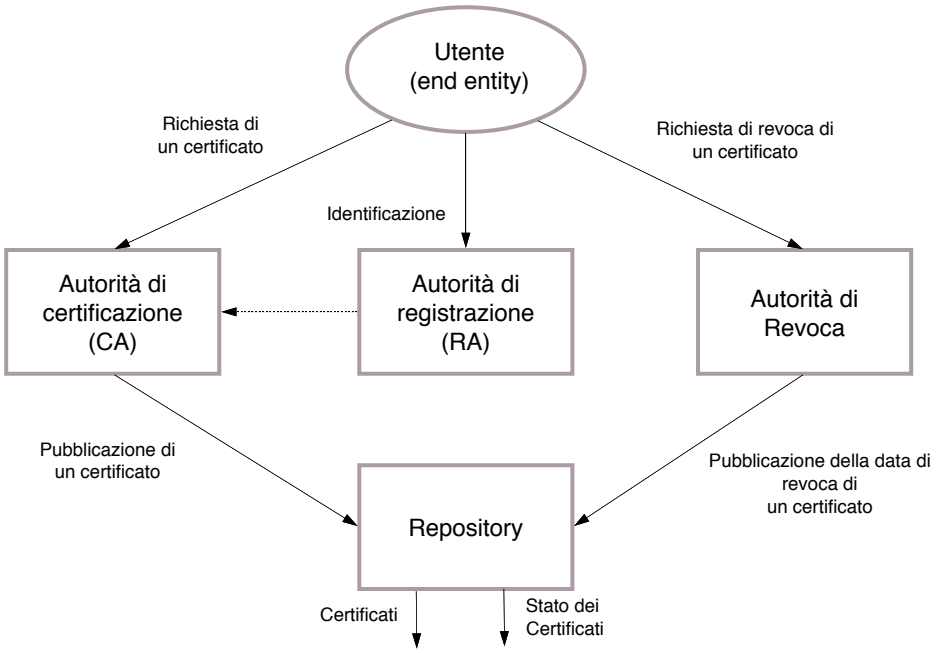


Figure 10.9: Actors and functions of a Public Key Infrastructure

The schema of a PKI is shown in Figure 10.9: the user (viewed as the end entity of the scenario - reminding that the entity involved in asymmetric cryptography is usually a person- requests a personal certificate from the Certification Authority (CA), which acts as a notary, guaranteeing the identity of the parties and also acting as a Registration Authority (RA). Alternatively, the CA may rely on a trusted third party for the RA role, responsible for identifying the user through a specific phase of credential acquisition. Certificates are stored in a Repository appropriately shared within the cooperative environment, which is accessed to verify the identity of a party (e.g., an Internet Banking Server where sensitive data are being entered). This Repository must also release information on the status of certificates, which have an expiration date or may be revoked at the request of the CA or RA. A Revocation Authority is responsible for revoking expired certificates. It is important that the revocation of a certificate be promptly disclosed in the environment to prevent fraud involving expired certifications.

An example of a certificate is shown in Figure 10.10, where the various fields also indicate the encryption algorithm used (RSA), the hash algorithm (MD5), the key length (1024 bit), the certification and issuing authorities (PoliMI), validity, and the holder of the signature.

To ensure that the certificate cannot be altered, it is protected with the digital signature of the issuing Certification Authority. When receiving a public key through a certificate, it is necessary to check the integrity and validity of the sig-

Campi:	Esempio:
• version	1
• serial number	2430
• signature algorithm	RSA with MD5, 1024
• issuer	C=IT, O=PoliMi
• validity	1/1/2006 - 31/12/2020
• subject	C=IT, O=PoliMi CN=MariagraziaFugini
•subjectpubickeyinfo	RSA, 1024, xxxx…xxx
•digital signature	yy…..yyy

Figure 10.10: Example structure of a public key digital certificate

nature: the digest computed on the certificate (i.e., on the public key and the associated data) is compared with the digest extracted from the digital signature affixed by the CA that issued the certificate.

If this comparison is positive, the certificate is certainly intact, but it remains to be verified whether the CA that issued it is a trusted CA. For this reason, a list of trusted CA public keys is maintained.

The validity of a digital certificate is guaranteed by the CA. For this reason, a digital certificate is a document containing information on who owns the public key, the public key itself, and it is all signed by the CA to ensure that no one has altered the certificate.

Examples of use of public keys and certificates include: e-mail message exchange, proprietary applications (for remote control, or for implementing security modules), access control to hardware devices (routers or network hardware), at the application level (e.g., for e-commerce applications or for implementing secure websites), and obtaining signed, thus trusted, software components.

10.4 User management and access control

Figure 10.11 provides an overview of secure user management through the various mechanisms illustrated so far and in the rest of this section.

10.4.1 Authentication and authorization

Before any security check, it is necessary to *authenticate* agents before allowing them to open a communication channel or access information and applications located in the Intranet or protected areas. Authentication mechanisms allow the system to reliably recognize an agent, requiring credentials that take them a step further in accessing the system compared to a simple registration (used, for example, for

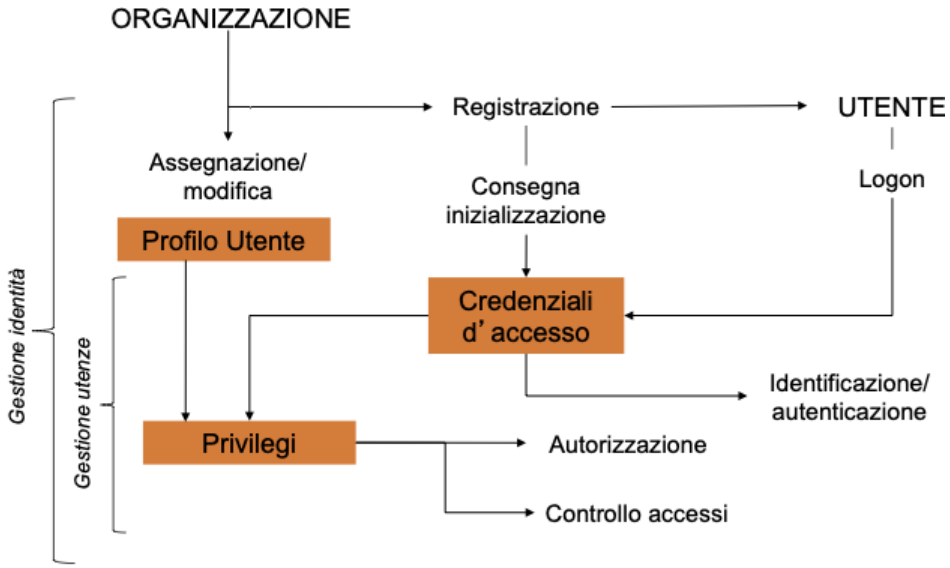


Figure 10.11: User management with authentication methods and other security mechanisms

browsing information). With authentication, it is possible to filter access modes to application data and from the DBMS or other components of the information system.

An agent can be identified by exploiting one or more of the following characteristics:

- SOMETHING YOU KNOW - SYK: a known piece of information (e.g., password).
- SOMETHING YOU HAVE - SYH: an object possessed (e.g., a magnetic card).
- SOMETHING YOU ARE - SYA: a physical characteristic – in the case of people (e.g., fingerprint, retina, DNA).

Simple authentication is based on userid and password, while strong authentication is based on mechanisms such as One-Time Passwords (OTP), on challenge-response systems, where the password is never transmitted but only used to perform a calculation that indirectly proves knowledge of it: response = F(challenge, password), or by using personal devices (tokens) as with smart cards, or finally on biometric characteristics.

In the next section, multi-factor authentication techniques are illustrated.

10.4.2 Two- or multi-factor authentication

Multi-Factor Authentication (MFA) is an increasingly practiced solution for rapid, simple, and reliable authentication in communications, online payments, and access

rights management. MFA is continuously evolving because the conventional method based on passwords to protect account access is vulnerable to a growing number of attacks: smudge attacks, i.e., extracting passwords from touchscreen devices using fingerprint smudges; guessing attacks, i.e., brute force techniques; shoulder-surfing attacks, a social engineering technique based on observing the victim's behavior.

Two- or multi-factor authentication (*strong authentication*) is currently the most secure protection system.

To access any digital system (computer, ATM, websites, or others) one must first "present" themselves by entering their username, then "prove" that they are who they claim to be (using SYA, SYH, SYK techniques).

It is called 2FA if at least two of the three factors listed above are used. The condition for it to be defined as "two-factor authentication" occurs only when the two factors used are of different natures: for example, using "something you know (SYK)" + "something you have (SYH)". For example, authentication through two passwords (two factors of the same nature) cannot be considered 2FA.

There is also three-factor authentication (3FA), still rarely used. One of the best-known cases where it is employed is in SPID (the Public System of Digital Identity) level 3 (available with Poste Italiane and Aruba). Here authentication is based on all three factors: password, the PosteID App, and the SPID PIN. Conversely, version 2 of SPID uses 2FA: password and the PosteID App.

In 2FA, the second factor, which in most cases is a numerical code, is obtained via smartphone (SMS or dedicated application) or via a physical token.

Unlike a password, the second code is practically unassailable, because it is generated pseudorandomly according to a specific algorithm and has a very limited lifetime (usually 30 seconds). For this reason, it is also called an OTP: *one time password*.

There are numerous applications for tokens: the most well-known – all free – are Authy, Google Authenticator, Microsoft Authenticator. Even the best password management systems offer this functionality.

Except for internet banking services, where it is mandatory, in all other sites we are not required to use 2FA. It is an optional but highly recommended feature for services such as Amazon, Apple ID (iCloud), Dropbox, Evernote, Facebook, Google, LinkedIn, Microsoft, PayPal, Twitter, Yahoo!, WordPress, and so on. In practice, all major sites make it available (and in many cases recommend it). On the web it is possible to consult the complete list of hundreds of sites where it can be activated, divided by category. For each site, the available options are indicated (SMS, hard token, soft token, etc.). An additional advantage of using 2FA: if we activate it, we will no longer be asked security questions.

10.4.3 Data access control systems

While authentication makes it possible to verify the validity of an identifier (credentials), associating an agent with a precise *identity*, *authorization* is the phase in which it is verified whether the agent is authorized to perform the requested function, such as operating system functions (e.g., file access), application functions (business process, task), and programs.

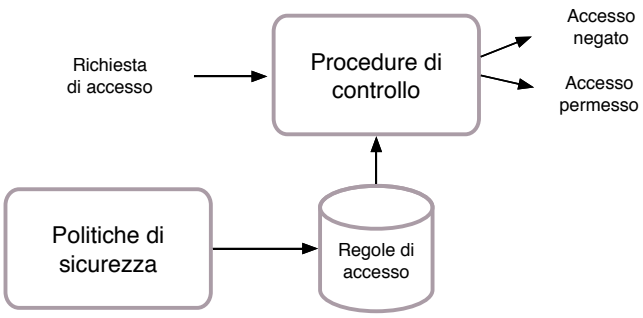


Figure 10.12: Access control system

In particular, *access control* is the set of procedures that verify whether the agent is authorized to perform elementary functions (read, write, execute). It can be carried out by the operating system, by the DBMS, or by specific security programs/services, thereby limiting the damage that can be caused by illegitimate users and/or malicious programs because security services are designed specifically for the resources to be protected.

In the next section, access control for data stored in databases is examined in more detail. When talking about *secure DBMS*, this refers to the DBMS policies and mechanisms for protecting data from illegitimate access. Compared to the access controls implemented by an operating system, for the DBMS the important aspects are the granularity of the protected objects, the semantic correlations of the data, the existence of metadata (dictionaries), which must also be protected, the multiplicity of data types, the fact that static objects and dynamic objects (e.g., views) exist, and the data life cycle (longer than that of an operating system object). Security mechanisms in DBMSs have as requirements the need to handle data at different granularities, to filter different access modes and types of control (inference, flow, access controls), to manage dynamic authorization (grant/revoke), and to ensure the absence of backdoors, while overall providing reasonable performance [8].

In general, the access control system works according to the scheme shown in Figure 10.12.

Access controls in DBMSs are based on *access rules* composed of:

- *Authorizers*: when the concept of ownership of resources applies (i.e., whoever creates a piece of data becomes its owner), the owner of a resource is allowed to manage its accesses selectively, thus becoming the authorizer for that resource. More generally, the authorizer is, for example, the Database Administrator (DBA) or the security officer, who have a control function over the data. For centralized databases, it is useful to have a single authorizer, while in distributed databases there are various authorizers organized, for example, in hierarchies.

- *Subjects*: these are users (defined by their user profile), administrators and application programmers, groups or classes, transactions, or applications. If the user belongs to multiple groups and/or uses various applications, the subject is identified by a tuple of the type:
  - (USER-ID, GROUP-ID) or
  - (USER-ID, APPL-ID) or
  - (USER-ID, ROLE-ID), which identifies the user through their business role (role-based access control).
- *Objects*: the choice of granularity of protected objects is crucial for the effectiveness/efficiency of controls. A protected object is, for example, a table, a column, a tuple, an element. Schemas and dictionaries must also be protected.
- *Rights*: indicates access modes (e.g., read, write, execute for programs, or CRUD – create, read, update, delete for data).

The definition of rules is dictated by *security policies*. In general, it is possible to distinguish between two approaches: *closed system* and *open system*. Access policies in a closed system state that the permitted accesses are all and only those specified by the rules. Therefore, everything not explicitly permitted is to be considered forbidden. Conversely, in an open system, permitted accesses are all except those expressly prohibited by rules. Therefore, everything not explicitly prohibited is to be considered allowed.

The access rules defined by the authorizers are based on a *basic three-component model*:

(subject, object, right)

The model was then extended to the four-component model: (subject, object, right, constraint), where constraints express content-dependent access restrictions. Constraints make it possible to address aspects of data semantics, because they allow the expression of access filters dependent on the business meaning of the data – for example, the attribute SALARY may be granted in "read" only to the salary holder themselves. Other constraints may also be expressed, denying, for example, the possibility of combined access to "related" data – which would lead to inferring data for which there is no reading right, but which can be deduced as combinations of data accessed across various, apparently unrelated transactions (history-dependent constraints).

An example of a relational SYSAUTH table used to express and manage access rules by the security administrator is shown in Figure 10.13.

In the figure, access to EMP (relation with attributes NAME, ADDRESS, AGE) may be performed in any mode (ALL) by managers (mgr), in read-only mode (READ) on attributes NAME and ADDRESS for employees for whom "mgr" (i.e., the user authenticated in the system with MANAGER role credentials) is not the manager. The subject employee may access their own data in READ and UPDATE,

subj	obj	right	constraint
mgr	EMP	ALL	= mgr of NAME
	NAME	READ	NOT mgr of NAME
	ADDRESS		
employee	NAME	READ,UPDATE	employee=NAME
	AGE		
	ADDRESS		
	NAME	READ	mgr(NAME)=
	AGE		mgr(employee)
	ADDRESS		

Figure 10.13: Example of access rules table for the EMP relation

and may read (READ) the data of employees managed by the same manager (e.g., belonging to the same department).

The 'constraint' part of the four-component model is implemented through *mechanisms* based on Protection Views on the database: the constraint defines a view of the legitimate data for a user by defining a view. The model provides name-dependent access control (e.g., based on table/attribute name), content-dependent access control, as just illustrated, but still does not provide controls on the administration of the set of access rules (who can change, manage them, etc.) nor on the delegation of access privileges. Models with five or more components are needed to express more granular access rules linked to data semantics and to the dynamic nature of the data.

Before illustrating other rule models, *data access policies* are introduced, particularly grouped into two categories:

- Discretionary policies.
- Mandatory policies.

In a system managed based on discretionary data access policies (*Discretionary Access Control* – DAC), it is at the "discretion" of users to administer the data they own; in fact, in DAC systems the concept of *owner* applies, who may authorize other

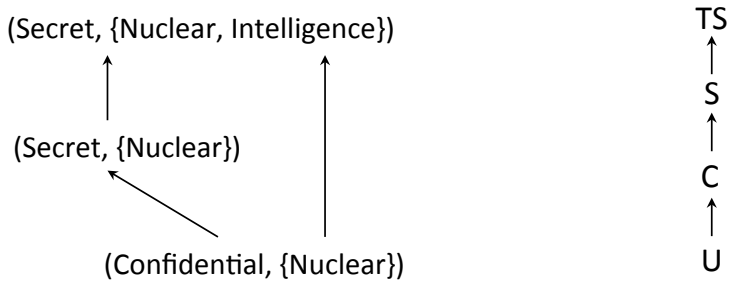


Figure 10.14: Security Classes and sensitivity levels (MAC)

users' access, define the type of access to be granted (grant) to others (read, write, execute, append), define selective accesses, for example, based on name, content, system parameters, history, or data aggregation.

Systems managed based on mandatory data access policies (*Mandatory Access Control* – MAC) are suitable for databases with sensitive data (government, military, healthcare, ...) where DAC is not sufficient because vital information is stored, with different levels of sensitivity, and where controls on data flow, data release, and user behavior are also necessary. A characteristic of such sensitive systems is that they are subject to sophisticated attacks by determined users (e.g., Trojan Horse-based attacks). MAC policies classify objects, assigning them a *sensitivity level*, and subjects, assigning them a *clearance level*.

Security classes (SC) are also defined as:

SC = (hierarchical-component, set-of-categories)

where the hierarchical component identifies the level (with a (partial) ordering between levels) and the categories identify the data ownership areas (e.g., departments or application areas). A dominance relation is defined between SCs (see, for example, [22] for MAC-type databases).

Security mechanisms implementing MAC policies must ensure that all subjects have access only to data for which they hold the appropriate clearance. The basic rules are:

1. *No-Read-Up* (for read operations), called the Simple Security Property: A subject S is authorized to read an object O only if $L(S) >= L(O)$.

2. *No-Write-Down* (for write operations, which could degrade the data, when written, for example, to a file classified lower than the clearance level of the writer). This is called the *-property (Star Property): a subject S is authorized to write to an object O only if $L(S) <= L(O)$.

These rules, combined, prevent *data flows* between "high" and "low" subjects.

Typical classification levels (data sensitivity and user clearance) are: Unclassified (U), Confidential (C), Secret (S), Top Secret (TS).

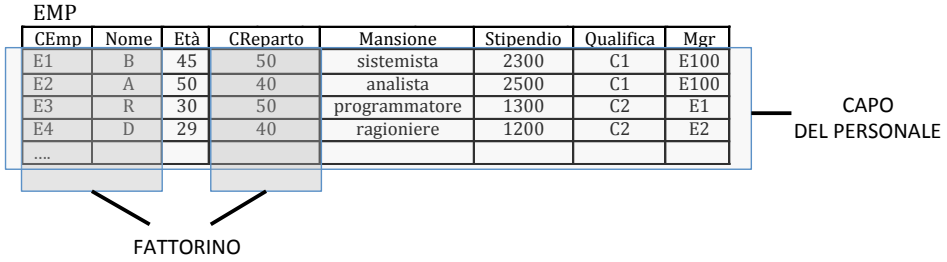


Figure 10.15: Role-based access (DAC)

Examples of categories are: 1. Nuclear, NATO, Intelligence, or 2. Production, Personnel, Engineering.

Figure 10.14 shows examples of comparisons between levels and between SCs. On the left, the "dominates" relationship between SCs can be seen; on the right, the hierarchy of levels is shown, from Unclassified up to Top Secret.

The implementation of protection in MAC DBMSs for controlled access to multilevel data uses a Trusted Computing Base, that is, a set of trusted hardware and software components for the storage and management of classification labels directly in the database, possibly in encrypted form or in a separate database managed by specific filters. Performance varies in the different cases.

DAC and DBMS

In Figure 10.15 it is shown how access control based on the name of the EMP table and its attributes can be implemented according to the user role.

Access rules can be specified in the user profile, or associated with the object description. Basic mechanisms for specifying authorizations can be inserted in the DBMS Query Language or in OS commands.

In Figure 10.16 the access control based on the content of EMP, shaded in gray, is shown. In this case, a query defines the protection view D50 containing only the data of employees in the department with CReparto=50, a view that is then granted (grant) in read-only mode to the user "John".

For example, in a relational DBMS we can have:

```
GRANT INSERT, SELECT ON EMP TO John
```

The GRANT clause assigns the rights to perform insertion (INSERT) and query (SELECT) operations on the EMP table to the user John.

Alternatively, if a view D50 is defined on the EMP relation as in Figure 10.16, access rights on the view can be specified as follows:

```
CREATE VIEW D50(CEmp,Name,CReparto) AS
   SELECT CEmp,Name,CReparto
```

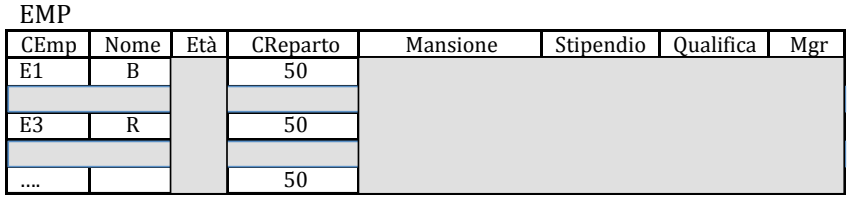


Figure 10.16: Content-based access (DAC)

```
    FROM EMP
    WHERE CReparto=50

GRANT SELECT ON D50 TO John
```

Another mechanism is *query modification*, where the query is modified by adding restrictions relative to the original query, to enforce the rules defined for data access.

For example, if John can read EMP data only for salaries below 1500, and then submits the request:

```
SELECT * FROM EMP
```

the DBMS executes the following modified query:

```
SELECT *
FROM EMP
WHERE Salary < 1500
```

It is now possible to illustrate the six-component model in DAC environments:

$(gtor, gtee, obj, right, constraint, prop_rule)$

where *prop-rule* is the privilege propagation rule, which specifies if a privilege can be granted (and under what conditions) to other users in DAC environments. There are three possibilities: 1. no propagation, 2. unconditional propagation (Unbounded Propagation), 3. limited propagation (Bounded Propagation, e.g., for a limited chain of users or for a maximum time).

In any case, the problem of *revoking granted privileges* must be addressed, especially for privileges that a user may have received multiple times from different users (see [8, 22] for details on revocation).

DAC mechanisms in DBMSs can be *role-based*, e.g., using user roles in an organization (manager, HR head, employee, warehouse worker, courier, etc.). Through roles, authorizations are associated with the role rather than individual users, facilitating the writing and maintenance of the set of access rules, which are then stored

Name	CName	Dept	CDept	Salary	CSalary	CTuple
BoB	S	Dept1	S	10 K	S	S
Ann	S	Dept2	S	20 K	TS	TS
Sam	TS	Dept2	TS	20 K	TS	TS

Figure 10.17: Example of a multi-level relation with MAC-classified data

in a dedicated system repository to be accessed when deciding whether to grant access. This provides flexibility in authorization management and alignment with the organizational structure.

MAC and DBMS

For MAC databases, multi-level relational tables are created with different granularities of data classification: relation level, attribute, record (tuple), element (data). A fundamental mechanism is *polyinstantiation*, i.e., multiple physical copies of the same table are instantiated separately according to their security classification. First, the TCB (Trusted Computing Base) mediates all subject-object accesses in a "trusted" manner, i.e., via components that cannot be tampered with (Reference Monitor concept).

Polyinstantiation occurs through rules for classifying elements in the database where a *Multi-Level Relation* (MLR) is defined as a relation R:

$$R(A1, C1, ..., An, Cn, TC)$$

where TC is the Tuple Classification attribute, which assigns a level to the row (tuple) of the table, and C_i is the sensitivity of attribute A_i.

Access to an MLR occurs as follows:

- Read: a subject can read MLR instances at their own level or below.
- Write: polyinstantiation may occur if the element to be inserted already exists with that name. In particular, if the requesting subject has a lower or incomparable classification relative to the element to be written (non-visible element), polyinstantiation prevents illicit inference from legally accessed data. If the requesting subject has a classification that dominates the element to be inserted, polyinstantiation prevents denial of service.

An example of a relation in which data is classified according to the MAC method is shown in Figure 10.17. Each attribute has a corresponding attribute describing its sensitivity (CAttributeName). The first tuple contains Secret data and is globally labeled Secret (S). The other two tuples are each labeled at the highest level contained, i.e., Top Secret (TS), because they contain at least one Top Secret attribute.

Name	CName	Dept	CDept	Salary	CSalary	CTuple
BoB	S	Dept1	S	10 K	S	S
Ann	S	Dept2	S	20 K	TS	TS
Sam	TS	Dept2	TS	20 K	TS	TS

Figure 10.18: Top Secret level instance

Name	CName	Dept	CDept	Salary	CSalary	CTuple
BoB	S	Dept1	S	10 K	S	S
Ann	S	Dept2	S	–	S	S

Figure 10.19: Secret level instance (S)

For the relation illustrated in Figure 10.17, instances corresponding to different access classes are created, in this case Top Secret (Figure 10.18) and Secret (Figure 10.19).

The Top Secret instance contains the full table content shown in Figure 10.17, while the Secret instance in Figure 10.19, which contains the instantiation of the table at Secret clearance level, does not have a value for Ann's salary.

In multi-level relations, a null value can therefore have two meanings: either a true null, or an attribute whose classification is higher than the instance.

Since sometimes the absence of a value allows inference of other values (e.g., TS values for Ann), in the Secret instantiation of the table, fictitious values compatible with TS can be inserted (e.g., a salary of 10K instead of the TS value of 20K).

An alternative is to have polyinstantiation in the table, which allows multiple rows with the same primary key. In this case, two tuples with the same primary key but different sensitivity levels can exist, as shown in Figure 10.20. Here, the Top Secret instantiation provides Ann's salary as 20K, while the Secret instance provides 15K.

The Top Secret instance will contain both tuples (polyinstantiation) and have the same content as in Figure 10.20.

Name	CName	Dept	CDept	Salary	CSalary	CTuple
BoB	S	Dept1	S	10 K	S	S
Ann	S	Dept2	S	20 K	TS	TS
Ann	S	Dept2	S	15 K	S	S
Sam	TS	Dept2	TS	20 K	TS	TS

Figure 10.20: Example of multi-level relation with polyinstantiation

Example of a database attack: SQL Injection

For example, using a database to authenticate users with the following query:

```
SELECT username, password FROM users
WHERE username = 'inputUsername' AND password = 'inputPassword'
```

the logic is that if the query returns a tuple, the user is authenticated; otherwise not. The problem arises when the user enters a valid Username and as Password the following string:

```
'AString' OR 'a'='a'
```

The resulting query would be:

```
SELECT username, password FROM users
WHERE username = 'inputUsername'
           AND password = 'AString' OR 'a'='a'
```

The user is still authenticated because the query always returns at least one row. This is an example of an SQL injection attack.

10.5 Infrastructure Security Mechanisms

Tools capable of providing countermeasures against attacks on an information system at the physical architecture level mainly consist of creating a secure configuration for the operating system, clients, and servers, and implementing, through appropriate *architectural components*, policies for user authentication, access control, and activity logging.

This section illustrates the architectural components for security, specifically firewalls and/or proxy servers, placed at the connections with public external networks to filter access, and Intrusion Detection Systems (IDS) capable of monitoring events occurring in the system and on the network.

In particular, a Firewall is a set of hardware and/or software components that connect a trusted network to untrusted networks. It controls incoming and outgoing traffic from the secure network by implementing appropriate security policies, with the goal of preventing, detecting, and stopping unauthorized attacks and requests. An Intrusion Detection System (IDS) is a hardware and/or software component that monitors events in a system or a computing network, aiming to detect intrusions, defined as attempts to compromise the confidentiality, integrity, or availability of the system.

10.5.1 Firewall

The term *firewall* generically identifies a set of components and services aimed at controlling and limiting traffic between a network to be protected and the external world.

The architectural schemes according to which firewall components (mainly routers and application gateways) can be arranged to protect a network are numerous and must be evaluated by the designer based on several factors, including security, flexibility with respect to controlling new protocols and applications, performance of the firewall subsystem and the network as a whole. Installation and maintenance costs are also relevant.

Firewalls represent the first line of defense for network security. They constitute a barrier between secure internal networks (Intranet and/or Extranet) and potentially unreliable external networks, such as the Internet. Installing a firewall does not guarantee security: an unconfigured firewall is useless. The level of security provided by a firewall depends on the correctness of the specified rules. Moreover, adopting a firewall must follow three main laws:

- The firewall must be the only point of contact between the internal network and the outside.
- All packets must be blocked except those explicitly "authorized" (according to company policy).
- The firewall itself must be a "secure" system.

The basic components that allow traffic monitoring by a firewall are of two types: i) Screening routers or Packet filtering and Packet-Inspection, and ii) Application gateways. Various configurations for firewalls also exist, such as Dual-homed host, Bastion host, Screened host gateway, among others.

Screening router firewalls are devices capable of blocking or routing packets traveling between the internal and external network, based on their header (packet-filtering) or content (packet-inspection). These devices operate at the network and/or transport layer and are therefore transparent to the application layer.

Firewalls are also subject to attacks aimed at reconfiguring packet-filtering rules. To protect firewalls, any remote configuration functions should be disabled. Other attacks aim to modify internal routing tables, bypassing other firewalls protecting the internal network. Such attacks can be carried out, for example, using fake Internet Control Message Protocol (ICMP) packets—a service protocol for packet-switched networks that transmits information about malfunctions, control information, or messages between network components. Therefore, firewalls must be configured to disable dynamic routing functions.

More generally, firewalls based on *Packet filtering* evaluate packets based on their header (see Figure 10.21). These devices are configurable through an ordered list of rules that allow or deny packet passage based on header information (source IP and port, destination IP and port, TCP flags, IP options, etc.). Many router vendors apply filtering rules only to outbound packets, not inbound ones. In this case, artificially generated packets with a source address of an internal host could be routed into the internal network as if they were legitimately from the spoofed host.

By inspecting inbound packets, the router can detect that a packet with a source address matching an internal host cannot originate from the external network and

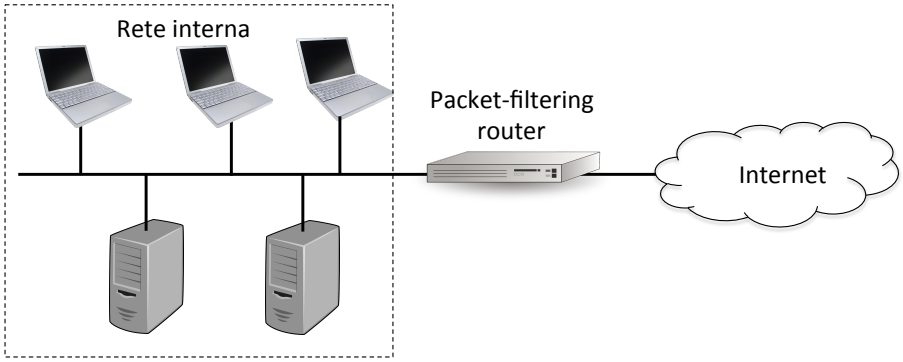


Figure 10.21: Firewall architecture with Packet filtering

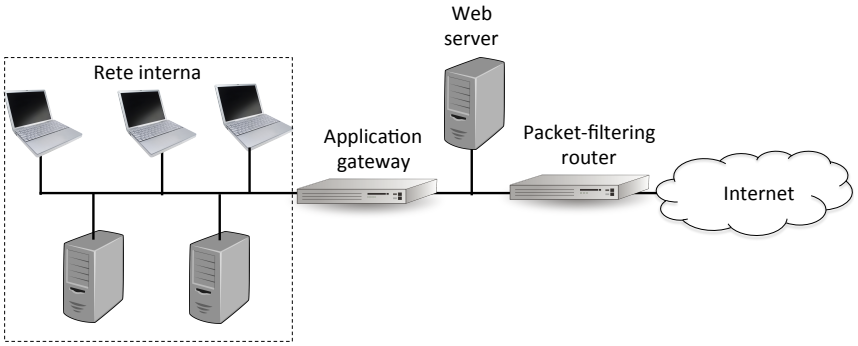


Figure 10.22: Firewall architecture with Application Gateway

should therefore be blocked. The main challenges of this solution are defining a correct and complete set of rules according to the security requirements and identifying potential configuration errors, which can be difficult due to the lack of automated verification tools and the incompleteness of logs generated by the security system.

Packet-inspection firewalls, in particular, evaluate packets based on their payload. These devices understand a predefined set of protocols encapsulated in IP and, in some cases, can monitor the state of related sessions (stateful inspection). Despite their complexity, they offer great flexibility in defining filtering rules (so-called smart rules) that, operating at the TCP session level (session filtering), are comparable to those offered by application gateways. However, unlike application gateways, they do not provide proxy functionality. They do not reconstruct packets but allow only those packets that pass the filtering rules.

Application gateway firewalls currently offer the highest level of control over traffic between an internal network and the Internet. They are generally implemented via a host configured with specific software capable of providing, alongside packet-filtering and packet-inspection services (as in screening routers), so-called

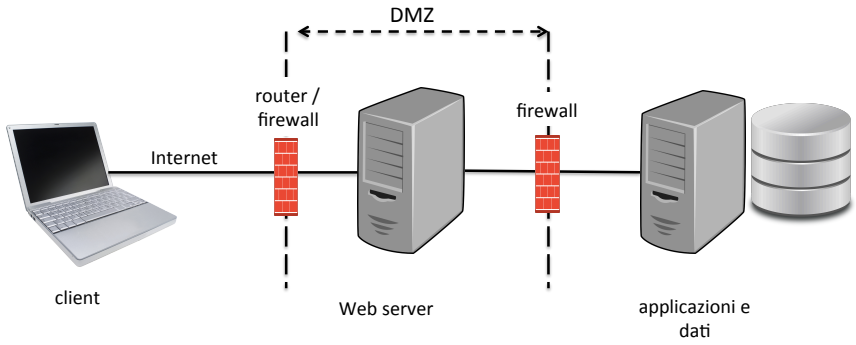


Figure 10.23: Firewall in a 3-tier architecture with web server in DMZ

application proxy services. An application proxy is a process executed on the gateway that operates at the application layer in communications between components of a specific application for which it was designed (see Figure 10.22). Commercial examples of application proxies include traffic control for Telnet, FTP, and HTTP.

For client-server applications, an application proxy communicates with the client by simulating the server, and vice versa, communicates with the server by simulating the client; packets do not travel directly between client and server. The proxy intercepts, interprets at the application level, reconstructs based on protocol knowledge, and routes the packets.

Compared to Packet-inspection routers, Application gateways (or more precisely, the application proxies running on them) have advantages and disadvantages. By reconstructing traffic, an application gateway minimizes the probability of attacks exploiting server weaknesses related to unexpected message sequences. Reconstructing outgoing packets requires analysis time, making application gateways slower than packet-inspection routers. Moreover, operating at the application layer, an application gateway can monitor fewer protocols and requires the installation and configuration of a new proxy for each new protocol.

Two examples of three-tier and five-tier architectures for web-based information systems with firewalls are shown in Figures 10.23 and 10.24. The zone between the firewall receiving traffic from the external network and the downstream firewall filtering traffic toward the internal network is called a *DMZ (DeMilitarized Zone)*. The DMZ is considered a zone exposed to security attacks. The DMZ adds an extra security layer to the external network. External networks can access only the resources exposed in the DMZ, while the rest of the organization's network is behind the second firewall. The DMZ functions as an isolated subnet between the Internet and the private network. The term DMZ comes from "demilitarized zone," an area between nations where military operations are prohibited. Figure 10.24 shows how multiple security levels can be created to protect the different layers of the architecture.

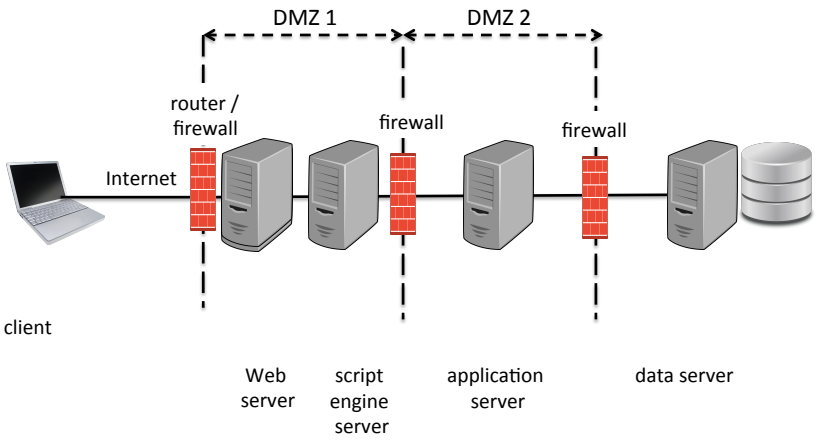


Figure 10.24: 5-tier architecture with three firewalls

10.5.2 Intrusion Detection Systems - IDS

The term *intrusion* refers to a set of actions aimed at compromising one or more security properties (mainly integrity, confidentiality, availability) of a system or, more generally, of computing or network resources. *Intrusion Detection (ID)* is defined as the process of monitoring events of a system or a network, looking for evident traces (e.g., logs) of intrusions [28]. *Intrusion prevention* is the extension of the ID concept with access control aspects aimed at protecting systems from malicious exploitation of resources. *IDS* are systems that automate the Intrusion Detection process, identifying intrusions by unauthorized users (crackers) and authorized users (insider threats).

Sometimes IDS interact with firewalls. For certain intrusions, an IDS can request the closure of active connections, block any other connections from the attacker's IP address, and notify the administrator that a probable attack is underway.

The main assumptions underlying IDS are:

- System activities are observable.
- Normal and intrusive activities leave different evidence.
- From an algorithmic perspective, *Features* capture evidence of intrusions, and *Models* allow combining such evidence.
- From an architectural perspective, various components are required: processors for audit data, a knowledge base, a *decision engine*, and alert and response generators.

Figure 10.25 shows the basic architecture of an IDS.

In detail, based on the schematic architecture of Figure 10.25, an IDS uses:

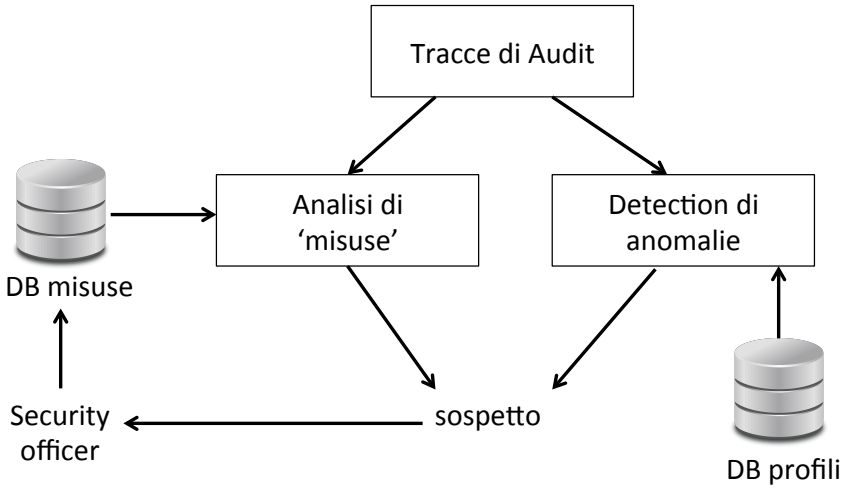


Figure 10.25: Simplified IDS architecture

- A set of *Sensors (Information Sources)*, which collect elements (e.g., log traces) that can reveal traces of intrusion;
- Analysis algorithms, which implement ID based on the events collected by the sensors, determining whether an attack is currently underway or has already occurred;
- *Alerting and Response* components: active components that, when an attack is detected, take actions ranging from reporting alarms to more complex actions (e.g., closing a TCP port).

The personnel responsible for the IDS is the Security Officer, who monitors and consults the misuse database and receives the system's "suspected" alerts (which must be checked manually or under supervision) generated by the analysis and detection algorithms. The profile database contains patterns of normal user behavior: deviations from these patterns indicate possible anomalies.

One possible *classification of IDS* is based on the Information Source:

1. A *Network-based IDS* detects attacks by analyzing network traffic (e.g., packets) to determine if any malicious packets are in transit with the intent of disabling the IDS itself. Such IDS require that sensors be distributed across the network. Advantages include the ability to monitor large networks with few sensors and minimal impact on existing network configuration and performance. Disadvantages include reduced efficiency under network congestion, inability to analyze encrypted protocols, and inability to detect if an attack has succeeded. Some attacks generating anomalous or malformed packets may evade detection.

Examples of network-based IDS tools include Ethereal (an open-source GUI-based packet viewer) and tcpdump, a packet capture tool (www.tcpdump.org) with code and interfaces for multiple operating systems and sensor types.

2. A *Host-based IDS* is software (similar to antivirus software) installed on each machine to protect. This software operates on information collected from individual hosts at the OS or application level. It can analyze attacks with high precision, determining which process and user were involved. These IDS can infer the outcome of attacks. Advantages: can detect attacks not visible to network-based IDS, can operate on encrypted data, and can detect Trojan Horses and software integrity violations. Disadvantages: difficult to manage, especially in terms of scalability; can be disabled by Denial-of-Service attacks; high impact on OS performance.

Another *classification of IDS* is based on the type of analysis (see Figure 10.25):

1. *Misuse detection* analysis aims to identify already known behaviors, i.e., attacks (e.g., via pattern matching). These IDS analyze system activity by comparing it to *signatures* of known attacks. Advantages: detect attacks with few false positives and can quickly identify the use of tools or techniques generating specific attacks. Disadvantages: detect only known attacks, require frequent signature updates, may miss attack variants, and need continuous maintenance to keep up with evolving attack patterns.

2. *Anomaly detection* analysis identifies significant deviations from normal behavior. These IDS detect abnormal host or network behavior, based on the hypothesis that a system under attack behaves differently from normal. They build profiles representing typical user, host, and network behavior using historical data and define aggregated indicators to measure deviation. Thresholds are set for each indicator, and exceeding these thresholds triggers an alert. Advantages: can detect attacks without knowing their full details, and can help define signatures for misuse detection systems. Disadvantages: produce many false positives due to unpredictable user behavior, require extensive training on normal behavior, and cannot provide detailed information about the type of attack detected.

Finally, *Response* refers to the IDS reaction when an attack is detected. Two types of response exist:

- *Active response*: acquiring more information by increasing the sensitivity of information sources, or modifying the environment to block the ongoing attack through connection resets or reconfiguration of routers and firewalls.

- *Passive response*: notifying the system administrator about the ongoing attack and leaving the decision on actions to the operators.

10.6 Elements of Cybersecurity

Cybersecurity applies to the cyberspace, where the term *cyberspace* conventionally refers to the physical and digital environment where activities using the Internet take place. In this space, actions involve both human and computational (e.g., software, robots) actors in an interleaved and often real-time manner, thanks to reduced network access costs and to the availability of broadband connectivity. Cyberspace opens national and transnational markets, making information systems more vulnerable to attacks by those (criminals, hackers, terrorists) seeking to compromise, damage, or exploit them to obtain personal or commercial information fraudulently [26].

The term *cybersecurity* refers to the physical, logical, and organizational measures implemented to defend against cyber attacks. It is now an essential component of the organizational value chain toward clients and stakeholders, for whom organizations have explicit protection obligations.

In cyberspace, malicious events can occur almost instantly and globally, originating from locations physically distant from, or external to the organization. Crimes such as fraud and industrial espionage can now be committed remotely and on a large scale in seconds due to the availability of distributed computing resources. Consequently, improving cyberspace defences is increasingly guiding investment decisions, as organizations cannot afford to forego adequate cyber defence capabilities. The evolution of threats is driven by factors such as the proliferation of devices, remote work, software vulnerabilities, reduced costs, and lower technical barriers to executing complex attacks.

Vulnerabilities of applications and information systems stem from the fact that networks and systems forming the cyberspace were designed with usability and, at most, *resilience* in mind, without incorporating security considerations from the outset. These vulnerabilities are exploited by individuals and groups for illicit purposes, such as stealing sensitive information (e.g., customer lists, patents, strategic data) or sabotaging IT infrastructure, sometimes performing *ransomware* attacks, where malware (malicious software) is used to restrict access to enterprise resources (e.g., a portion of data) until a ransom is paid.

10.6.1 Attacks, Victims, and Countermeasures

Cyber attacks range from fraud and extortion to identity theft, data breaches, espionage, sabotage, and vandalism, sometimes purely emulative.

Potential *victims* include public and private institutions of all sizes, from multinational corporations to small and medium-sized enterprises (SMEs). Small organizations are typically more vulnerable, and the consequences for them are proportionally greater. There is also an informational asymmetry between the victim and the attacker: smaller enterprises face higher costs for protective measures and greater difficulty in mitigating economic and reputational damages deriving from a cyber incident.

A key *countermeasure* is the development of *strategic plans*. Many countries are implementing national strategic plans involving public, private, and research

sectors. These plans include measures against cybercrime, awareness campaigns, and coordination initiatives (organizational measures, training, etc.). Interventions must include regulatory actions to protect victims of cyber attacks, considering the diversity of target organizations. SMEs should be encouraged to strengthen their security culture.

The rise of *Crime-as-a-Service* (CaaS) demonstrates a market for specialized products and services to carry out criminal activities or cyber threats. This model transforms traditional hierarchical criminal organizations into fluid, temporary networks formed around specific projects, often leveraging freelance cyber-criminals. CaaS increases the offensive capabilities of criminal actors lacking technical expertise.

Three aspects of cyberspace require international cooperation:

1. *Cybersecurity*: States must formally commit to combatting cyber threats (e.g., botnets or cyber attacks, potentially causing military escalations if state-sponsored). Public-private partnerships should establish alert structures to receive notifications of attacks.

2. *Internet governance*: Should follow a multi-stakeholder model. States must ensure Internet activities are not controlled solely by private entities. The ICANN[1] should become more transparent, structured, accountable, and inclusive.

3. *Freedom of expression*: Ensured through UN initiatives, with scientific communities promoting safe and widespread Internet awareness to bridge the digital divide. Universal Internet access should be guaranteed as a freedom and as protection of cyberspace as a global common good.

Challenges are not only technological but also social, economic, and organizational.

10.6.2 Zero Trust: A Methodological Approach to Cybersecurity

Faced with rapidly evolving cyber attack methodologies, including AI-based attacks, organizations can adopt a *Zero Trust* (ZT) approach, which is particularly relevant given the increase in remote work due to the Covid-19 pandemic. This model is based on the principle that access to IT applications and systems, whether internal or external, must be granted only when strictly necessary, without implicit trust, and continuously verified.

The ZT approach emerged in 2010 [16] to provide advanced protection against threats from cloud computing, distributed IT, and remote or virtualized work. Previously, organizations viewed assets as residing *inside* a network perimeter. Modern work patterns render the *network perimeter* obsolete. Users and applications can

[1]Internet Corporation for Assigned Names and Numbers – international body managing IP address allocation, root servers, etc.

exist outside traditional protective measures (firewalls, access controls, encryption), and internal resources are no longer inherently trusted.

The Covid-19 pandemic highlighted the necessity of a ZT cybersecurity model based on "trust no one," to counter new cyber attack types and manage remote work security.

Context

According to the *The Global Risk Report 2024* [11] by the World Economic Forum, cybersecurity is one of the major global risks alongside climate change, geopolitical tensions, and technological acceleration. Many organizations acted urgently without fully considering evolving technological risks. Since late 2019, cyber attack scenarios have changed drastically, with criminal enterprises operating as structured companies, with budgets, product development capabilities, and human resources similar to any market-driven organization.

The ENISA (European Union Agency for Cybersecurity) [9] highlights evolving security issues. For example, ransomware demands now focus on corporate reputation rather than data recovery. Attackers demand payment to prevent disclosure of data, which could cause reputational damage exceeding operational disruption. GDPR-regulated data or industrial secrets are particularly at risk.

VPNs (Virtual Private Networks), widely used to support remote work, are now considered risky. Traditionally, VPNs extend a secure private network geographically, assuming trust in users, access methods, and devices. Current realities show that user awareness cannot be assumed, and devices may be unpatched, increasing the *attack surface*. Endpoint security (that protects devices like desktops, laptops, mobile phones, and servers—known as endpoints) is critical to prevent threats at these vulnerable access points.

The Approach

Zero Trust is a military-inspired approach where no entity is trusted by default. Key principles:

1. All entities are untrusted.
2. Access is based on least privilege.
3. Continuous security monitoring.

ZT systems continuously verify user credentials before granting access to digital and physical workspaces. Authentication and authorization occur before data transfer or access, regardless of location. Monitoring includes behavior analysis, filtering, and logging of users, programs, and devices to detect potential attacks. Any deviation triggers heightened scrutiny.

ZT implementations include Zero Trust Architecture, Zero Trust Network Access (ZTNA), and Zero Trust Secure Web Gateway (SWG). ZT is also called "perimeterless protection," implemented through integrated components enforcing:

- Visibility and control of on-premise/cloud environments and IoT devices.
- Network flow control among all resources.
- Identity verification and differentiated cloud access.
- Network and application segmentation.
- Multi-factor authentication (MFA).
- Fine-grained access policies.
- User access based on least privilege.
- Minimal reliance on VPNs and firewalls, as these older hub-and-spoke, castle-and-moat models increase risk in modern distributed environments.

ZT also emphasizes cyber intelligence for corporate executives (CEO, CIO, managers) to assess risk and understand criminal trends. The *deception* methodology simulates espionage: it does not block attacks immediately but identifies attackers' early moves, helping organizations understand *defense evasion* techniques used by criminals.

10.7 Evolution of Threats and Security Techniques

The evolution of attacks and related defences mainly stems from the spread of the concept of cyberspace, a term used to indicate the environment in which operations using the Internet take place. The reduction of network access costs and the development of broadband lead to a continuous expansion of cyberspace. Information systems become more vulnerable to attacks from those (criminals, hackers, terrorists) who intend to compromise, damage, or exploit them to fraudulently obtain personal or commercial information. Moreover, as said earlier, harmful events can occur almost instantaneously, on a global scale, and originate in physically distant or external locations to organizations. Crimes such as fraud and theft of trade secrets can occur remotely and on a large scale within seconds. Therefore, the improvement of cyberspace defences is increasingly driving investments.

Evolution of Threats.

System threats are evolving due to several factors, such as the increased pervasiveness of devices, reduced cost of attacks, and decreased level of IT expertise required to perform attacks of a certain complexity. The victims of cybercrime are entire systems (companies, organizations, public institutions of any size, etc.). Small and medium-sized enterprises are more vulnerable, and for them, the negative consequences are proportionally greater.

There is an information asymmetry between the victim and the attacker, meaning that smaller companies must bear relatively higher costs to implement protection systems and have greater difficulty responding to the economic and reputational damage caused by a cyber violation.

Evolution of Cryptography.

Cryptographic algorithms are also evolving. Some traditional algorithms will likely become obsolete, such as the asymmetric RSA algorithm, which has faced various challenges due to recent advances in quantum computing, particularly due to the emergence of new classes of factorization algorithms, such as Shor's algorithm, which can efficiently factor large integers [15]. Consequently, many organizations are moving to more secure cryptography suitable for post-quantum computing (2030 is considered the critical year). These are usually public-key cryptographic algorithms believed to be secure against cryptanalytic attacks from a quantum computer. In fact, if quantum computers become sufficiently powerful, traditional cryptographic methods like RSA will no longer be secure, and new post-quantum cryptography technologies will be needed. While some researchers focus on building more powerful quantum computers, others are trying to improve the RSA algorithm.

The problem with currently used asymmetric algorithms, such as RSA, is that their security relies on one of three hard but weak mathematical problems: integer factorization, discrete logarithm, and elliptic curve discrete logarithm. These problems can be easily solved on a medium-power quantum computer.

The RSA algorithm, for example, is based on the computational difficulty of factoring the product of two large prime numbers. Classical computers require exponential time to factor large integers, making RSA secure. On the other hand, with quantum computers, factorization can occur in polynomial time, leading to the rapid breaking of RSA (see reference in footnote[2] for an overview).

Most current *symmetric* cryptographic algorithms and hash functions are relatively secure against quantum computer attacks (see the report of the Italian National Cybersecurity Agency[3]). One of the first algorithms theorized for quantum computers is Grover's algorithm, based on the problem of searching for an element in a database. While in classical cryptographic algorithms, for a database of N elements, N operations are required, Grover's algorithm reduces this number to $\sqrt{N}$. From a cryptographic perspective, this corresponds to a quadratic speed-up for all 'brute force' type algorithms. To maintain the current security level against classical (non-quantum) attacks, it is sufficient to double the length of secret keys for symmetric ciphers (even though this increases the data to be exchanged and thus reduces performance). Quantum computer attacks therefore do not currently require a new symmetric cryptography standard.

Research and standardization of post-quantum cryptography (Post Quantum Cryptography - PQC) are ongoing, with several candidate algorithms aimed at protecting information from risks related to advances in quantum computing, while maintaining compatibility with existing infrastructures. Examples of PQC algorithms include lattice-based cryptography, hash-based cryptography, code-based cryptography, and multivariate polynomial cryptography. For a presentation of these algorithms, see [3].

[2] https://www.archonsecure.com/post-quantum-cryptography-guide

[3] https://www.acn.gov.it/portale/documents/20119/85999/ACN_Crittografia_Quantum_Safe.pdf

10.8 Questions

- Explain the mechanism for generating and verifying digital signatures.
- Consider the integrity requirement and discuss techniques to prevent possible attacks violating this property.
- Describe the main types of access control for databases.
- Describe cryptographic techniques: types of algorithms, role of cryptographic keys, applications of cryptography.
- Explain the types of Firewalls.
- Define security properties and indicate which can be guaranteed through cryptography and by which techniques.
- Explain the differences between symmetric and asymmetric cryptography.
- Describe how digital signatures work.
- Digital certificates: explain their functions, the contexts in which they are used, and the infrastructure that allows their issuance and management.
- Explain the functionalities of an IDS and the types of IDS.
- Describe the main security policies for an information system.

Part IV

Information System Design

Chapter 11

The Design of Information Systems

11.1 Information System Management Process

Often, in the development of an application, design takes on a non-systematic character and lacks a clear methodology. This approach is typical in cases where one wants to create a system in a very short time, but it can be dangerous because the construction of the system will proceed through successive adjustments based on user or client requests. Furthermore, such an approach can be difficult and lead to errors if the goal is to achieve interaction with corporate information systems that is controlled, secure, and with consistent management of applications and data. The problems that may arise in systems planned and designed in a non-systematic way range from the risk of system unavailability, due, for example, to unexpected interruptions (crashes), to the impossibility of maintaining the system, to the inability to meet new requirements or manage growth (scalability).

To manage the complexity arising from the various aspects related to the design and management of information systems, the solution is to follow a systematic process. In particular, the activities of planning, design, development, and maintenance of an information system fall within what is defined as the **information system management process** [24, 20, 2]. This process includes not only technological issues, but must also take into account the non-technological elements of the information system, such as the impact that changes to the information system have on the entire organization and on organizational strategies.

The need to design and manage an information system in a structured way, both in its operation and evolution, considering not only technological aspects but also organizational ones, arose from the very beginning when information systems entered companies. Consider, for example, Business System Planning (BSP) proposed by IBM in the 1970s, up to more complete frameworks such as TOGAF[1], whose purpose is to design and manage an Enterprise Architecture.

[1] http://www.opengroup.org/subjectareas/enterprise/togaf

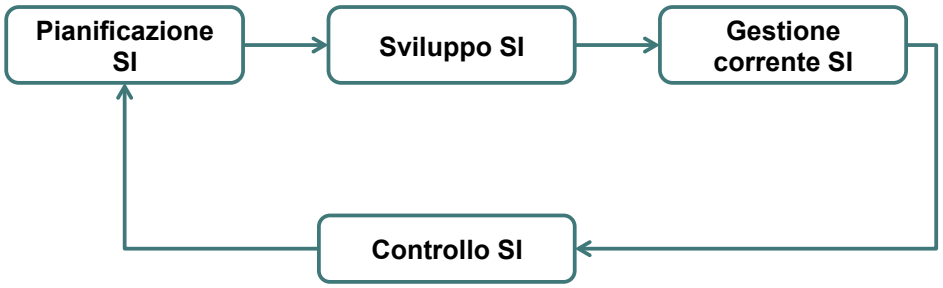


Figure 11.1: Information system management

Regardless of the method used, the management process is itself a process, defined as a set of interrelated activities carried out to achieve a goal: to obtain good design and management of an information system, it is useful to follow a set of operational phases defined and organized in a structured way. In general, beyond the specific method adopted, the management of the information system can be represented by four main phases, organized in a cycle, with a view to continuous improvement, as illustrated in Figure 11.1:

- *Planning*: the phase in which strategic guidelines, the role of organizational components, and operational instructions for implementation are defined.
- *Development*: collection and analysis of requirements, software development, and hardware architecture definition.
- *Current management*: routine operations performed for system maintenance.
- *Control*: operations carried out periodically to evaluate the adequacy of the information system.

The first thing to note is that information system management is not limited to software development alone, but includes other fundamental activities aimed at understanding what needs to be done and how to do it. Furthermore, once implemented, the information system is continuously monitored to analyze the need for changes. The changes that may drive the evolution of the information system may occur due to:

- *Exogenous to the organization*: for example, changes in laws or in the market in which the company operates, which require technological innovations in order to compete with other players in the sector.
- *Endogenous to the organization*: internal factors, which may involve the need for technological renewal due to technological obsolescence or the need for new technologies dictated by strategic choices (for example, the decision to sell new products/offer services online).

Various theories argue that change is necessary for the company to maintain a competitive position within its reference sector. In fact, there are methodologies (for example, ITIL[2]) that help companies identify the phases and steps for change management.

Change management depends on the complexity and impact of the intervention. In fact, interventions on the information system may be:

1. Routine interventions, local to some elements of the system: this category includes hardware maintenance or software updates.

2. Occasional and complex interventions: these refer to interventions that have a significant impact on the existing architecture of the information system. Examples of such interventions include migration to a new technology (for example, cloud or mobile systems, or systems based on sensors and *wearable* sensor technologies) or the addition of new functionalities.

It should be noted that occasional and complex interventions to the information system have radically different connotations and consequences depending on whether a new organization or an existing one is considered. In the first case, there are no pre-existing systems and there are no constraints in the design of the information system dictated by the need to integrate with an already used system. In the second case, the design of new functionalities requires analysis of the existing system and consideration of architectural constraints.

11.2 Planning

The goal of planning is to prepare a project proposal that the client can approve. The roles involved in this phase are: (i) the client, who is the one who approves/funds the project, (ii) the internal or external party/parties (e.g., consultants) to the company who prepare the plan, (iii) the company stakeholders who are those interested in the project (e.g., shareholders, operators, external users, customers, or third parties involved in the value chain).

The planning activity consists of two main activities:

- *Strategic planning*: in this phase, the objectives requiring IT interventions are identified.

- *Feasibility study*: once the need for an IT intervention is established, design alternatives are identified.

11.2.1 Strategic planning

The *strategic planning* phase is based on acquiring knowledge of the company's current situation and identifying the areas and/or processes that require IT interventions. It is a complex and delicate activity that aims to achieve the much-desired alignment between IT and business, according to which the information systems

[2]http://www.itil-officialsite.com/

strategy should be aligned with business objectives. The main activities of strategic planning are:

- Analysis of opportunities for the development of the information system: by analyzing technologies, strategies, and processes, possible developments of the IT system are outlined.
- Analysis of information needs: a more detailed analysis that leads to the modeling of information and the information exchanges necessary for the functioning of the organization.

This phase can be supported by the identification of *business drivers*.

Business drivers can be defined as the key factors for the success of the organization and motivate the design or evolution of the information system by identifying one or more strategic objectives. A relevant aspect is the need to make these objectives measurable, through *indicators* (grouped into Key Performance Indicators - KPI, i.e., indices that monitor their trend), which are defined to be able to assess afterwards whether the expected improvements have been achieved and to identify any critical issues.

Business drivers are distinguished into two categories: *effectiveness* and *efficiency*.

In defining objectives, reference will be made to the model presented in Figures 11.2 and 11.3, characterized by three sets of variables: input, actual output, and expected output. Inputs refer to the resources needed to carry out activities in the company. As already illustrated in Chapter 1, resources can be of various types: material (raw materials), human resources, and immaterial (e.g., information, services, or financial resources). When talking about output, it means the result to be achieved through the execution of a process. The actual output measures the result actually obtained, while the expected output is a predictive measure indicating the goal to be achieved.

Effectiveness measures the company's ability to achieve strategic objectives and can be defined as (Figure 11.2):

$$\text{Effectiveness} = \text{Actual Output} \,/\, \text{Expected Output}$$

where the expected output defines a strategic objective to be achieved, and the actual output measures its achievement.

Examples of effectiveness indicators are as follows:

- Increase market share by 5% by the end of the year.
- Reach new customer categories; appropriate indicators and KPIs must be defined to assess their achievement. For example, by segmenting customers by age group, one may aim to reach the objective of acquiring customers in an age group that did not previously use the product; or reach customers in new locations, in which case an indicator will also need to be defined to measure the result, such as the number of new locations, or the number of customers in new locations.

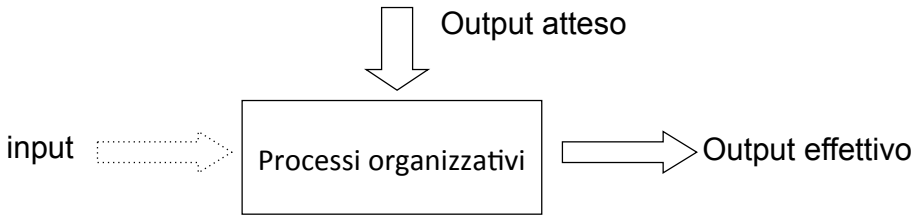


Figure 11.2: Effectiveness

- Reduce the number of complaints.

In evaluating effectiveness, we distinguish two dimensions: *reach* and *richness*.

Reach captures the achievement of an objective that expands the company's scope of activity, from a spatial, temporal, or customer communication channel perspective. The following dimensions of *reach* are defined:

- *Geographic*: for example, from local to national.
- *Temporal*: for example, increasing opening hours or the number of opening days, up to covering 24 hours a day, seven days a week (24/7).
- *Mode (channels)*: multichannel approaches characterize interactions in many categories of services. Interaction can take place through traditional channels (store, counter), but also through different types of online channels such as Web, email, instant messaging, social media (e.g., Facebook). An extension may involve adding social media to traditional online channels (Web, email), which are characterized by very different timing and management methods, as they require constant monitoring of the channel to respond to customer requests promptly.

Richness measures the intensity of communication, i.e., how intense the company's relationships are with the parties it interacts with (e.g., customers and suppliers). To increase the richness dimension, one can act on factors such as the frequency of interactions, the level of interactivity, the level of detail, the type of data used (images, videos, ...), and the personalization of interactions.

The definition of **efficiency** refers to the ratio between actual output and input (see Figure 11.3):

$$\text{Efficiency} = \text{Actual Output} \,/\, \text{Input}$$

where Input is measured in terms of the resources needed to obtain the output.

Efficiency can also be viewed as a strategic objective. An increase in efficiency can be achieved in two ways: obtaining the same result with fewer resources, or increasing output with the same resources. Examples of efficiency improvements are cost or production time reductions with the same resources employed.

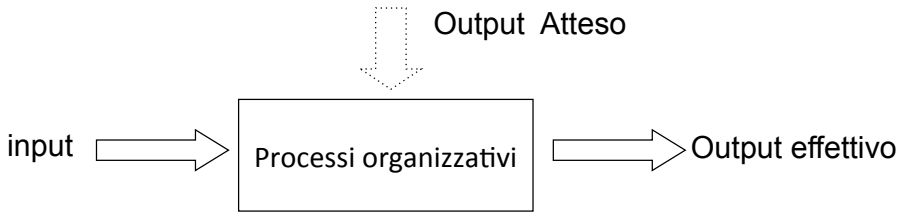


Figure 11.3: Efficiency

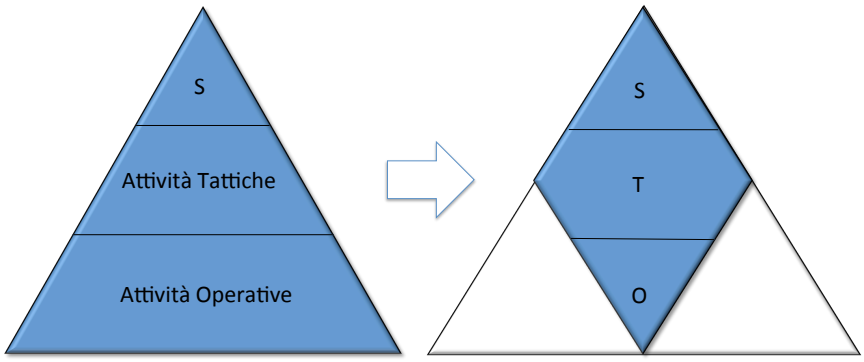


Figure 11.4: Efficiency and impacts on organizational structure

We can better examine efficiency by considering an organization as a three-level structure based on Anthony's model introduced earlier. In a traditional organization, the size of the levels in terms of personnel involved (or *effort*) is shown on the left in Figure 11.4: the operational level is the largest and the strategic level the smallest, considering the pyramid shape. A significant increase in efficiency often translates into automating functions at the operational level: routine tasks are fully automated, and human intervention goes primarily into exception handling, so the size of the operational level is greatly reduced. This can change the shape of the pyramid into a diamond shape as shown on the right in the figure, where the tactical level is the largest. In this case, an important task of the tactical level is to continuously configure the parameters of the automated system that executes operational processes, so that they function as autonomously as possible.

An increase in efficiency may also result in an optimization of information flows between different levels, leading to a reduction of intermediate levels in the organization (Figure 11.5). This will simplify control activities carried out at the operational level and management activities carried out at the managerial level of control and planning, due to the greater availability of updated and aggregated data for decision-making and planning activities.

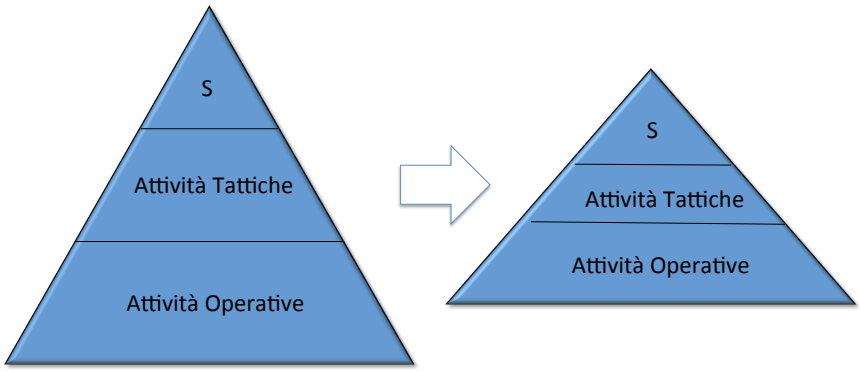


Figure 11.5: Efficiency and impacts on organizational structure – disintermediation between levels

Figure 11.6: Phases of a feasibility study (adapted from [20])

11.2.2 Feasibility study

The *feasibility study* specifies the characteristics of a potential intervention in the information system. Consequently, it focuses on the definition of IT projects. The phases that make up a feasibility study are shown in Figure 11.6.

Definition of objectives and functional specifications

The *Definition of objectives and functional specifications* aims first of all to define the organizational areas that need to be addressed and to outline the improvements to be made for each area considered. Once the scope in which IT projects are to be defined is determined, the next step is drafting the functional requirements: the high-level functionalities that the system should provide are described. Finally, this first activity must specify the constraints to be considered in drafting the projects. These constraints are predictive constraints on non-functional specifications[3]. In

[3]Related to the way in which the functionalities must be provided.

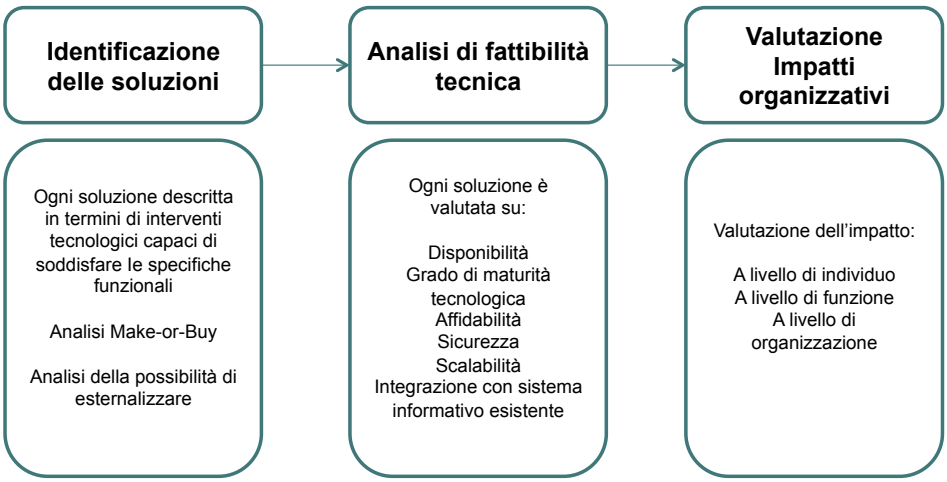


Figure 11.7: Phases of solution design

particular, the focus is on three variables: cost, time, and quality. It should be noted that usually these variables cannot be optimized simultaneously, but only in pairs. Indeed, it is possible to implement low-cost projects in a short time, but at the expense of quality. Alternatively, one may opt for low-cost, high-quality projects, but realizable over long timeframes. Another option is costly IT projects that guarantee high quality and short timelines.

Solution design

The *Solution design* consists of three phases (Figure 11.7):

1. *Identification of solutions*: based on the functional specifications and the constraints defined in the previous phase, IT solutions capable of meeting them are defined. Each solution is described by specifying the technological interventions in detail.

   The analysis of solutions usually involves two types of analysis: Make-or-Buy analysis and outsourcing analysis.

   In the Make-or-Buy analysis, the decision is whether to use off-the-shelf software components available on the market (Buy) or to develop the components within an in-house software development project (Make) (see Section 11.2.3).

   Once it has been defined how the system will be implemented, the next step is analyzing the possibility of outsourcing system management (*sourcing strategy*). Management can be internal (in-house) or external to the company (outsourcing). *IT outsourcing* is the term used to describe the situation in which the implementation or management of IT services is entrusted to an external provider (for details see Section 11.2.4).

2. *Technical feasibility analysis*: in this phase, the possible solutions to meet the needs defined in previous phases are described.

   Each solution must be evaluated to determine its adequacy from a technological point of view. The first evaluation concerns technical characteristics. In particular, architectural quality dimensions such as the following are evaluated:

   (a) Availability: assesses whether the proposed solution can actually be implemented (or acquired on the market) within the planned time.

   (b) Level of technological maturity: evaluates the stage of the life cycle of the technology to be used.

   (c) Reliability: considers the exposure to failures of the identified solution. Reliability refers to a system's ability to ensure continuity in its operation.

   (d) Security: assesses the impact of the identified solution on the security of the information system and whether it requires the adoption of new security mechanisms such as encryption, for example.

   (e) Scalability: evaluates the propensity for system evolution. In particular, it considers the cost in terms of resources (time and money) of expanding the system in a later period.

   Once the technical characteristics have been analyzed, it is necessary to evaluate how the identified solution *integrates with the existing information system* (if present). Often, the existing information system (also called the *legacy system*) imposes architectural constraints on solutions, especially if information flows need to be managed between the existing system and the proposed solution. If the two systems are heterogeneous and incompatible, one must take into account that in addition to the development cost of the solution, there will also be additional costs arising from systems integration. These costs are often significant, and development times non-negligible. The technical feasibility analysis concludes with the evaluation of the solution's alignment with the specified objectives, measuring its adequacy and strategic importance.

3. *Evaluation of organizational impacts*: the introduction of a new IT solution usually has several impacts within a company. It is necessary to assess this impact to identify potential limitations in system feasibility. Organizational impact can be evaluated at the individual level, the functional level, and the organizational level. At the individual level, it is important to consider the reaction of users of the new system. Resistance to change or limited skills can compromise project success. At the functional level, one must evaluate whether the new solution creates the need to redesign roles or redefine tasks. Finally, at the organizational level, one must assess whether the solution implies the need to collaborate with other entities, giving rise to partnerships of various kinds and levels.

Economic feasibility evaluation

With regard to the *economic feasibility evaluation*, the proposed solutions, in addition to being technically feasible, must also be economically viable. In other words, the benefits associated with implementing the solution must outweigh the costs.

The *costs* must be accounted for by considering all resources associated with the solution: hardware, software, and human resources.

Each cost must be correlated with the phase of the solution life cycle in which it occurs and the functional unit to which it is attributed.

The *benefits* are generally quantified by considering only tangible and direct benefits, those that can be easily monetized. These benefits often materialize in reduced operating costs (for example, reduced execution time of automated activities), reduced financial charges (for example, lower inventory levels), or increased revenues (for example, higher sales).

A detailed analysis of these aspects is not covered in this text. For further details, see [18].

11.2.3 Make-or-Buy Analysis

The **Make-or-Buy** analysis takes into consideration the fact that for some interventions there are ready-made software products available on the market: the so-called COTS (Commercial off-the-shelf or commercially available off-the-shelf). The adoption of such software (the *Buy* alternative) is an option compared to the possibility of designing and developing a custom product (the *Make* alternative).

The **Buy** (purchase) alternative has the advantages of being more economical than the Make alternative and of requiring less implementation time.

The **Make** (development) alternative is often preferred because it leads to the creation of a more flexible product that is closer to business needs. There is also the possibility of opting for a hybrid approach in which parts of the solution are acquired according to the Make mode and others according to the Buy mode, in order to maximize the advantages of both approaches and achieve projects that are limited in terms of time and costs, but still able to fully meet business needs.

Figure 11.8 summarizes the main advantages and disadvantages of the Make and Buy solutions.

11.2.4 Analysis of sourcing strategies

The management of an information system can be internal (in-house) or external to the organization (outsourcing).

Outsourcing brings many advantages, especially to small companies that do not have IT expertise in-house. In particular, it leads to (i) an improvement in service quality thanks to the skills and professionalism of the external provider, (ii) the possibility for the company to focus on its core business activities, (iii) a reduction in IT costs, and (iv) a greater propensity for technological updating and evolution.

The main benefits are the possibility of using skills not available within the company and the potential cost savings. The main disadvantages concern the loss of

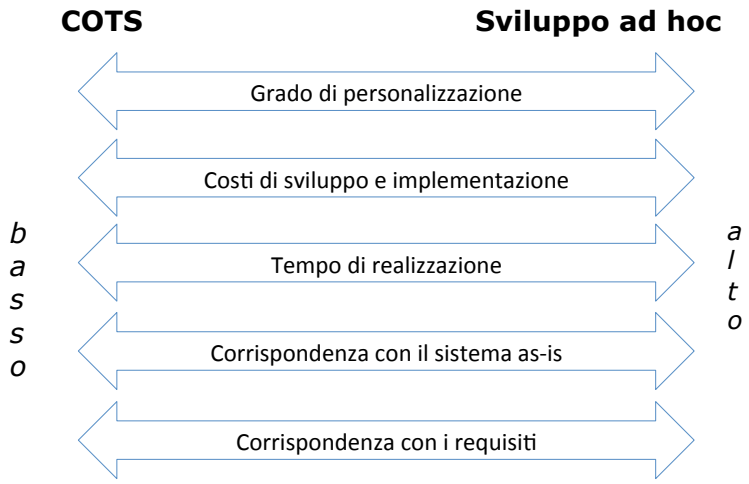


Figure 11.8: Comparison between Make and Buy solutions

internal expertise and dependency on the service provider. Figure 11.9 summarizes the main advantages and disadvantages of outsourcing.

Vantaggi	Svantaggi
Rimanere focalizzati sul core business	Perdita di controllo strategico dell' IT
Disponibilità di competenza altamente specializzate (solitamente non presenti nell'azienda)	Alta dipendenza dall'azienda esterna
Riduzione dei costi	Gap tra la visione dell'azienda e la visione dell'azienda esterna
	Bisogno di gestire le relazioni con l'azienda esterna
	Bisogno di monitorare i livelli di servizio (SLA)

Figure 11.9: Advantages and disadvantages of outsourcing

Focusing on the opportunities for managing the physical infrastructure, different options for outsourcing services can be distinguished, classified in the following levels (Figure 11.10):

- *Level 0*: organizations have no component managed outside the company and manage their systems *in-house*.
- *Level 1*: also called *shared services*, this level refers to the case where the organization is supported by an IT company that works exclusively for the

Tipologia	Livello	Caratteristiche
In house	0	Servizi sono gestiti internamente
Servizi condivisi (Insourcing)	1	Una organizzazione (unità) fornisce servizi IT a tutte le unità
Supporto esterno	2	I servizi sono gestiti internamente, ma con il supporto di aziende specializzate e società di consulenza
Consorzio	3	Un gruppo di organizzazioni con bisogni simili creano un consorzio specializzato per alcuni servizi (e.g., call center, data center)
Outsourcing selettivo	4	Alcuni servizi, solitamente non core, sono gestiti da aziende esterne
Outsourcing completo	5	Tutto l'IT è gestito da una ditta esterna. Se l'infrastruttura appartiene al cliente il servizio è chiamato facility management

Figure 11.10: Levels of outsourcing

organization itself. Thus, one can say that the organization still manages its systems internally. This option is widely used by banks, where the various branches are supported by an IT company that provides services exclusively to the entire group.

- *Level 2*: organizations without IT expertise can also choose the option of *external support*, where IT management takes place internally within the company but with the support of an external organization (e.g., a consulting firm).

- *Level 3*: at this level, some components of the system begin to move outside the organization. The *Consortium* option allows organizations to share the use of certain resources to divide the investment costs. Usually, this option involves sharing very expensive facilities such as call centers or data centers.

- *Level 4*: *Selective outsourcing* is a partial outsourcing in which the management of certain components of the information system is transferred outside the organization. Often, companies decide to outsource only applications that are not critical to the core business (e.g., outsourcing market analysis, simulation tools, or complex processing).

- *Level 5*: *Complete outsourcing*, on the other hand, involves moving the entire management of the information system outside the company.

The adoption of different options often depends on the size of the organization and the presence of IT expertise within it. For example, large companies manage their systems using the options at levels 0, 1, 2, and 3. Medium-sized organizations

also use selective outsourcing and in some cases even complete outsourcing, while small organizations predominantly use complete outsourcing.

Organizations that provide outsourcing services are called *Service Providers*, and traditionally, from the perspective of physical infrastructure, their external support can be further distinguished into:

- *Housing*: leasing to a user a physical space (in racks), where the customer's server is placed. Typically, servers are hosted in Webfarms or Data Centers, where careful management of hardware, software, and infrastructure aspects is ensured.

- *Hosting*: provision of network services (e.g., for websites hosted on a provider's servers).

If the Service Provider offers its services using a Cloud infrastructure, then companies can consider the outsourcing options already discussed for the Cloud:

- *Infrastructure as a Service (IaaS)*: the physical infrastructure is provided as a service by a Cloud Provider.

- *Platform as a Service (PaaS)*: the technological platform is provided as a service by a Cloud Provider.

- *Software as a Service (SaaS)*: access to an entire application is provided as a service by a Cloud Provider.

When an infrastructure is outsourced or external services are used, it is important for the organization to have the necessary guarantees regarding the performance offered by the Service Provider. For this reason, it is necessary to:

- Define service levels (SLA – Service Level Agreement).

- Define KPIs (Key Performance Indicators) for the outsourced service/process.

- Define in detail the *service management mechanisms*:

  - Service description.
  - Service provision.
  - Definition of service levels.
  - Monitoring.
  - Problem management.
  - Reporting.
  - Documentation.

11.3 Information System Development Life Cycle

The feasibility study leads to the choice of the solution by considering different valid options that are feasible both from a technical and organizational point of view, also evaluating the alternatives based on economic convenience.

Once the solution has been chosen, the intervention is carried out through the development activity. The development phase has been the subject of various studies both in the field of information systems and in the field of software engineering.

The phases of a project vary depending on whether a strategy of new software development (Make) or acquisition of components available on the market (Buy) is adopted.

Leaving to software engineering textbooks the detailed discussion of software development life cycles, we emphasize the key role played by EA in planning and developing the information system. In fact, EA makes it possible to define the overall structure of the system and to identify the impact of changes on components at different levels. Once the general structure of the EA has been defined in planning, it remains as a reference for system development in all subsequent phases.

11.3.1 Development cycle with Make strategy

In the case of a development process with the Make alternative, the development cycle is the one represented in Figure 11.11 and consists of a feedback cycle with the following activities: **Detailed planning** of the chosen solution, **Requirements Gathering and Analysis** in order to define the functionalities that the system must provide, **Analysis** of the functionalities to be provided (**Interface Design** and **Detailed Analysis**), **Detailed Design and Implementation** of the aforementioned functionalities. The feedback consists of evaluation activities carried out in the deployment phase (**Deployment & Provisioning**) and system management (**Operations & Management**), aimed at ensuring the alignment of the project and its implementation with the initial requirements and project objectives.

All these phases make up the Software Development Life Cycle (SDLC), and the discipline of software engineering deals, among its various aspects, with providing methods and tools to best support such a life cycle. Historically, the so-called *waterfall* approach required the execution of the various activities that make up the life cycle in a strict sequence. However, this leads to checking the compliance between what has been developed and the client's expectations only at the end of the work. This implies that any discrepancies can only be discovered at the end of development, with heavy economic consequences.

The *iterative* approach aims to solve this type of problem by providing for incremental development of the solution. Starting from initial prototypes with few functionalities followed by further, increasingly complete releases, the client is continuously involved during the life cycle. This allows more frequent verification of compliance between requirements and product and the resolution of any discrepancies before they become too wide.

In recent years, *agile* approaches have been gaining more and more ground, providing for the incremental release of several versions, with less structuring of the

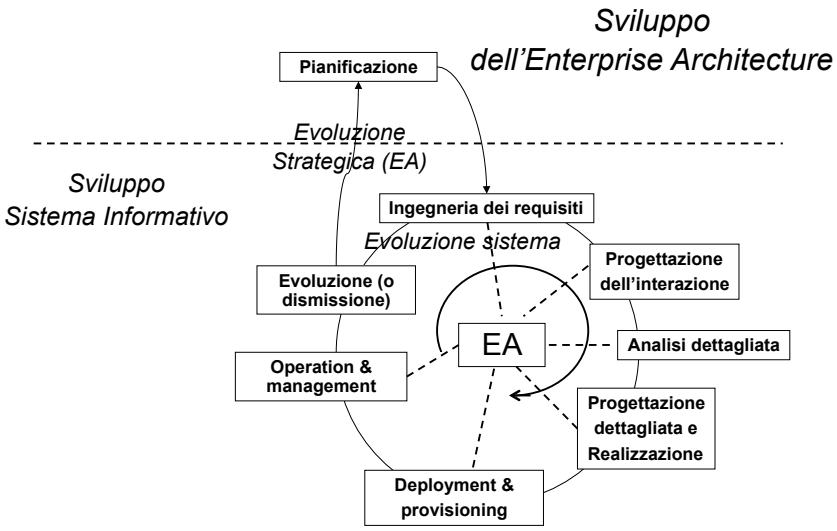


Figure 11.11: The development life cycle of an information system (Make strategy)

various activities.

11.3.2 Development cycle with Buy strategy

Given the availability of numerous COTS software for organizations, it is worth noting that the development process of a Buy alternative has a development cycle slightly different from that of the Make alternative (see Figure 11.12). The development and implementation cycle is preceded by the purchasing cycle, in which the suitable solution is selected according to business requirements. The development cycle then supports the customization, implementation, and integration of the chosen solution.

Software selection

As discussed in the part concerning information system planning, some COTS-type software can be acquired in Buy mode. In this case, the company's difficulty lies mainly in selecting the package most suitable for its needs. In this section, we discuss the variables to be considered in the selection process, which consists of several phases (Figure 11.12).

The first step toward software selection is the activity of requirements gathering and analysis (requirements engineering). As with any design activity, it is necessary to define both functional and non-functional requirements. The former will result in a list of functionalities that the software must provide, while the latter will define properties and architectural constraints.

The requirements guide the search for packages available on the market, which must be evaluated through a series of indicators that allow them to be described

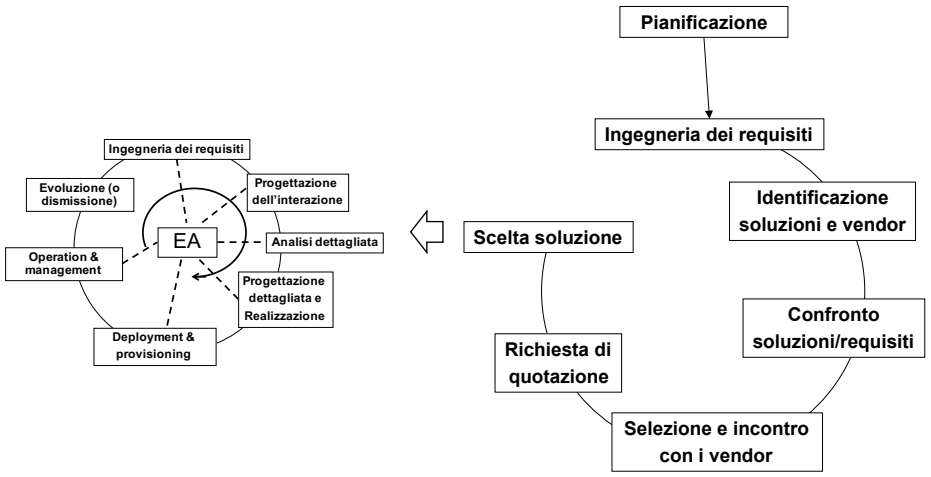


Figure 11.12: Development cycle of an information system (Buy strategy)

and classified with sufficient detail. The indicators that can differentiate the various market solutions are:

- *Functional indicators*: these are the functionalities supported by the different modules that make up the software system under consideration.
- *Architectural indicators*: technological aspects that are critical at the time of choosing the software system and especially for its scalability over time.
- *Solution costs.*

In addition to the characteristics of the solution, it is necessary to evaluate the company that provides it. Thus, the evaluation of the solution must be associated with an evaluation of the package provider and any other associated providers who might be involved in the installation phase (for example, for integration with legacy systems). Figure 11.13 summarizes the various evaluation aspects to be considered (the dashed line in the figure indicates that not only the provider is being evaluated, but also its context).

As far as **Functional indicators** are concerned, the first priority should be given to the *degree of completeness (functional coverage)*. The degree of completeness is a metric that measures the functional completeness of software systems as the ratio between the sum of the functionalities provided by the package and the total number of functionalities required by the user. For example, in a case like the one shown in Figure 11.14, one can see how different packages can provide the same functionality in different ways: in the selection, the solution that best meets the company's needs should be preferred, i.e., the one with the highest degree of coverage.

Another relevant functional indicator is the *Customizability* of the solution, which indicates whether the solution can be adapted to the company's needs through

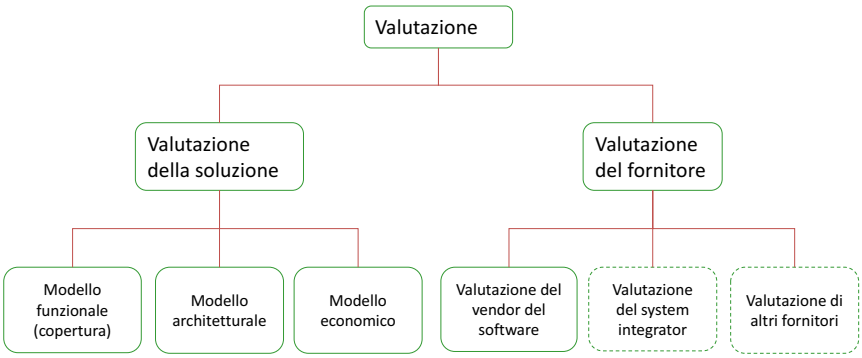


Figure 11.13: Evaluation tree

Processi	Sistema attuale	Obiettivi	Vendor 1	Vendor 2	Vendor 3
Gestione degli ordini del cliente	Solo registrazione su sistema locale; ordini non visibili dal cliente	Portale per cliente Stato ordine visibile Simulare la schedulazione	Portale per cliente	Simulazione della schedulazione	Registrazione degli ordini
Schedulazione produzione	Manuale	Schedulazione a capacità finita	Manuale	Schedulazione a capacità finita	Schedulazione a capacità finita, ottimizzazione delle risorse. Facilmente configurabile
Produzione	Registrazione delle bolle	Rilevazione strumentale e manuale dell' avanzamento	Rilevazione con terminali portatili	Rilevazione manuale dell' avanzamento	Rilevazione strumentale e manuale dell' avanzamento

Figure 11.14: Example of coverage comparison

parameterization and customization, or only through parameterization. Some companies (fortunately few) do not offer the possibility of altering the code, as this would require reprogramming the module according to user requests.

Architectural indicators can be different and depend on the nature of the package. The most common indicators are:

- *Infrastructure scalability*: the methodology must provide guidelines to support the choice of a system that is scalable over time, in order to maintain (or improve) performance as the transactional load and functionalities of the information system increase. The scalability index can be assessed by considering the number of stations manageable by the software, the number of architecture layers, or the way the database is managed (see Fig. 11.15).
- *The degree of interoperability of the solution*: the methodology must help to understand how much the application can be considered an open entity toward the external world.

- *The system security level*: this variable measures the security ensured by the system, both in terms of access control and in terms of protocols (e.g., cryptographic protocols) used for exchanging confidential data.

Variabile	Dimensione	Metrica per il software	Valori Possibili
Scalabilità	**Estensionale**	Numero di stazioni gestibili dal software	∈ **N**
	Intensionale	Numero di moduli installabili separatamente	∈ **N**
		Gestione distribuzione dati	0 = Non distribuita 1 = Distribuita
		Gestione replicazione dei dati	0 = Gestita 1 = Non gestita
		Numero di livelli	∈ **[2, N]**

Figure 11.15: Example of architectural indicators

Once the applications that meet both functional and non-functional requirements have been identified, the reliability of the package providers is assessed. The **credibility** and **capability** of the vendor are assessed considering the experience gained, the national and international presence, organizational reputation, and attention to product evolution over time. Therefore, well-known vendors with a certain territorial coverage and a consolidated reputation, also built on the basis of client companies, are to be preferred. Also not to be underestimated is the frequency of product updates, which indicates attention to the product and its continuous improvement and evolution. System integrators (those who carry out software package integration) and any other suppliers are also evaluated in the same way.

The list of suitable packages is usually reduced following the economic evaluation. Given a certain budget, it is necessary to consider the **solution costs** to verify the feasibility of the investment. The costs of solutions do not include only license costs but other factors must be considered. The costs classified as *project investments* are related to license costs (which depend on functionalities and the number of workstations), hardware costs, human resource costs, and change management costs. In addition, *management costs* must be considered, including maintenance fees, infrastructure costs, customization costs, and software update costs.

As far as ERP systems are concerned, the average numerical composition of the investment is generally divided as follows: consulting (30%), hardware (25%), software development (15%), software products (15%), and personnel training (15%). In addition, an annual cost of about 15% of the initial investment must be considered for operational support, assistance, and maintenance.

The solutions that also pass the economic evaluation are analyzed in detail through demos and meetings with suppliers. Meetings are essential to clarify user requirements, better understand the product, and make the final decision.

11.4 Questions

- Describe the phases of the development process.
- Illustrate the differences between functional and non-functional requirements.
- Discuss the criteria used to decide between selecting software components (Buy approach) and developing new ad hoc software (Make approach).
- Discuss the criteria used to decide between the different options for managing the information system (sourcing strategies).

Bibliography and analytical index

Bibliography

[1] R.N. Anthony. *Planning and Control: a Framework for Analysis*. Cambridge MA: Harvard University Press, 1965.

[2] D. Ardagna, M.G. Fugini, B. Pernici, and P. Plebani. *Sistemi informativi basati su web*. Franco Angeli, Serie Sistemi Informativi, vol. 6, 2006.

[3] Jean-Philippe Aumasson. *Serious cryptography: a practical introduction to modern encryption*. No Starch Press, Inc, 2024.

[4] P. Baltzan and A. Phillips. *Business Driven Information Systems*. McGraw-Hill, 2019.

[5] Scott A. Bernard. *An Introduction To Enterprise Architecture: Third Edition*. AuthorHouse, 2012.

[6] Giampio Bracchi, Chiara Francalanci, and Gianmario Motta. *Sistemi Informativi d'Impresa*. McGraw Hill, 2010.

[7] Cinzia Cappiello, Pierluigi Plebani, and Monica Vitali. Business Process Modeling Notation (BPMN), 2015.

[8] Silvana Castano, Mariagrazia Fugini, Giancarlo Martella, and Pierangela Samarati. *Database Security*. Addison-Wesley & ACM Press, 1995.

[9] ENISA. The goal of cybersecurity certification under the cybersecurity act. https://www.enisa.europa.eu/topics/certification/cybersecurity-certification-framework, 2024.

[10] Lois Evans. Protecting information assets using ISO/IEC security standards. *Information Management*, 50(6):28, 2016.

[11] World Economic Forum. Global Risks Report 2024. https://www.weforum.org/publications/global-risks-report-2024/, 2024.

[12] Mariagrazia Fugini, Fabrizio Maio, and Pierluigi Plebani. *Sicurezza dei Sistemi Informatici*. Apogeo, 2001.

[13] Gerben Wierda. *Mastering ArchiMate - Edition 3.1*. R&A, 2021.

[14] Paul Grefen. *Beyond E-Business: Towards networked structures*. Routledge, 2016.

[15] Matthew Hayward. Quantum computing and shor's algorithm. *Sydney: Macquarie University Mathematics Department*, 1, 2008.

[16] ISO/IEC/IEEE. Iso/iec/ieee 42010:2011 - system and software engineering: architecture description. IEEE, December 2011.

[17] Marc Lankhorst. *Enterprise Architecture at Work: Modelling, Communication and Analysis*. Springer, 2016.

[18] Kenneth C. Laudon and Jane P. Laudon. *Management Information Systems: Managing the Digital Firm, 15th Edition*. Pearson, 2017.

[19] Maurizio Pighin and Anna Marzona. *Sistemi informativi aziendali: struttura e processi*. Pearson, 2011.

[20] Federico Pigni, Aurelio Ravarini, and Donatella Sciuto. *Sistemi per la gestione dell'informazione*. Apogeo, 2009.

[21] M. Porter. *Competitive Advantage: Creating and Sustaining Superior Performance*. Free Press, 1985.

[22] Pierangela Samarati and Ravi Sandhu. *Database Security Volume X: Status and prospects*. Springer, 2016.

[23] Signavio. BPMN editor, http://academic.signavio.com.

[24] Craig Van Slyke and France Belanger. *Information Systems for Business: An Experiential Approach*. John Wiley & Sons Inc, 2011.

[25] J.F. Sowa and J.A. Zachman. Extending and formalizing the information systems architecture. *IBM Systems Journal*, 31(3):590–616, 1992.

[26] William Stallings. ICAM: A foundation for trusted identities in cyberspace. *IT Professional*, 18(1):26–33, 2016.

[27] The Open Group. *The ArchiMate Enterprise Architecture Modeling Language, 3.0*. The Open Group, 2016.

[28] HA Madni Uppal, Memoona Javed, and MJ Arshad. An overview of intrusion detection system (IDS) along with its commonly used techniques and classifications. *International Journal of Computer Science and Telecommunications*, 5(2):20–24, 2014.

[29] Wil Van Der Aalst. *Data science in action*. Springer, 2016.

[30] Wil MP van der Aalst and Josep Carmona. *Process Mining Handbook*. Springer Nature, 2022.

[31] Carlo Vercellis. *Business Intelligence: modelli matematici e sistemi per le decisioni*. McGraw-Hill, 2006.

[32] Mathias Weske. *Business Process Management - Concepts, Languages, Architectures, 2nd Edition*. Springer, 2012.

[33] John A. Zachman. A framework for information systems architecture. *IBM Syst. J.*, 26(3):276–292, September 1987.

Index

Sales Force Automation (SFA), 73

accuratezza, 93
agente, 136
alberi di decisione, 93
albero di decisione binario, 94
albero di decisione generale, 94
albero di decisione multivariato, 94
albero di decisione univariato, 94
Algoritmi a chiave asimmetrica (o a chiave pubblica), 139
Algoritmi a chiave simmetrica (o a chiave segreta), 139
Anthony's Pyramid, 8
application logic - A, 62
application portfolio, 55
APS - Advanced Planning and Scheduling, 57
Archi, 31
ArchiMate, 29
as-is, 26
association rules, 92
attacchi, 137
autenticazione, 136
autenticità, 136
authentication, 149

benefits, 184
BI, *see also* Business Intelligence
BPMN (Business Process Modeling Notation), 44
BPMS - Business Process Management Systems, 58
Business Intelligence, 55, 77
business process, 3, 43
business rules, 65
business service, 33

Certification Authority - CA, 148
classificazione, 93
cloning, 127
cloud computing, 132, 134
Cloud Computing deployment models, 133
cluster, 96
clustering, 96
community cloud, 134
components (elements) in ArchiMate, 32
confidence, 92
conformance checking, 100
costs, 184
COTS, 57, 63, 67, 184, 189
CRM, 57, 66, 69, 98
Customer Relationship Management, *see also* CRM
Cyberspace, 168

data access - D, 62
data mart, 81
data mining, 75, 90
data warehouse, 55, 72, 75, 78, 80
DBMS, 14, 57, 61
Denial of Service - DoS, 137
dice, 89
digital signature, 145
Dimensional Fact Model (DFM), 83
Discretionary Access Control (DAC), 154
disponibilità, 136
drill-down, 87
DW, *see also* data warehouse

E-Business, 57
EB, *see also* E-Business
effectiveness, 178
efficiency, 178
elasticity, 127

enhancement, 100
Enterprise Architecture - EA, 16, 23, 175
Enterprise Resource Planning, *see also* ERP
entropia, 94
ERP, 57, 63
ETL, 82, 89

falso negativo, 95
falso positivo, 95
FASMI, 80
feasibility study, 177, 181
firewall, 160
framework di Zachman, 23
funzione di somiglianza, 97

hash functions, 144
HOLAP, 85
Human Resource, 10
hybrid cloud, 134

IDS (Intrusion Detection System), 164
indicators, 178
information system management, 175
Infrastructure as a Service, 133
integrità, 135
interpretabilità, 93

k-means, 96
key management, 147
Key Performance Indicator (KPI), 178

layer, 61
layers in ArchiMate, 29
legacy system, 62, 183
life cycle, 188

Make-or-Buy, 57, 184
  Buy, 63, 184, 189
  Make, 184
Mandatory Access Control (MAC), 155
Manifacturing Execution Systems, *see also* MES
MES, 57
metadata, 83
minacce, 136
MOLAP, 84
Multi Factor Authentication (MFA), 150

objectives, 181
OLAP (On-line Analytical Processing), 14, 78, 80
OLTP (On-line Transaction Processing), 14, 78
One Time Password (OTP), 139, 151
Outsourcing, 187
outsourcing, 182

partitioning, 127
Platform as a Service, 133
PLM (Product Lifecycle Management), 66
policy, 139
Porter
  primary activities, 9
  support activities, 9
  value chain, 9
Precision, 96
presentation - P, 61
privacy, 136
private cloud, 134
process, 8
process discovery, 99
process mining, 99
proprietà di sicurezza, 135
public cloud, 134
Public Key Certificate - PKC, 147
Public Key Infrastructure - PKI, 147

QoS - Quality of Service, 114
quality
  availability, 183
  reliability, 183
  scalability, 175, 183
  security, 183
  technological maturity, 183
quality dimensions, 183

RACS - Reliable Array of Cloned Services, 127
RAPS - Reliable Array of Partitioned Services, 128
Recall, 96
relationships in ArchiMate, 32

requirements pull, 18, 21
resource, 6
REST - Representational state transfer, 115
reti Bayesiane, 93
reti neurali, 93
riservatezza, 136
robustezza, 93
ROLAP, 85
roll-up, 87
Rule Engine, 58

SaaS, 134
scalability, 126, 127
scalabilità, 93
SCM (Supply Chain Management), 66, 68
security policies, 153
server farm, 127
service orchestration, 116
service registry, 115
services, 29, 58, 113
shared-disk, 127
shared-nothing, 127
Signavio, 47
single tiered, 121
SLA - Service Level Agreement, 114
slice, 89
snowflake schema, 85
SOA - Service Oriented Architecture, 113, 115
SOAP - Service Oriented Architecture Protocol, 115
Software as a Service, 133
solution design, 182
stakeholder, 22
star schema, 85
strategic planning, 177
supervised learning, 90
support, 92

technical feasibility, 183
technology push, 18, 21
test set, 93
three tiered (3-Tiers), 122
tier, 121
to-be, 26
training set, 93
transaction, 14
two tiered (2-Tiers), 122

UIS - User Interface Systems, 58
unsupervised learning, 91
utility computing, 132

velocità, 93
vero negativo, 95
vero positivo, 95
violazioni, 136
virtual machine - VM, 130
virtualization, 129

Zero Trust, 168